



Python Programming Language I

CSC241

Table of Contents

Module One: Introduction to Python Programming	1
Unit 1 — Overview of Python	2
Unit 2 — Setting Up Python	8
Module Two: Python Basics and Syntax	14
Unit 1 — Python Language Syntax	15
Unit 2 — Python Variables and Data Types	28
Unit 3 — Python Operators	40
Module Three: Control Flow and Data Structures	65
Unit 1 — Decision Making In Python	66
Unit 2 — Repetitive Structure: Loops And Branching In Python	83
Unit 3 — Python Data Structures	94
Unit 4 — Python Data Structures: Sets And Dictionaries	115
Module Four: Functions, Modules and File Handling	130
Unit 1 — Functions In Python	131
Unit 2 — Modules And Packages	144
Unit 3 — File Handling In Python	153
Unit 4 — Exception Handling In Python	161
Module Five: Python Database and GUI Development	168
Unit 1 — Python and Databases	169
Unit 2 — Python GUI Programming	188
Answers To Self Assessment Questions	203
Module One	204
Module Two	205
Module Three	208
Module Four	211

Module **Five**

215

GLOSSARY

217

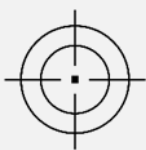
Module One

Introduction to Python Programming

Unit 1 — Overview of Python

INTRODUCTION

In this unit, you will learn about the fundamentals of the Python programming language, its applications across various industries, and its importance in modern technology.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the key features and uses of Python.
- Identify Python's applications in web development, data science, AI, automation, cybersecurity, and other fields.
- Describe Python's role in creating real-world applications such as web applications, machine learning models, and desktop GUIs

1.1.1 Introduction to the Python Programming Language

Python is a high-level, interpreted, general-purpose programming language that is powerful, flexible, easy to learn, and it is widely used across various industries for software development. Guido van Rossum created Python and released it in 1991. Python emphasises code readability and allows developers to write programs with fewer lines of code compared to other programming languages.

1.1.2 Uses of the Python Programming Language

Developers use Python in many areas of modern technology because it is easy to learn and can solve different types of problems. Below are some key areas where Python is commonly used:

1. Web Development

Developers use Python for building dynamic websites and web applications. Popular web frameworks include:

- Django: A high-level framework for rapid development
- Flask: A lightweight and flexible web framework
- FastAPI: Used for building modern and high-performance APIs

Examples: Instagram, YouTube, Spotify, Pinterest

2. Data Science & Machine Learning

Data scientists widely use Python for data science and machine learning because of its rich ecosystem of libraries:

- Pandas: Data manipulation and analysis
- NumPy: Numerical computing
- scikit-learn: Machine learning algorithms
- TensorFlow & PyTorch: Deep learning frameworks

- Matplotlib & Seaborn: Data visualisation

Examples: Predictive analytics, recommendation systems (Netflix, Amazon), fraud detection

3. Artificial Intelligence (AI) & Robotics

Developers use Python in AI-driven applications such as:

- Natural Language Processing (NLP): Chatbots, voice assistants (Siri, Alexa)
- Computer Vision: Face recognition, image processing (OpenCV)
- Robotics: Automation and self-driving cars (ROS - Robot Operating System)

Examples: Google Assistant, Tesla's Autopilot, IBM Watson

4. Automation & Scripting

Developers use Python to automate repetitive tasks, such as:

- Web Scraping: Extracting data from websites (BeautifulSoup, Scrapy)
- Task Automation: Automating file handling, emails, and reports
- DevOps Automation: Writing deployment scripts (Ansible, Docker)

Examples: Automating Excel reports, form-filling bots

5. Cybersecurity & Ethical Hacking

Security professionals use Python to build tools for penetration testing, malware analysis, and network security.

- Popular tools: Metasploit, Scapy, PyCrypto
- Uses: Password cracking, vulnerability scanning, network security
- Examples: Ethical hacking, cybersecurity analysis

6. Game Development

Developers use Python to develop simple and complex video games.

- Pygame: 2D game development
- Panda3D: 3D game development
- Unity (via Python scripting): Game design

Examples: Civilisation IV, Battlefield 2, Disney's Toontown Online

7. Embedded Systems & IoT (Internet of Things)

Engineers use Python in embedded systems and IoT applications.

- MicroPython & Raspberry Pi: Used in hardware programming
- IoT Development: Smart home automation, IoT security

Examples: Smart devices, automated home systems, industrial sensors

8. Cloud Computing & DevOps

Developers use Python in cloud computing and DevOps automation, including:

- AWS, Google Cloud, Microsoft Azure: Cloud-based applications
- Automation & Deployment: Infrastructure as Code (IaC) using Python scripts

Examples: Serverless computing, cloud automation

9. Finance & FinTech

Financial analysts use Python for risk analysis, trading algorithms, and fraud detection.

- Quantitative Finance: Data analysis for stock market predictions
- Cryptocurrency & Blockchain: Bitcoin trading, smart contracts

Examples: Stock market analysis, cryptocurrency exchanges

10. Desktop GUI Applications

Developers use Python to create desktop applications with GUI frameworks like:

- Tkinter: Built-in GUI toolkit
- PyQt & PySide: Advanced GUI applications
- Kivy: Cross-platform mobile/desktop applications

Examples: File management systems, media players

Developers use Python in many real-world applications and for software development, such as:

- Building desktop applications
- Developing mobile applications
- Writing system administration scripts
- Creating scalable web applications

1.1.3 Characteristics of the Python Programming Language

Several key features that make Python popular among developers include:

- Easy to Learn and Use: Simple and readable syntax
- Interpreted Language: No need for compilation before execution
- Cross-Platform Compatibility: Runs on Windows, macOS, and Linux
- Object-Oriented and Functional Programming Support: Supports multiple paradigms
- Rich Standard Library: Built-in modules and libraries for various tasks
- Dynamically Typed: No need to declare variable types
- Extensive Community Support: Large global community for troubleshooting and support

1.1.4 Implementations of Python

Python has multiple implementations, including:

1. CPython: The most widely used implementation, written in C
2. Jython: A Python implementation that runs on the Java Virtual Machine (JVM)
3. PyPy: A fast alternative implementation of Python with a JIT compiler
4. IronPython: A Python implementation that integrates with Microsoft's .NET framework

1.1.5 Python Career Opportunities

Python's widespread adoption creates many career paths, such as:

1. Python Developer: Web and software development
2. Data Scientist: Data analysis and machine learning
3. Automation Engineer: Writing automation scripts
4. Cybersecurity Expert: Security scripting and penetration testing

5. Game Developer: Building games using Python frameworks
6. Cloud Computing Engineer: Working with cloud platforms like AWS, Google Cloud

In-Text Questions (ITQs)

1. What does it mean that Python is a “high-level” programming language?
2. Why is Python popular in data science and machine learning?
3. What does it mean that Python is “dynamically typed”?

In-Text Answers (ITAs)

1. A high-level programming language is designed to be easy for humans to read and write. It abstracts complex machine-level operations, allowing developers to focus more on problem-solving rather than hardware details.
2. Python is popular in data science and machine learning because it has powerful libraries like Pandas, NumPy, scikit-learn, TensorFlow, and PyTorch that simplify data analysis, model building, and algorithm implementation.
3. A dynamically typed language means you don’t need to declare variable types before using them. Python automatically detects the data type based on the assigned value

1.1.6 Conclusion

In this unit, you have learned about the fundamentals of Python programming, its wide range of applications, and its importance in modern technology. Python’s simplicity, versatility, and readability make it essential in fields like web development, data science, AI, automation, and more. These foundational skills prepare you to explore Python’s vast potential and apply it effectively in real-world contexts.

Summary

In this unit, you have learnt that Python is a versatile and widely-used programming language known for its simplicity and readability. It powers applications across

diverse sectors, including web development, data science, AI, automation, cybersecurity, game development, and more.

Self-Assessment Questions

1. What are the primary characteristics of Python that make it a preferred choice for beginners?
2. Name three popular web frameworks in Python and their applications.
3. How is Python used in data science and machine learning?

Unit Activities

1. Explore the official Python website (python.org) and review the latest updates and community resources. (20 minutes)
2. Install Python on your computer and practice writing basic programs using IDEs like PyCharm or Jupyter Notebook. (40 minutes)

Tutor-Marked Assignment

1. List and briefly explain four different areas where Python are applied in the real world. Provide one example of each application.

References/Further Readings

Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Unit 2 — Setting Up Python

INTRODUCTION

In this unit, you will learn about the process of setting up Python on different operating systems, including Windows, macOS, and Linux. You will also explore how to verify Python installation, install Jupyter Notebook, and write your first Python program to confirm that your Python environment is correctly configured.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Differentiate between Python 2.x and Python 3.x.
- Install Python on Windows, macOS, and Linux operating systems.
- Verify Python installation and check the installed version.
- Install and launch Jupyter Notebook using the command line.
- Write and execute a simple Python program.

1.2.1 Setting Up Your Programming Environment

Before writing and running Python programs, you need to set up a proper programming environment. This section covers installing Python, choosing an IDE, and preparing your system for efficient coding.

Python Versions

Python has two main versions:

- **Python 2.x (Deprecated)** – The Python community discontinued Python 2.x after January 1, 2020.
- **Python 3.x (Recommended)** – The Python Software Foundation currently maintains Python 3.13.2 as the latest stable release.

Latest Stable Version: the latest stable release of Python is Python 3.13.2 (always check the official Python website for the latest release: <https://www.python.org>)

Installing Python in Various Operating Systems

1. Installing Python on Windows

- Download Python:
 - Visit <https://www.python.org/downloads/>
 - Download the latest Python Windows installer (preferably 64-bit)

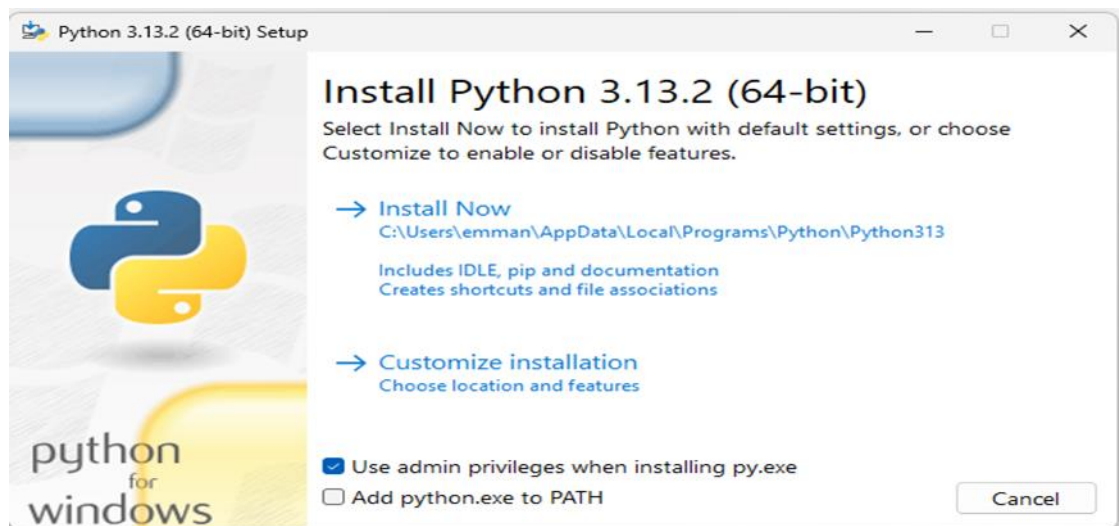


Figure 1: Python Installation

- Install Python:
 - Run the downloaded installer.
 - Check the box "Add Python to PATH" before clicking "Install Now".
 - Follow the installation steps.

To check if Python is installed on Windows and verify the version, do the following:

- Open Command Prompt (cmd).
- Type either of the following commands and press Enter:
 - `python --version` or `python3 --version`

```
Microsoft Windows [Version 10.0.22631.5039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\emman>python --version
Python 3.10.11
```

If Python is installed, the system displays the installed Python version.

2. Installing Python on macOS

- Open Terminal and run:
- `brew install python`
 - (Ensure Homebrew is installed before running this command.)
- Verify the installation: `python3 --version`

3. Installing Python on Linux

- For Debian-based Linux (Ubuntu, Debian, etc.), run:
 - `sudo apt update`
 - `sudo apt install python3 python3-pip`
- For Fedora-based Linux (Fedora, CentOS, etc.), run:
 - `sudo dnf install python3`

Installing Jupyter Notebook

Jupyter Notebook is an interactive web-based environment used for writing and running Python code.

Installing Jupyter Notebook

- Open Command Prompt.
- Install Jupyter` using pip:
 - `pip install notebook`
- Wait for the installation to complete

How to Open Jupyter Notebook via Command Line

After installation, open Command Prompt and type:

```
jupyter notebook
```

This command opens Jupyter Notebook in your web browser.

Writing a Simple “Hello, World” Code to Confirm Python is Working

To check if Python is running correctly, open Command Prompt or Jupyter Notebook and type:

Using Command Prompt (cmd) or Terminal:

- Open cmd/terminal.
- Type: python
 - This will open the Python interactive shell.
- Type the following command:
 - `print("Hello World!")`

```
Microsoft Windows [Version 10.0.22631.5039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\emman>python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World!")
Hello World!
>>>
```

Figure 2: Python command line interface

Using Jupyter Notebook:

- Open Jupyter Notebook.
- Create a new notebook.
- In a cell, type:
 - `print("Hello, World!")`
- Press Shift + Enter to run the code.

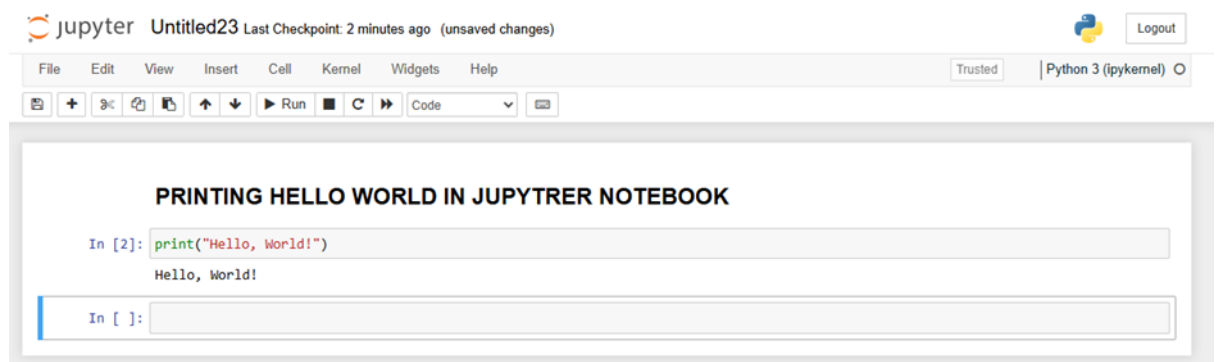


Figure 3: Jupyter notebook interface

In-Text Questions (ITQs)

1. Why is Python 3.x recommended over Python 2.x?
2. What command can you use on Windows to check if Python is properly installed?
3. What does the command Jupyter notebook do?

In-Text Answers (ITAs)

1. Python 3.x is recommended because it is actively maintained, has new features, and Python 2.x is no longer supported after January 1, 2020.
2. You can use the command `python --version` or `python3 --version` in the Command Prompt to check if Python is installed.
3. The command `Jupyter notebook` opens the Jupyter Notebook interface in your web browser, allowing you to write and run Python code interactively.

1.2.2 Conclusion

In this unit, you learned how to set up Python on different operating systems, verify your installation, and configure a basic programming environment. You also practised installing and launching Jupyter Notebook and running a simple Python program to confirm your setup, laying a foundation for confident Python programming.

Summary

In this unit, you learned that Python 3.x is the recommended version, as Python 2.x is no longer supported. You explored installing Python on Windows, macOS, and Linux, verifying the installation via the command line, and setting up Jupyter Notebook. Finally, you practised running a simple Python program to confirm your setup.

Self-Assessment Questions

1. What is the recommended version of Python for current development?
2. How can you check if Python is installed on your Windows system?
3. What command is used to install Jupyter Notebook via the command line?

4. What is the purpose of writing the "Hello, World!" program after Python installation?

Unit Activities

1. Download and install Python: Visit the official Python website, download the latest version for your operating system, install it, and verify the installation via the command line. (20 minutes)
2. Install and Launch Jupyter Notebook: Install Jupyter Notebook using pip install notebook and open it from the command line with jupyter notebook. (10 minutes)
3. Write and Run a Python Program: Create a simple Python program in Jupyter Notebook, for example: print ("Python is working!"), and execute it to verify everything is working. (10 minutes)

Tutor-Marked Assignment

1. List three operating systems where Python can be installed and briefly describe the steps to install Python on each.

References/Further Readings

Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

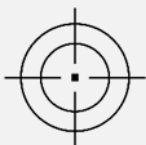
Module Two

Python Basics and Syntax

Unit 1 — Python Language Syntax

INTRODUCTION

In this unit, you will learn about the foundational syntax of the Python programming language. You will explore the two primary modes of Python programming, such as interactive and script mode. You will also understand how to create and run Python programs. Additionally, you will learn about displaying output using the `print ()` function, writing comments, using proper indentation, and understanding identifiers, keywords, string handling, and debugging.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Differentiate between interactive and script modes in Python.
- Create and run Python programs in both modes.
- Use print statements, comments, and string quotations.
- Identify valid identifiers and keywords and apply indentation.
- Understand debugging and types of errors in Python

2.1.1 Modes of Programming in Python

When you install Python, the installation includes the Python interpreter.

The interpreter is a program that reads Python statements and executes them. (Sometimes, it is simply called the interpreter.)

Python supports two primary modes of programming:

1. Interactive Mode (REPL - Read, Evaluate, Print, Loop)
2. Script Mode

Interactive Mode

Definition: Interactive mode allows executing Python commands one at a time in the Python shell (REPL).

In interactive mode, the interpreter waits for Python statements to be typed. After a statement is entered, the interpreter executes it and then waits for the next statement.

NOTE: When the Python interpreter is running in interactive mode, it is commonly called the Python shell.

How to Open Interactive Mode

- Open Command Prompt (Windows) or Terminal (Mac/Linux).
- Type `python` or `python3` and press Enter.
- You will see the Python prompt: `>>>`
- Now, you can type Python commands one by one.

```
Microsoft Windows [Version 10.0.22631.5039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\emman>python
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Figure 4: Interactive mode

The `>>>` symbol is a prompt that shows the interpreter is ready to receive a Python statement. One of the simplest actions in Python is printing a message on the screen.

For Example:

```
>>> print ("Hello, World!")
```

```
Hello, World!
```

```
>>>
```

After the message is displayed, the `>>>` prompt appears again, showing that the interpreter is ready for the next statement. In the following example, two statements are entered:

```
>>> print ("Hello, World!")
```

```
Hello, World!
```

```
>>> print ("Python Programming is Fun!")
```

```
Python Programming is Fun!
```

```
>>>
```

If an incorrect statement is entered in interactive mode, Python displays an error message, providing instant feedback and making it useful for learning.

To exit: press Ctrl+Z then Enter (Windows) or Ctrl+D (Mac/Linux/UNIX).

Advantages:

- Good for quick testing of small code snippets.
- Provides instant feedback.

Disadvantages:

- Code is not saved for future use.
- Not suitable for writing long programs

Script Mode

Definition: In script mode, Python code is written in a file (with a .py extension) and run as a complete program.

Interactive mode lets you test code quickly, but the statements aren't saved. To keep the code, write it in a file and run it in script mode, where the interpreter reads and executes each statement from the file.

How to Create and Run a Script:

1. Open a text editor (VS Code, Notepad++, PyCharm, or IDLE).
2. Write Python code and save it as example.py.
3. Open Command Prompt/Terminal, navigate to the file location, and run: `python example.py`

Example (example.py file):

```
# This is a Python script  
  
print ("Hello, World!")
```

NOTE: You can use a word processor to create a Python program, but you must save it as a plain text file. Otherwise, the Python interpreter cannot read the program.

Advantages:

- Allows writing and saving complex programs.
- Better for debugging and reusability.

Disadvantages:

- Slower testing compared to interactive mode

Creating and Running Python Scripts with the Print Function

A Python program file is simply a text file with a .py extension.

Steps to Create a Python File:

- Open Notepad/VS Code/PyCharm/IDLE.
- Type some Python code (e.g., `print("Welcome to Python!")`).
- Save the file as `my_script.py`.
- Run the script using: `python my_script.py`

Displaying Output with the print Function

The print function in Python displays output on the screen. It is one of Python's built-in functions used to show results or messages.:

```
Print ("Hello, World!")
```

When executed, it displays:

Hello, World!

To display data, type print followed by parentheses containing the data (argument) you want to show. The quotation marks are not shown in the output; they only mark the start and end of the text.

2.1.2 Python Identifiers and Keywords

Python Identifiers

Definition: Identifiers are the names used for variables, functions, classes, and objects in Python. Identifiers consist of letters and digits, in any order, except that the first character must be a letter.

Rules for Naming Identifiers:

- Must begin with a letter (A-Z or a-z) or an underscore (_).
- Can contain letters, digits (0-9), or underscores.
- Case-sensitive (name and Name are different).
- Cannot use Python keywords as identifiers.

Examples of Valid Identifiers:

```
my variable = 10
```

```
_number1 = 5
```

```
CamelCase Example = "Python"
```

Examples of Invalid Identifiers (Will cause errors):

```
2variable = 10    # Cannot start with a number
```

```
my-variable = 5   # Cannot use hyphen (-)
```

```
class = "Python"  # 'class' is a reserved keyword
```

Python Keywords

Definition: Keywords are reserved words with special meanings in Python.

Python comes with its own set of keywords. It is important to be aware of these when you're naming variables: your variable names cannot be the same as these keywords or you'll overwrite the functions.

Examples of Python Keywords:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Each of the following keywords has a specific meaning. Using any of them as a variable name will cause an error.

Checking All Python Keywords:

```
import keyword

print(keyword.kwlist)
```

2.1.3 Indentation, Splitting and Joining Python Statements

Lines and Indentation

Python uses indentation instead of curly brackets {} to define blocks of code.

Example of a correct indentation: (# four spaces indentation)

if True:

```
    print("Python uses indentation")
```

Example of an incorrect indentation (This will cause an error):

if True:

```
print("This will cause an indentation error")
```

```
# Missing indentation
```

Output:

IndentationError: expected an indented block

Best Practice: Use four spaces per indentation level (avoid using tabs).

Splitting and joining Statements

Most programming statements fit on one line, but long statements can be hard to read or print. In Python, you can split a long statement across multiple lines using the backslash (\) as a line continuation character.

For example:

```
total = 10 + 20 + 30 + \  
        40 + 50 + 60
```

```
print(total)                # Output: 210
```

The line continuation character (\) at the end of a line tells the interpreter that the statement continues on the next line.

Python also allows statements enclosed in parentheses to be split across multiple lines without using the continuation character. For example, consider the following statement:

```
print("Monday's sales are", monday,  
      "and Tuesday's sales are", tuesday,  
      "and Wednesday's sales are", wednesday)
```

The following code shows another example:

```
total = (value1 + value2 +  
value3 + value4 +  
value5 + value6)
```

Joining Statements

Python allows multiple statements in one line using a semicolon (;).

For example:

```
x = 10; y = 20; print(x + y) # Output: 30
```

Best Practice: Avoid excessive use of semicolons for better readability.

Writing Code Blocks

A code block refers to multiple statements grouped together under an if, loop, function, or class.

For example (Function Code Block):

```
def greet():  
    print("Hello, Python!")  
    print("This is a function block.")  
  
greet()
```

Note: Python requires proper indentation for defining blocks.

2.1.4 Comments and Quotations in Python

Comments in Python

Definition: Comments are notes of explanation that document lines or sections of a program. Comments are part of the program, but the Python interpreter ignores them. They are intended for people who may be reading the source code.

In Python, a comment starts with #, and everything after it on the line is ignored.

Single-Line Comment:

```
# This is a single-line comment  
  
print ("Hello, Python!") # Another comment
```

Multi-Line Comment (Using Triple Quotes):

```
"""  
This is a multi-line comment.  
Python ignores these lines.  
"""  
  
print("Python is fun!")
```

Best Practice: Use comments to improve code readability, but avoid over-commenting. Clear comments explain the purpose and logic of code, making it easier to understand, maintain, and collaborate on, especially in large or complex programs.

Quotations in Python

Programs almost always work with data of some type. For instance, consider the following three pieces of data:

```
'Kate Austen'  
'123 Full Circle Drive'
```

```
'Asheville, NC 28899'
```

These are examples of strings, which are sequences of characters used as data. When a string appears directly in the source code, it is known as a string literal. In Python, string literals must be enclosed in quote marks to indicate where the string begins and ends.

Types of Quotes in Python

Python offers flexibility in how string literals are written. You can use:

- Single quotes (')
- Double quotes (")

Both are functionally the same and can be used interchangeably. For example:

```
print("Kate Austen")  
print("123 Full Circle Drive")  
print("Asheville, NC 28899")
```

This produces the output:

```
Kate Austen  
123 Full Circle Drive  
Asheville, NC 28899
```

Handling Apostrophes and Quotes in Strings

If a string contains an apostrophe ('), you can avoid a syntax error by using double quotes to enclose the string:

```
print("Don't fear!")  
print("I'm here!")
```

Output:

```
Don't fear!  
I'm here!
```

Similarly, if a string contains double quotes, you can enclose it with single quotes:

```
print('Your assignment is to read "Hamlet" by tomorrow.')
```

Output:

```
Your assignment is to read "Hamlet" by tomorrow..
```

Triple-Quoted Strings

Python also supports triple-quoted strings using either triple double quotes ("""") or triple single quotes (```). These are useful in two main cases:

Including both single and double quotes in the same string without escaping:

```
print("""I'm reading "Hamlet" tonight.""")
```

Output:

```
I'm reading "Hamlet" tonight.
```

Creating multi-line strings, where the text spans several lines:

```
print("""One  
Two  
Three""")
```

Output:

```
One  
Two  
Three
```

Triple quotes are especially handy for writing long passages of text or when preserving the format and line breaks of the original string is important.

2.1.5 Debugging

Debugging is the process of detecting and removing of existing and potential errors (also called 'bugs') in a program that can cause it to behave unexpectedly or crash. To prevent incorrect operation of a software or system, debugging is used to find and resolve bugs or defects.

Three kinds of errors can occur in a program: syntax errors, runtime errors, and semantic errors. It is useful to distinguish between them in order to track them down more quickly.

Syntax error:

"Syntax" refers to the structure of a program and the rules about that structure. For example, parentheses have to come in matching pairs, so (1 + 2) is legal, but 8) is a syntax error.

A syntax error in Python occurs when the code's structure or format is incorrect, making it impossible for the interpreter to run, such as with missing punctuation or wrong indentation. When this happens, Python displays an error message and stops execution.

Beginners often spend time fixing such errors, but with experience, they become less frequent and easier to identify

Runtime error:

The second type of error is a runtime error, so called because the error does not appear until after the program has started running. These errors are also called exceptions because they usually indicate that something exceptional (and bad) has happened.

A runtime error in Python is an error that occurs while the program is running, often caused by illegal operations like dividing by zero or accessing a variable that has not been defined. These errors stop the program during execution.

Runtime errors are rare in the simple programs you will see in the first few chapters, so it might be a while before you encounter one.

Semantic error:

The third type of error is “semantic”, which means related to meaning. If there is a semantic error in your program, it will run without generating error messages, but it will not do the right thing. It will do something else. Specifically, it will do what you told it to do.

A semantic error in Python occurs when the code runs without crashing but produces incorrect or unexpected results because the logic does not do what the programmer intended.

Identifying semantic errors can be tricky because it requires you to work backwards by looking at the output of the program and trying to figure out what it is doing.

In-Text Questions (ITQs)

1. What is the difference between interactive and script mode in Python?
2. Why is indentation important in Python?
3. Can Python strings be enclosed in either single or double quotes?
4. What is a semantic error in Python?

In-Text Answers (ITAs)

1. Interactive mode executes one command at a time for testing, while script mode runs a saved file as a complete program.

2. Indentation defines code blocks; incorrect indentation causes an `IndentationError` and stops program execution.
3. Yes, Python allows both single (') and double (") quotes for strings, which can be used interchangeably.
4. A semantic error occurs when a program runs without crashing but produces incorrect results due to flawed logic

2.1.6 Conclusion

In this unit, you have learned about the essential syntax and structure of Python programming. You can now distinguish between interactive and script modes, create and run Python programs, use print statements, apply proper indentation, and work with identifiers, keywords, and strings. You also gained an understanding of writing comments and the basics of debugging common errors.

Summary

In this unit, you have learnt that Python supports two programming modes—interactive mode for quick testing and script mode for writing full programs. You explored how to create program files, display output with `print()`, and use identifiers, keywords, indentation, comments, and quotations correctly. You also learnt how Python handles string literals, how to split and join statements, and the importance of debugging and recognising different types of program errors.

Self-Assessment Questions

1. What are the two primary modes of writing and executing Python programs?
2. How can you run a Python script saved as `example.py` from the command line?
3. What rules guide the creation of valid Python identifiers?
4. What happens when a Python code block is not properly indented?

Unit Activities

1. Open the Python interactive shell and run a simple print statement such as `print("Hello, Python!")`.
2. Create a Python script file (welcome.py) that prints a welcome message and test it.
3. Identify and list five valid and five invalid Python identifiers, then experiment with different string formats (single, double, triple quotes).

Tutor-Marked Assignment

1. Explain the difference between interactive mode and script mode in Python, stating one advantage and one disadvantage of each.
2. Identify and correct the syntax error in this code:

```
#This program prints a greeting message  
  
print("Hello, students!")
```
3. From the following, identify which are valid Python identifiers and which are invalid, giving reasons for the invalid ones:
`_name, 2ndItem, full name, class, user_id`
4. Write a short Python script that prints your name on one line and a short message on the next line.

References/Further Readings

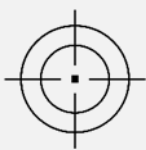
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Unit 2 — Python Variables and Data Types

INTRODUCTION

In this unit, you will learn about the concept of variables and how data is stored and manipulated in Python using different data types. You will explore variable assignment, naming conventions, the `print()` and `input()` functions, and how to read and convert user input. You will also examine built-in Python data types, including numbers, strings, Booleans, and how Python handles dynamic typing.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Create variables and assign values using proper naming rules.
- Identify and use Python's core data types (int, float, str, bool, and complex).
- Use the `input()` function to read user input and convert it to other types.
- Display multiple items using the `print()` function.
- Explain dynamic typing and variable reassignment in Python

2.2.1 Python Variables

A variable is a name that represents a value stored in the computer's memory. Think of a variable as a labelled box that stores data which your program can use and manipulate later.

Programs store data in memory and perform operations on it. For example, in online shopping, item details are stored as you add them to a cart, and the program calculates totals, taxes, and shipping costs.

Programs use variables to manage this data. A variable like tax may store the sales tax, while distance may store the distance between cities. When a variable holds a value, it references that value.

Creating Variables with Assignment Statements

In Python, variables are created using assignment statements. An assignment statement assigns a value to a variable name using the = operator.

```
name = "Alice"  
age = 25  
height = 5.7
```

Here, name, age, and height are variables storing a string, an integer, and a floating-point number. When executed, each variable is linked to its assigned value in memory.

The general format is:

variable = expression

The equal sign (=) is called the assignment operator. The variable is written on the left and the value or expression on the right.

Example:

```
width = 20  
length = 10  
print(width)  
print(length)
```

Output:

```
20  
10
```

Do not use quotation marks around variable names when printing; quotes make them strings.

```
print('width') # Displays the word 'width', not the value
```

Only variables can appear on the left side of the assignment operator. Using a literal value will cause an error:

```
25 = age  
# SyntaxError: cannot assign to literal
```

Example :

```
# This program demonstrates a variable.  
room = 503  
print('I am staying in room number')  
print(room)
```

Output:

```
I am staying in room number  
503
```

A variable must be assigned before it is used. Using a variable without assignment results in an error. For example:

```
temperature = 74.5 # Create a variable  
print(tempereture) # Misspelled name  
# NameError: name 'tempereture' is not defined
```

Python is **case-sensitive**, meaning variable names with different cases are treated as distinct:

```
temperature = 74.5  
print(Temperature)  
# NameError: name 'Temperature' is not defined
```

Variable Naming Rules

When naming variables in Python, follow these rules:

- Do not use Python keywords as variable names.
- Variable names cannot contain spaces.
- The first character must be a letter (A–Z or a–z) or an underscore (_).
- Subsequent characters can be letters, digits (0–9), or underscores.
- Variable names are case sensitive (e.g., `ItemsOrdered` and `itemsordered` are different).

Use meaningful names that describe the variable's purpose. For instance, use `temperature` for a temperature value or `speed` for a car's speed rather than vague names like `x` or `b2`.

For names with multiple words, use underscores for clarity (snake_case), such as:

- gross_pay
- pay_rate
- hot_dogs_sold_today

Alternatively, you may use camel case (camelCase), where the first word starts in lowercase and each subsequent word begins with an uppercase letter:

- grossPay
- payRate
- hotDogsSoldToday

The name camelCase comes from the “humps” formed by uppercase letters within the name.

Table 1: Summary of Legal and Illegal Python Variable Names

Variable Name	Legal or Illegal
units_per_day	Legal
dayOfWeek	Legal
3dGraph	Illegal (cannot begin with a digit)
June1997	Legal
Mixture#3	Illegal (only letters, digits, and underscores allowed)

Displaying Multiple Items with the print() Function

Python allows multiple items to be displayed in one print() statement by separating them with commas.

Example:

```
# This program demonstrates a variable.
room = 503
print('I am staying in room number', room)
```

Output:

```
I am staying in room number 503
```

In this example, both the string literal and the variable room are passed to the print() function, which automatically separates the items with a space.

Variable Reassignment

Variables are called “variable” because they can store different values while a program runs. When you assign a value to a variable, it holds that value until you assign another.

In Program below, the statement on line 3 creates a variable named **dollars** and assigns it the value 2.75. Later, line 8 changes the value of dollars to 99.95. The original value (2.75) remains in memory but is no longer accessible since it is not linked to any variable. Python automatically clears such unused values from memory through a process called **garbage collection**.

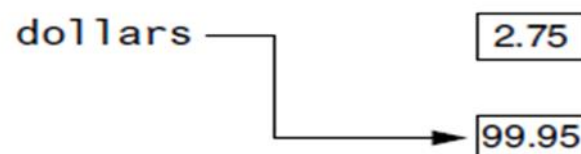
Program (Example of Variable Reassignment in Python):

1. >>> # This program demonstrates variable reassignment.
2. >>> # Assign a value to the dollars variable.
3. >>> dollars = 2.75
4. >>> print('I have', dollars, 'in my account.')
5. I have 2.75 in my account.
6. >>>
7. >>> # Reassign dollars so it references a different value.
8. >>> dollars = 99.95
9. >>> print('But now I have', dollars, 'in my account!')
10. But now I have 99.95 in my account!

The dollars variable after line 3 executes.



The dollars variable after line 8 executes.



2.2.2 Datatypes

A datatype in Python is a classification of data items based on the type of value they hold. It defines the possible values and the operations that can be performed on the data. Python provides various built-in datatypes, including numeric types (e.g., int, float), sequence types (e.g., list, tuple, range), mapping types (e.g., dictionary), set types (e.g., set, frozenset), and text types (e.g., str), among others.

Python uses data types to organise values in memory. For instance, whole numbers are stored as integers (int) while decimal numbers are stored as floating-point numbers (float).

Implicit Declaration of Data Types

Python is dynamically typed, meaning you do not need to declare a variable's data type before using it. The interpreter automatically determines the type based on the assigned value:

```
x = 10          # Integer
y = 3.14        # Float
z = "Hello"     # String
```

Python detects the type automatically, making programming faster and simpler. You can check a variable's data type using the **type()** function:

```
print(type(x))  # Output: <class 'int'>
```

Numbers

Python supports three numeric types:

- **int** – whole numbers (e.g., $a = 15$)
- **float** – numbers with decimals (e.g., $\pi = 3.14$)
- **complex** - numbers with real and imaginary parts (e.g., $z = 3 + 4j$)

Python automatically classifies numeric literals based on their form. For instance, 7 is an integer and 2.5 is a float.

Strings

A string (str) is a sequence of characters enclosed in single or double quotes, such as 'Hello' or "Python". Strings can contain letters, numbers, and special characters, and support operations like concatenation (+), repetition (*), indexing, and built-in methods such as upper(), lower(), and replace().

Booleans

The Boolean (bool) datatype has two values: True and False. Booleans are used for decision-making in conditions, loops, and comparisons (e.g., $x > y$ returns False).

Table 2: Summary of Python Datatypes

Data Type	Description	Example
Int	Whole numbers	$a = 100$
Float	Decimal numbers	$b = 3.14$
complex	Numbers with imaginary part	$z = 2 + 5j$
Str	Text or sequence of characters	<code>name = "Alice"</code>
Bool	True or False values	<code>is_valid = True</code>

2.2.3 Dynamic Typing and User Input in Python

Reassigning a Variable to a Different Type

In Python, a variable can be reassigned to store data of a different type, even after being used previously. This is possible because Python is dynamically typed and does not require explicit type declarations.

Consider the following example:

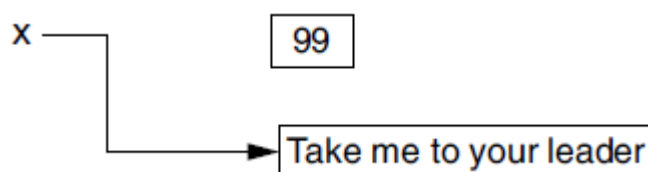
```
>>> x = 99 Enter
>>> print(x) Enter
99
>>> x = 'Take me to your leader' Enter
>>> print(x) Enter
Take me to your leader.
>>>
```

In line 1, a variable `x` is created and assigned the integer value 99. Line 2 prints this value, producing the output in line 3. In line 4, `x` is reassigned to hold a string value. After this, `x` no longer refers to an integer but to the string "Take me to your leader". Lines 5 and 6 show the new value being printed.

Here, the variable `x` references an integer.



Here, the variable `x` references a string



Reading Input from the Keyboard

Programs often need input from users. Python provides the `input()` function, which displays a prompt, pauses the program, and waits for the user to type and press Enter. The `input()` function returns user input as a string and is usually stored in a variable.

The general format is:

```
variable = input(prompt)
```

Here, ***prompt*** is the message shown to the user.

Example:

```
name = input('What is your name? ')
print(name)
```

Output:

```
What is your name? Holly
Holly
```

In this example, the user types **Holly**, and the input is stored in name.

You can also collect multiple inputs and use them together, as shown below:

Program (string_input.py):

```
first_name = input('Enter your first name: ')
last_name = input('Enter your last name: ')
print('Hello', first_name, last_name)
```

Sample Output:

```
Enter your first name: Vinny
Enter your last name: Brown
Hello Vinny Brown
```

Note: Add a space at the end of each prompt (e.g., 'Enter your first name: ') to keep the user's input neatly separated from the prompt.

Reading Numbers with the input() Function

The **input()** function always receives user input as a string, even when numbers are entered. To perform calculations, the input must be converted to numeric types using **int()** or **float()**.

Conversion Functions

int(item): Converts a string to an integer.

float(item): Converts a string to a floating-point number.

Example:

```
hours = int(input('How many hours did you work? '))
```

Here, the input is taken as a string, converted to an integer, and stored in **hours**.

Another Example:

```
pay_rate = float(input('What is your hourly pay rate? '))
```

The program converts the string input to a float and stored in **pay_rate**.

Program (get_user_info.py):

```
# Program to get user details
name = input('What is your name? ')
age = int(input('What is your age? '))
income = float(input('What is your income? '))

print('Here is the data you entered:')
print('Name:', name)
print('Age:', age)
print('Income:', income)
```

Sample Output:

```
What is your name? Chris
What is your age? 25
What is your income? 75000.0
Here is the data you entered:
Name: Chris
Age: 25
Income: 75000.0
```

Important Note: Python raises a **ValueError** when the user enters non-numeric input.

```
>>> age = int(input('What is your age? '))
What is your age? xyz
ValueError: invalid literal for int() with base 10:
'xyz'
```

This is a runtime error known as an exception, and it stops the program unless handled properly.

In-Text Questions (ITQs)

1. What function is used to determine the data type of a value?
2. What data type is returned by the input() function?
3. What happens when a variable is reassigned a new value of a different type?

4. What will this code output?

```
x = 4 + 3j  
  
print(x.real, x.imag?)
```

In-Text Answers (ITAs)

1. The `type()` function is used to check a value's data type.
2. The `input()` function always returns a string.
3. The variable takes the new value and type, replacing its previous one.
4. Its outputs: 4.0 3.0 — the real and imaginary parts of the complex number.

2.2.4 Conclusion

In this unit, you have learned how Python uses variables and built-in data types to store and manipulate data. You can now assign values, follow variable-naming rules, and display output with `print()`. You also explored dynamic typing, checked data types with `type()`, and converted user input from strings to numbers using `int()` and `float()`. These basics prepare you to write simple, interactive Python programs that handle user input effectively.

Summary

In this unit, you have learned that variables in Python are symbolic names that reference values in memory. You explored how to create variables using assignment statements, follow naming rules, and display values with `print()`. You also examined Python's dynamic typing and used the `type()` function to identify data types. Additionally, you learned about built-in types such as integers, floats, complex numbers, strings, and Booleans. Finally, you practised using `input()` to collect user data and convert it to suitable types for processing.

Self-Assessment Questions

1. What is a variable in Python, and how is it created?
2. What does the assignment operator (`=`) do in Python?

3. What is dynamic typing in Python?
4. How can you check the data type of a value in Python?

Unit Activities

1. Create variables of different data types (string, integer, float, Boolean, and complex), display their values, and use `type()` to show each variable's data type.
2. Write a Python script that collects user input (e.g., name, age, and CGPA) and prints the information using a single `print()` statement.
3. Reassign variables to new data types and work with a complex number by displaying its real and imaginary parts.

Tutor-Marked Assignment

1. Write a Python program that prompts the user for their full name, age, and current salary. Convert the age to an integer and the salary to a float, then display a summary sentence. Briefly explain what happens if the user enters invalid data (e.g., non-numeric age).

2. Given:

```
x = 5
y = 2.0
z = "5"
```

Predict and explain the output of:

- `print(x + y)`
- `print(x + int(z))`
- `print(str(x) + z)`
- `print(x + float(z))`

Describe why each result occurs

3. Explain dynamic typing and variable reassignment in Python. Then write a short script showing a single variable first holding an integer, then a string, and finally a Boolean. Show the output for each reassignment.

References/Further Readings

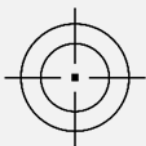
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Unit 3 — Python Operators

INTRODUCTION

In this unit, you will learn about the different types of Python operators used in programming. These include arithmetic, comparison, logical, identity, and assignment operators. You will also study how operator precedence influences expression evaluation and how to convert algebraic expressions into valid Python code. Additionally, the unit introduces named constants, mixed-type expressions, and formatting outputs in Python.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Identify and apply Python arithmetic, comparison, logical, identity, and assignment operators.
- Demonstrate the correct use of operator precedence and parentheses in expressions.
- Translate algebraic expressions into Python code accurately.
- Use named constants and understand their purpose in improving code clarity and maintainability.
- Write Python programs that apply operators in real-world problem-solving scenarios, including expression evaluation and output formatting

2.3.1 Operators in Python

In Python, operators are special symbols or keywords that perform operations on values and variables. These values are called operands. Python has several types of operators, each serving a unique purpose. Understanding them is foundational to programming effectively in Python.

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Python provides several operators for performing mathematical calculations, which are essential in most real-world algorithms. These operators allow programmers to create mathematical expressions that perform computations and produce values. Table 3 below lists the mathematical operators available in Python.

Table 3: Summary of Python Arithmetic Operators

Operator	Sym- bol	Description	Exam- ple	Re- sult
Addition	+	Adds two operands	5 + 3	8
Subtraction	-	Subtracts the second operand from the first	10 - 4	6
Multiplication	*	Multiplies both operands	6 * 7	42
Division	/	Divides the first operand by the second (float result)	8 / 2	4.0
Integer or Floor Division	//	Divides and returns the integer part of the quotient	9 // 2	4
Modulus	%	Returns the remainder of the division	10 % 3	1
Exponentiation	**	Raises the first operand to the power of the second	2 ** 3	8

Note: Python automatically handles the type of the result (e.g., float or int).

The following is an example of a simple math expression:

$$12 + 2$$

The values on the right and left of the + operator is called **operands**. These are values that the + operator adds together. If you type this expression in interactive mode, you will see that it gives the value 14:

```
>>> 12 + 2 Enter
14
>>>
```

Variables may also be used in a math expression. For example, suppose we have two variables named **hours** and **pay_rate**. The following math expression uses the ***** operator to multiply the value referenced by the **hours** variable by the value referenced by the **pay_rate** variable:

```
hours * pay_rate
```

When we use a math expression to calculate a value, normally we want to save that value in memory so we can use it again in the program. We do this with an assignment statement. The `string_math` program shows an example.

Program (simple_math.py):

1. `# Assign a value to the salary variable.`
2. `salary = 2500.0`
3. `# Assign a value to the bonus variable.`
4. `bonus = 1200.0`
5. `# Calculate the total pay by adding salary`
6. `# and bonus. Assign the result to pay.`
7. `pay = salary + bonus`
8. `# Display the pay.`
9. `print('Your pay is', pay)`

Program Output

```
Your pay is 3700.0
```

Line 2 assigns 2500.0 to the **salary** variable, and line 5 assigns 1200.0 to the **bonus** variable. Line 9 assigns the result of the expression **salary + bonus** to the **pay** variable. As you can see from the program output, the **pay** variable holds the value 3700.0.

Floating-Point and Integer Division

Python has two division operators. The `/` operator performs floating point division, and the `//` operator performs integer division.

Both operators divide one number by another. The `/` operator returns a floating-point result, while the `//` operator returns a whole number.

Example in interactive mode:

```
>>> 5 / 2 Enter
2.5
>>>
```

Using integer division:

```
>>> 5 // 2 Enter
2
>>>
```

The // operator removes the fractional part when the result is positive. When the result is negative, it rounds the value away from zero.

Example with a negative result:

```
>>> -5 // 2 Enter
-3
```

Operator Precedence

Python follows the standard order of operations when evaluating expressions. For example:

```
answer = 17 + x + 21 + y
```

Some expressions require careful consideration:

```
outcome = 12.0 + 6.0 / 3.0
```

Here, 6.0 could be divided first or added first. Python resolves this using operator precedence:

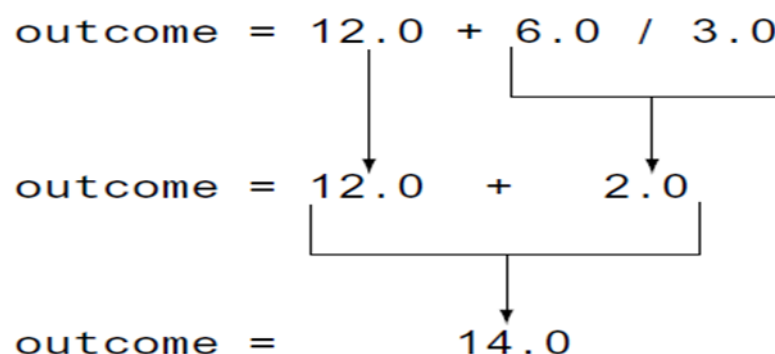
1. Exponentiation: **
2. Multiplication, division, and remainder: * / // %
3. Addition and subtraction: + -

Operators with the same precedence are evaluated left to right.

In the example above, division occurs before addition:

```
outcome = 12.0 + 6.0 / 3.0
outcome = 14.0
outcome = 12.0 + 2.0
```

The figure below illustrates the evaluation order:



The table 4 below shows some other sample expressions with their values.

Table 4: Sample Expressions with their Values

Expression	Value
$4 + 3 * 2$	10
$18 / 3 + 2$	8.0
$2 + 3 * 4 - 5$	9
$10 - 2 * 3 + 1$	5
$5 * 2 ** 3$	40
$2 ** 2 ** 2$	16
$20 / 5 * 2 + 6$	14.0
$15 - 4 + 2 * 3$	17
$6 + 4 / 2 ** 2$	7.0
$8 - 3 + 2 * 5$	15

Grouping with Parentheses

Parentheses in a mathematical expression group operation to control the order of evaluation. In:

$$\text{result} = (a + b) / 4$$

a and b are added first, then divided by 4. Without parentheses, b would be divided by 4 before adding to a. The table 5 below shows more expressions and their results.

Table 5: Sample Expressions using groupings with their Values

Expression	Value
$(4 + 3) * 2$	14
$18 / (3 + 3)$	3.0
$2 + (3 * (4 - 1))$	11
$((10 - 2) * 3) + 1$	25
$(5 * 2) ** 2$	100
$2 ** (2 + 2)$	16
$20 / (5 * (1 + 1))$	2.0
$(15 - (4 + 2)) * 3$	27
$6 + (4 / (2 ** 2))$	7.0
$(8 - 3 + 2) * (5 - 1)$	28

Exponential Operator (**)

In Python, the exponential operator ****** is used to raise a number to the power of another number. This operator performs exponentiation.

Two asterisks written together (******) is the exponent operator, and its purpose is to raise a number to a power.

Syntax:

```
result = base ** exponent
```

Example:

```
>>> 2 ** 3
8
>>> 5 ** 2
25
>>> 9 ** 0.5
3.0 # Square root of 9
```

Note: Exponentiation is **right-associative**. That means expressions with multiple ****** are evaluated from **right to left**.

```
>>> 2 ** 3 ** 2
512 # Equivalent to 2 ** (3 ** 2) = 2 ** 9 = 512
```

Remainder Operator (%)

In Python, the **%** symbol is the remainder operator. (This is also known as the modulus operator) The remainder operator performs division, but instead of returning the quotient, it returns the remainder. The following statement assigns 2 to leftover:

```
leftover = 17 % 3
```

This statement assigns 2 to leftover because 17 divided by 3 is 5 with a remainder of 2.

The remainder operator is useful in certain situations. It is commonly used in calculations that convert times or distances, detect odd or even numbers, and perform other specialized operations.

The remainder operator (also called the modulus operator) returns the remainder of a division operation.

Syntax:

```
result = dividend % divisor
```

Example:

```
>>> 10 % 3
1
>>> 20 % 4
0
>>> 7 % 5
2
```

Use Cases of the Remainder Operator:

Determine if a number is even or odd:

```
>>> 10 % 2
# Even
>>> 7 % 2
# Odd
```

Wraparound logic (e.g., clock arithmetic):

```
>>> (10 + 5) % 12
3
```

For example, the `time_converter` program gets a number of seconds from the user, and it converts that number of seconds to hours, minutes, and seconds. For example, it would convert 11,730 seconds to 3 hours, 15 minutes, and 30 seconds.

Program (`time_converter.py`):

1. `# Get a number of seconds from the user.`
2. `total_seconds = float(input('Enter a number of seconds: '))`
3. `# Get the number of hours.`
4. `hours = total_seconds // 3600`
5. `# Get the number of remaining minutes.`
6. `minutes = (total_seconds // 60) % 60`
7. `# Get the number of remaining seconds.`
8. `seconds = total_seconds % 60`
9. `# Display the results.`
10. `print('Here is the time in hours, minutes, and seconds:')`
11. `print('Hours:', hours)`
12. `print('Minutes:', minutes)`
13. `print('Seconds:', seconds)`

Program Output (with input shown in bold)

```
Enter a number of seconds: 11730 Enter
```

```
Here is the time in hours, minutes, and seconds:
```

Hours: 3.0

Minutes: 15.0

Seconds: 30.0

Let's take a closer look at the code:

Line 2 reads the total seconds from the user, converts it to a float, and stores it in `total_seconds`.

Line 5 computes the whole hours using integer division (`total_seconds // 3600`).

Line 8 computes remaining minutes by first getting total minutes (`total_seconds // 60`) and then taking the remainder when divided by 60.

Line 11 computes remaining seconds using `total_seconds % 60`.

Lines 14–17 display the hours, minutes, and seconds.

Converting Math Formulas to Programming Statements

You probably remember from algebra class that the expression $2xy$ is understood to mean 2 times x times y . In math, you do not always use an operator for multiplication. Python, as well as other programming languages, requires an operator for any mathematical operation.

Guidelines:

Use Python arithmetic operators (+, -, *, /, **, %) appropriately.

Follow Python's precedence rules or use parentheses to clarify grouping.

Ensure all variables are defined before use.

Table 6 shows some algebraic expressions that perform multiplication and the equivalent programming expressions

Table 6: Algebraic Expressions and their Equivalent Programming Expressions

Algebraic Expression	Operation Being Performed	Programming Expression
$7A$	7 times A	<code>7 * A</code>
$(5)(9)$	5 times 9	<code>5 * 9</code>
$2xy$	2 times x times y	<code>2 * x * y</code>
ab	a times b	<code>a * b</code>
$4(x + y)$	4 times (x plus y)	<code>4 * (x + y)</code>
$3x^2$	3 times x squared	<code>3 * x ** 2</code>
$2a(b + c)$	2 times a times (b plus c)	<code>2 * a * (b + c)</code>

$\frac{1}{2}bh$	one-half times b times h	<code>0.5 * b * h</code>
$(x + y)(x - y)$	(x plus y) times (x minus y)	<code>(x + y) * (x - y)</code>
πr^2	pi times r squared	<code>math.pi * r ** 2</code>

When translating algebraic expressions into Python, parentheses are often needed to clearly specify the order of operations, especially when algebraic grouping is implied but not explicit. For example:

$$x = \frac{a + b}{c}$$

To accurately reflect the intended mathematical operation in Python, the addition operation must be enclosed in parentheses:

$$x = (a + b) / c$$

Failure to use parentheses in such cases can lead to incorrect evaluation of the expression due to Python's operator precedence rules.

Table 7 below presents a set of algebraic expressions along with their equivalent Python programming expressions.

Table 7: Some Algebraic Expressions and their Equivalent Programming Expressions

Algebraic Expression	Operation Being Performed	Programming Expression
$y = 3\frac{x}{2}$	y equals 3 times x divided by 2	<code>y = (3 * x) / 2</code>
$z = 3bc + 4$	z equals 3 times b times c plus 4	<code>z = 3 * b * c + 4</code>
$a = \frac{5}{x + 2}$	a equals 5 divided by (x plus 2)	<code>a = 5 / (x + 2)</code>
$r = ab - cd$	r equals a times b minus c times d	<code>r = a * b - c * d</code>
$k = \frac{x^2 + y^2}{z}$	k equals (x squared plus y squared) over z	<code>k = (x ** 2 + y ** 2) / z</code>

Some examples include:

Example 1: Area of a Circle

Mathematical Formula: $A = \pi r^2$

Python code:

```
>>> import math
>>> radius = 5
>>> area = math.pi * radius ** 2
>>> area
78.53981633974483
```

Example 2: Future Value Formula

Mathematical Formula: $F = P \times (1 + r)^n$

Where:

F is the future value

P is the principal amount

r is the interest rate

n is the number of periods

Python code:

```
>>> P = 1000
>>> r = 0.05
>>> n = 3
>>> F = P * (1 + r) ** n
>>> F
1157.6250000000002
```

Mixed-Type Expressions and Data Type Conversion

In Python, arithmetic expressions can involve different data types like integers and floats. When this happens, Python automatically performs type conversion to avoid errors and ensure consistency.

How Python Handles Mixed Types

Python uses the following rules:

- Operations on two integers produce an `int`.
- Operations on two floats produce a `float`.
- Operations on an `int` and a `float` result in a `float` (the `int` is temporarily converted).

Example – Mixing `int` and `float`:

```
>>> 5 + 2.0
7.0
```

Here, the integer 5 is converted to 5.0, so the result is a float.

Explicit Type Conversion (Casting)

Sometimes, you may want to convert data types manually using **casting functions**. This ensures your variables behave as intended during operations.

Examples – Using `int()`, `float()`, and `str()`:

```
>>> int(4.9)
4
```

```
>>> float(3)
3.0
>>> str(100)
'100'
```

Example – Controlling float division:

```
>>> total = 15
>>> number = 2
>>> average = total / number
>>> average
7.5

>>> average = float(total) / number
>>> average
7.5
```

Comparison/Relational Operators

These operators compare the values of two operands and return a Boolean value: True or False. Table 8 provides a comprehensive summary of the various comparison operators in Python.

Table 8: Comparison Operators in Python

Operator	Symbol	Description	Example	Result
Equal to	==	Returns True if both operands are equal	5 == 5	True
Not equal to	!=	Returns True if operands are not equal	5 != 3	True
Greater than	>	Returns True if the left operand is greater	7 > 5	True
Less than	<	Returns True if the left operand is smaller	4 < 10	True
Greater than or equal to	>=	Returns True if left operand is greater or equal	7 >= 7	True
Less than or equal to	<=	Returns True if left operand is smaller or equal	2 <= 4	True

Useful for making decisions in if, while, and for statements.

The following is an example of an expression that uses the greater than (>) operator to compare two variables, length and width:

```
length > width
```

This expression checks whether length is greater than width, returning true if it is and false otherwise. Similarly, the less-than operator checks whether length is less than width:

```
length < width
```

You can test Boolean expressions in Python's interactive mode. Simply type the expression at the `>>>` prompt, and Python evaluates it to True or False. The example below shows a sample session (with line numbers added for clarity).

1. `>>> x = 1` Enter
2. `>>> y = 0` Enter
3. `>>> x > y` Enter
4. True
5. `>>> y > x`
6. False
7. `>>>`

Line 1 assigns 1 to x, and line 2 assigns 0 to y. Line 3 evaluates `x > y`, which outputs True in line 4. Line 5 evaluates `y > x`, which outputs False in line 6.

Increment Operators (and Decrement)

Python does **not** have the `++` or `--` increment/decrement operators found in some other languages like C++ or Java.

Instead, we use:

```
x = 5
x += 1 # Equivalent to x = x + 1
print(x) # Output: 6

x -= 1 # Equivalent to x = x - 1
print(x) # Output: 5
```

Compound Assignment Operators:

Compound assignment operators are Python code shorthand operators in Python that combine an arithmetic operation with an assignment in a single statement. Instead of writing both the operation and the assignment separately, you use a compound operator to update the value of a variable more concisely.

Table 9: Compound Assignment in Python

Operator	Description	Example
+=	Increment	x += 1
-=	Decrement	x -= 1
*=	Multiply and assign	x *= 2
/=	Divide and assign	x /= 3
//=	Floor divide and assign	x //= 2
%=	Modulus and assign	x %= 4
**=	Power and assign	x **= 3

Logical Operators

The logical and operator and the logical or operator allow you to connect multiple Boolean expressions to create a compound expression. The logical not operator reverses the truth of a Boolean expression.

Python provides a set of operators known as logical operators, which you can use to create complex Boolean expressions. The table 10 describes these operators.

It is used to combine conditional statements. They return True or False based on logic.

Table 10: Logical Operators in Python

Operator	Description	Example	Result
And	Returns True if both conditions are true	True and False	False
Or	Returns True if at least one condition is true	True or False	True
Not	Reverses the result (negation)	not True	False

Example:

```
x = 5
y = 10
print(x > 3 and y < 15)    # True
print(x > 6 or y == 10)    # True
print(not(x == 5))         # False
```

Identity Operators

It used to compare **memory locations** of two objects. These are: **is** and **is not**

Table 11: Identity Operators in Python

Operator	Description	Example	Result
is	Returns True if both variables point to the same object	a is b	True (if same object)
is not	Returns True if variables point to different objects	a is not b	True (if different)

Example:

```

a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a is b)      # True (same object in memory)
print(a is c)      # False (same content but different objects)
print(a == c)      # True (values are equal)

```

Operator Precedence

When multiple operators are used in an expression, Python follows **precedence rules** to determine which operation to perform first.

Precedence Order (from highest to lowest):

The Operator Precedence, as shown in Table 12 clearly from highest to lowest

Table 12: Operator Precedence

Precedence Level	Operator(s)	Description
1(Highest)	()	Parentheses – used to group expressions
2	**	Exponentiation
3	+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
4	*, /, //, %	Multiplication, Division, Floor Division, Modulus
5	+, -	Addition and Subtraction
6	<<, >>	Bitwise Shift Operators
7	&	Bitwise AND
8	^	Bitwise XOR
9		Bitwise OR
10	==, !=, >, <, >=, <=, is, is not, in, not in	Comparison and Membership Operators
11	not	Logical NOT

12	and	Logical AND
13	or	Logical OR
14 (Lowest)	=, +, -, *, /, etc.	Assignment Operators

Example:

```
result = 3 + 2 * 4 ** 2
# First: 4 ** 2 = 16
# Then: 2 * 16 = 32
# Finally: 3 + 32 = 35
print(result) # Output: 35
```

You can use parentheses () to override precedence

2.3.2 Datatypes More About Data Output with print ()

The print() function is a versatile way to display output in Python. Besides printing text, you can format the output to suit your needs.

1. Breaking Long Statements in Python

As programs get more complex, your statements might get too long to fit on one line. Python offers clean ways to split them for readability.

Using the Line Continuation Character (\)

When a single logical statement is too long, use \ to split it across lines.

Example – Line break using backslash:

```
result = var1 * 2 + var2 * 3 + \
        var3 * 4 + var4 * 5
```

The \ tells Python the line continues. Be sure it is the very last character.

2. Breaking Inside Parentheses, Brackets, or Braces

You can also split long lines **without** using backslashes, by enclosing the statement in () or [].

Example – Line break in parentheses:

```
total = (value1 + value2 +
        value3 + value4)
```

Example – Breaking a print() statement:

```
print("Monday's sales are", monday,  
      "and Tuesday's sales are", tuesday)
```

Example – Breaking a list across lines:

```
colors = [  
    'red', 'blue', 'green',  
    'yellow', 'purple', 'orange'  
]
```

Suppressing Newlines in Output

By default, `print()` adds a newline at the end. You can suppress this using the `end` parameter.

Example – Suppressing newlines:

```
>>> print("Hello", end="")
```

```
>>> print("World")
```

HelloWorld

Custom ending:

```
>>> print("Hello", end=" --- ")
```

```
>>> print("World")
```

Hello --- World

Customizing String Output and Text Formatting in Python

Changing the Separator Between Items

You can change the default space between printed items using the `sep` parameter.

Example – Custom separators:

```
>>> print("2025", "07", "17", sep="-")
```

2025-07-17

Escape Characters for Special Formatting

Escape characters are used to insert characters that are otherwise hard to type or display.

Table 13: Common escape sequences

Escape	Description
\n	Newline
\t	Tab
\\	Backslash
\"	Double quote

Example – Using escape characters:

```
>>> print("Line1\nLine2")

Line1

Line2

>>> print("She said, \"Python is cool!\")

She said, "Python is cool!"
```

Combining Strings with the + Operator

You can join strings using +, but all parts must be strings. Use str() to convert numbers before combining.

Example – String concatenation:

```
>>> name = "Alice"

>>> print("Hello, " + name + "!")

Hello, Alice!
```

Example – Using escape characters:

```
>>> print("Line1\nLine2")

Line1

Line2

>>> print("She said, \"Python is cool!\")

She said, "Python is cool!"
```

Combining Strings with the + Operator

You can join strings using +, but all parts must be strings. Use `str()` to convert numbers before combining.

Example – String concatenation:

```
>>> name = "Alice"
>>> print ("Hello, " + name + "!")
Hello, Alice!
>>> age = 25
>>> print ("Age: " + str(age))
Age: 25
```

2.3.3 Formatting Numbers with `format()` and f-string

Python offers flexible formatting tools like `format()` and f-strings to customize number displays.

Example – Controlling decimal places:

```
>>> value = 123.456789
>>> print("Value is {:.2f}".format(value))
Value is 123.46
>>> print(f"Value is {value:.2f}")
Value is 123.46
```

Scientific Notation

Use `e` or `E` to display numbers in scientific format.

```
>>> num = 0.000456
>>> print(f"{num:.2e}")
4.56e-04
>>> print(f"{num:.3E}")
4.560E-04
```

Adding Commas for Readability

Add commas to separate thousands in large numbers.

```
>>> population = 1250000000
```

```
>>> print(f'{population:,}')
```

```
1,250,000,000
```

```
>>> revenue = 9876543.21
```

```
>>> print(f'${revenue:,.2f}')
```

```
$9,876,543.21
```

Program Example – Display currency with commas:

```
monthly_pay = 5000.0
```

```
annual_pay = monthly_pay * 12
```

```
print('Your annual pay is $', format(annual_pay, ',.2f'), sep='')
```

```
# Output:
```

```
# Your annual pay is $60,000.00
```

Field Width and Text Alignment

Use format specifiers to align numbers or set minimum space width.

Examples – Field width:

```
>>> print(f'{42:5}')    # Right-aligned
```

```
42
```

```
>>> print(f'{3.14:10.2f}')
```

```
3.14
```

Aligning items in columns:

```
>>> print(f'{Item':<10} {'Price':>10}')
```

```
Item      Price
```

```
>>> print(f'{Apple':<10}{1.5:>10.2f}')
```

Apple 1.50

Formatting as Percentages

To display a number as a percentage, multiply by 100 and use % format specifier.

```
>>> ratio = 0.875
```

```
>>> print(f"{ratio:.0%}")
```

88%

```
>>> print(f"{ratio:.2%}")
```

87.50%

Integer Formatting (Base and Padding)

You can format integers in various ways such as binary, hexadecimal, or padded with zeros.

```
>>> num = 42
```

```
>>> print(f"{num:5d}")    # Right-align
```

42

```
>>> print(f"{num:05d}")    # Zero-padding
```

00042

```
>>> print(f"{num:b}")    # Binary
```

101010

```
>>> print(f"{num:x}")    # Hexadecimal (lowercase)
```

2a

```
>>> print(f"{num:X}")    # Hexadecimal (uppercase)
```

2A

Table 14: summary Table of Format Specifiers

Specifier	Meaning
.2f	2 decimal places (fixed-point)
[:,.2f	Comma separator + 2 decimals
.2%	Percentage with 2 decimals
:05d	Integer with zero padding

:b	Binary format
:x / :X	Hexadecimal (lowercase / uppercase)
.2e / .2E	Scientific notation

2.3.4 Named Constants

A named constant is an identifier (name) that is bound to a value that should not change during the execution of a program. In other words, a named constant represents a fixed value and provides a meaningful name to that value.

Unlike variables, whose values can change, constants remain the same throughout the program's runtime.

Why Use Named Constants?

1. Improves Code Readability

Consider the line:

```
amount = balance * 0.069
```

This line is ambiguous. Is 0.069 an interest rate, a fee, or something else?

Using a named constant:

```
INTEREST_RATE = 0.069
```

```
amount = balance * INTEREST_RATE
```

This version is clearer and tells readers exactly what the number represents.

2. Avoids Magic Numbers

A **magic number** is a hardcoded numeric value whose purpose is not immediately obvious. Magic numbers reduce code clarity and maintainability.

Avoid this:

```
price = quantity * 0.07
```

Use this instead:

```
TAX_RATE = 0.07
```

```
price = quantity * TAX_RATE
```

3. Easier Maintenance and Updates

When the constant value appears in multiple places, you only need to change it **once** in the constant declaration, instead of updating every occurrence.

```
INTEREST_RATE = 0.072 # Changed once
```

4. Helps Prevent Typographical Errors

If you mistype a number, it could go unnoticed:

```
amount = balance * .0069 # Wrong value!
```

If you mistype a constant name, Python throws an error:

```
amount = balance * INTRST_RATE # NameError!
```

This allows you to **catch bugs early**.

How to Define a Named Constant in Python

Python does not have built-in support for constants (like `const` in C++), but by **convention**, a named constant is:

- Written in uppercase letters
- Optionally separated by underscores (`_`) for readability

```
INTEREST_RATE = 0.069
PI = 3.14159
MAX_USERS = 100
```

Best Practices for Using Named Constants:

- **Use UPPERCASE names** – Capital letters clearly indicate a constant (e.g., `PI`, `MAX_RETRIES`)
- **Use descriptive names** – Choose meaningful identifiers like `TAX_RATE` or `DISCOUNT_PERCENT`
- **Group constants together** – Declare all constants at the **top of your script** for easy visibility and maintenance
- **Avoid reassignment** – Once defined, **do not change** the value of a constant during program execution

Example in Python Interaction Mode

```
>>> INTEREST_RATE = 0.069
>>> balance = 10000
>>> amount = balance * INTEREST_RATE
>>> amount
690.0
```

Now, if the interest rate changes:

```
>>> INTEREST_RATE = 0.072
>>> amount = balance * INTEREST_RATE
```

```
>>> amount  
720.0
```

Just **one change** in INTEREST_RATE updates its effect everywhere.

In-Text Questions (ITQs)

1. How do Python's arithmetic, comparison, and logical operators behave in expressions?
2. What is the difference between `is` and `==`, and how do assignment operators like `+=` work?
3. How does operator precedence influence expression evaluation, and when should parentheses be used?
4. How can algebraic expressions be accurately written in Python code?

In-Text Answers (ITAs)

1. Arithmetic operators perform calculations (`**`, `%`, `//`), comparison operators return Boolean results, and logical operators (`and`, `or`, `not`) combine or invert conditions.
2. `==` checks value equality, while `is` checks memory identity; assignment operators such as `+=` update a variable by applying an operation to its current value.
3. Operator precedence determines which operations run first—`**` is evaluated before `*`, `/`, `+`, and `-`; parentheses can be used to force a different evaluation order.
4. Algebraic expressions are written using Python syntax, such as `x = (a + b) / c` or `area = math.pi * r ** 2`, ensuring operations follow correct mathematical structure.

2.3.5 Conclusion

In this unit, you have learned to use Python operators such as arithmetic, comparison, logical, identity, and assignment used to build expressions and perform computations. You explored operator precedence, parentheses, converting algebraic

expressions into Python syntax, using named constants for clarity, and formatting outputs for readability.

Summary

In this unit, you have learnt that Python operators have specific functions and applications, and that operator precedence affects expression results unless overridden with parentheses. You also learnt how to translate algebraic formulas into Python code, use named constants for cleaner programs, and write Python programs that perform real-world calculations with well-formatted outputs.

Self-Assessment Questions

1. Differentiate between the assignment operator = and the comparison operator ==.
2. List five Python operators and classify them as arithmetic, comparison, logical, identity, or assignment.
3. Explain with an example how operator precedence affects the outcome of an expression.
4. What is the purpose of using parentheses in a complex expression? Provide a code example.
5. Why are named constants recommended in programming? Give two examples of how they are used.

Unit Activities

1. Write Python Programs Using Operators and Conditions:

Create short programs that evaluate arithmetic expressions (with and without parentheses) and check whether a user-entered number is even and positive.

2. Apply Constants and Comparison Logic in Simple Scripts:

Define a named constant (e.g., tax rate) to compute tax on a user's input, and write a program that compares two numbers to determine if they are equal.

Tutor-Marked Assignment

1. Write a Python program to evaluate:

$$z = (4 + 3) * 2 - 5 * 2 ** 2 + 18 // 3$$

Show operator precedence with intermediate steps.

2. Write a Python program to:

- Prompt the user for the radius.
- Calculate and display the area and perimeter using π (pi) and formatted output.

3. Write a Python program to:

- Input total seconds.
- Convert and display hours, minutes, and seconds using `//` and `%`.

References/Further Readings

Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

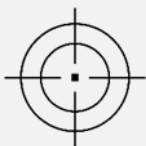
MODULE Three

Control Flow and Data Structures

Unit 1 — Decision Making in Python

INTRODUCTION

In this unit, you will learn about the control flow mechanism of decision-making in Python. A control flow mechanism is essential for creating programs that can choose between multiple paths of execution based on conditions. You will be introduced to the if, if-else, if-elif-else, and nested if statements, as well as Boolean expressions, relational operators, and string comparisons. These decision structures allow programs to respond dynamically to data and user input.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Describe control structures and differentiate between sequence and decision structures.
- Construct and implement simple and dual-alternative decision structures using if and if-else.
- Evaluate and apply Boolean expressions and relational operators in conditional statements.
- Develop nested decision structures to handle multiple levels of conditions.
- Use if-elif-else structures effectively to simplify complex branching logic.

3.1.1 What are Control Structures in Python?

A control structure is a logical design that controls the order in which a set of statements executes.

So far, we have used only the simplest type of control structure: the sequence structure. A sequence structure is a set of statements that execute in the order in which they appear. For example, the following code is a sequence structure because the statements execute from top to bottom:

```
name = input('What is your name? ')
age = int(input('What is your age? '))
print('Here is the data you entered:')
print('Name:', name)
print('Age:', age)
```

Although sequence structures are common in programming, they cannot handle every problem. Some tasks require decisions, such as a payroll program that adds overtime pay only if an employee works over 40 hours. These situations need a decision structure, which executes statements only when specific conditions are met.

Simple if structure

The if statement is used to create a decision structure, which allows a program to have more than one path of execution. The if statement causes one or more statements to execute only when a Boolean expression is true.

In its simplest form, a decision structure performs an action only when a condition is true; otherwise, the action is skipped. The flowchart in Figure 5 illustrates this logic using a diamond symbol for the true or false condition, directing the flow to either perform or skip the action.

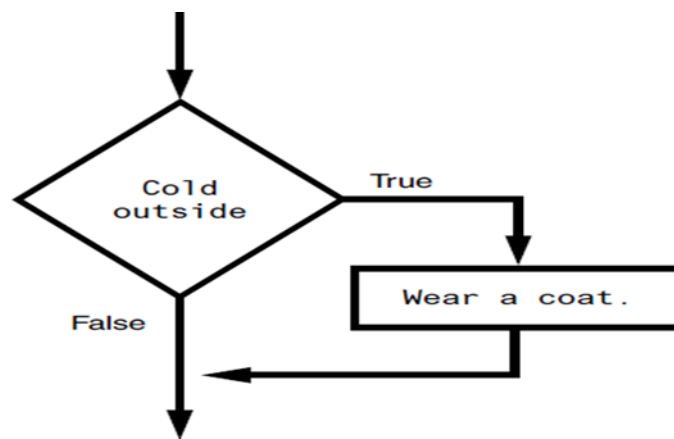


Figure 5: A simple decision structure

In the flowchart, the diamond symbol represents a condition to be tested. Here, it checks whether **Cold outside** is true or false. If true, the action **Wear a coat** is performed; if false, it is skipped. This action is conditionally executed because it occurs only when the condition is true. The structure shown in Figure 5 is a single alternative decision structure, meaning it provides one possible path of execution when the condition is true; otherwise, the program continues normally.

In Python, we use the **if** statement to write a single alternative decision structure. Here is the general format of the if statement:

```
if condition:
    statement
    statement
    etc.
```

For simplicity, we will call the first line the if clause. It begins with if, followed by a condition that evaluates to either true or false. A colon follows the condition, and the next line starts a block (a group of related statements). All statements in the block must be indented, as Python uses indentation to identify where the block begins and ends.

When the if statement runs, the condition is tested. If it is true, the block executes; if false, the block is skipped.

Figure 6 shows a more detailed example where three actions occur only when it is cold outside. It is still a single alternative decision structure because there is only one possible path of execution.

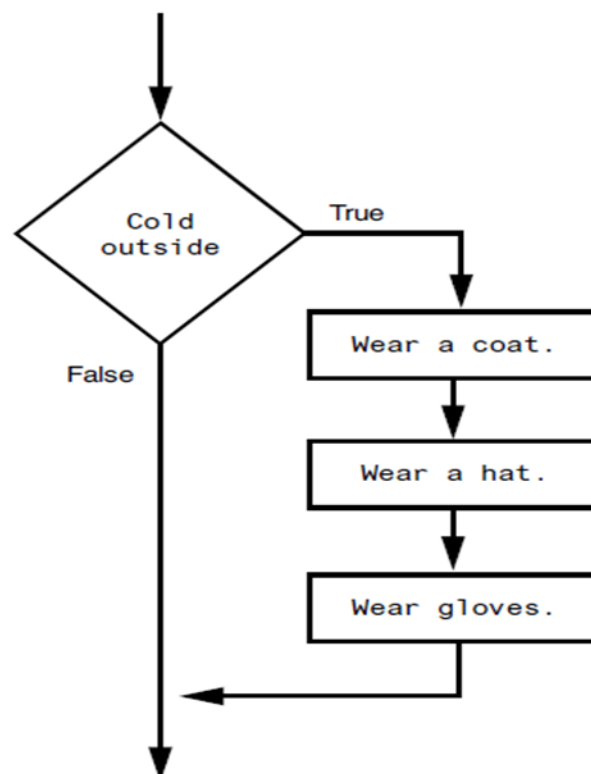


Figure 6: A simple decision structure (with three actions if the condition is true)

An if statement tests a Boolean expression to see if it is true or false. A Boolean expression usually contains a relational operator, which compares two values. For example, the greater than (>) operator checks if one value is greater, and the equal to (==) operator checks if two values are equal.

Let's look at the following example of the if statement:

```
if sales > 50000:
    bonus = 500.0
```

This statement checks if sales is greater than 50000 using the > operator. If true, bonus is set to 500.0; otherwise, the assignment is skipped. Figure 7 shows the flowchart for this code.

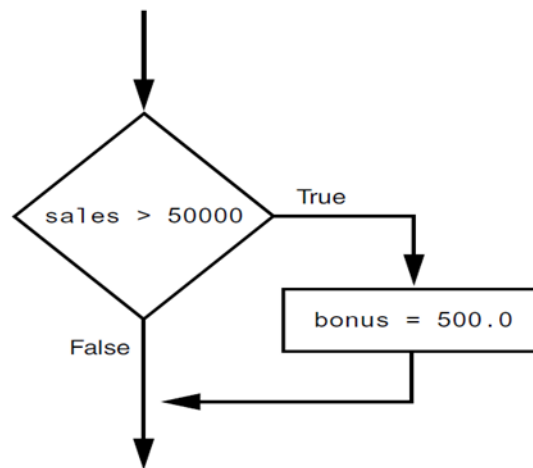


Figure 7: Example decision structure

The following example conditionally executes a block containing three statements.
Figure 8

shows a flowchart for this section of code:

```

if sales > 50000:
    bonus = 500.0
    commission_rate = 0.12
    print('You met your sales quota!')
  
```

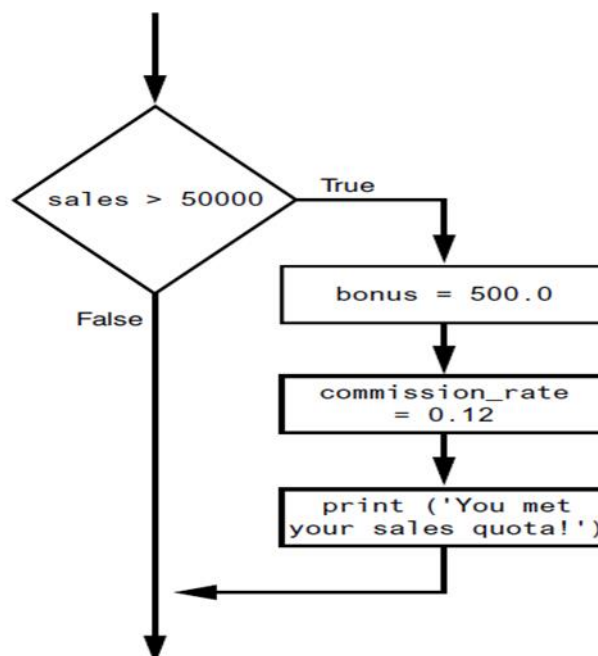


Figure 8: Example decision structure

The following code uses the `==` operator to determine whether two values are equal. The expression `balance == 0` will be true if the `balance` variable is assigned 0. Otherwise, the expression will be false.

```
if balance == 0:
```

```
# Statements appearing here will
```

```
# be executed only if balance is
```

```
# equal to 0.
```

The following code uses the `!=` operator to determine whether two values are not equal.

The expression `choice != 5` will be true if the choice variable does not reference the value 5. Otherwise, the expression will be false.

```
if choice != 5:
```

```
# Statements appearing here will
```

```
# be executed only if choice is
```

```
# not equal to 5.
```

If-else structure

An if-else statement will execute one block of statements if its condition is true, or another block if its condition is false. This is a dual alternative decision structure, which has two possible paths of execution—one path is taken if a condition is true, and the other path is taken if the condition is false. Figure 9 shows a flowchart for a dual alternative decision structure.

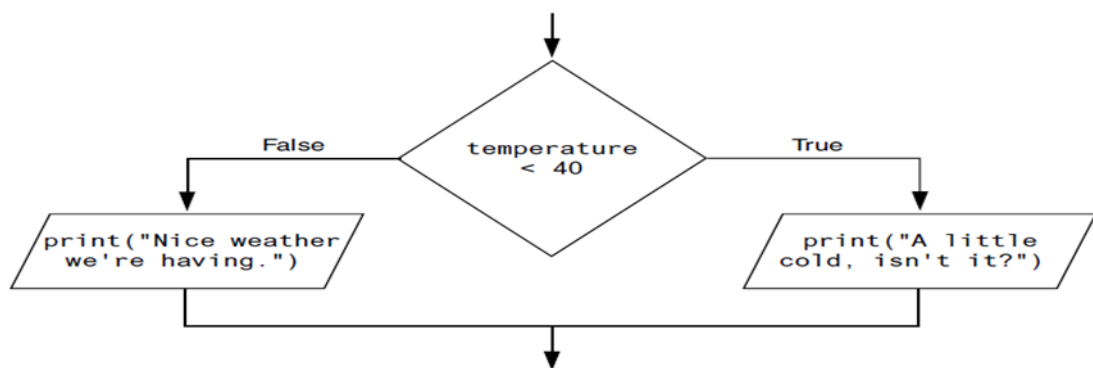


Figure 9: A dual alternative decision structure

The decision structure in the flowchart tests the condition `temperature < 40`. If this condition is true, the statement `print("A little cold, isn't it?")` is performed. If the condition is false, the statement `print("Nice weather we're having.")` is performed.

In code, we write a dual alternative decision structure as an if-else statement. Here is the general format of the if-else statement:

```
if condition:
    statement
    statement
    etc.
else:
    statement
    statement
    etc.
```

When this statement runs, the condition is tested. If true, the indented block after the **if** clause executes; otherwise, the block after the **else** clause runs. In both cases, control then moves to the next statement after the if else structure, as shown in Figure 10.

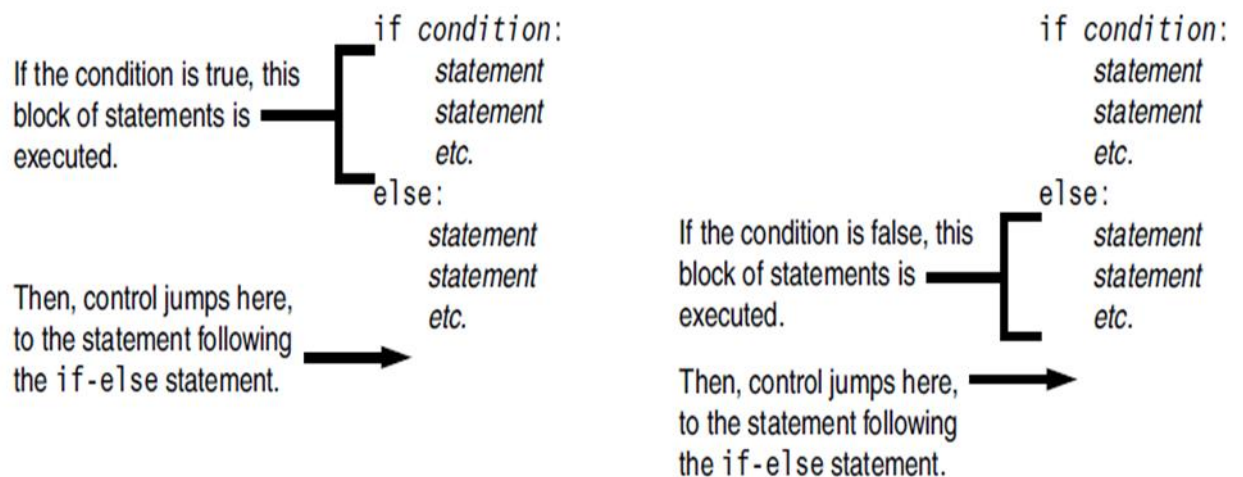


Figure 10: Conditional execution in an if-else statement

The following code shows an example of an if-else statement. This code matches the flowchart that was shown in Figure 9.

```
if temperature < 40:
    print("A little cold, isn't it?")
else:
    print("Nice weather we're having.")
```

Indentation in the if-else Statement

When you write an if-else statement, follow these guidelines for indentation:

Make sure the if clause and the else clause are aligned.

The if clause and the else clause are each followed by a block of statements. Make sure the statements in the blocks are consistently indented. This is shown in Figure 11.

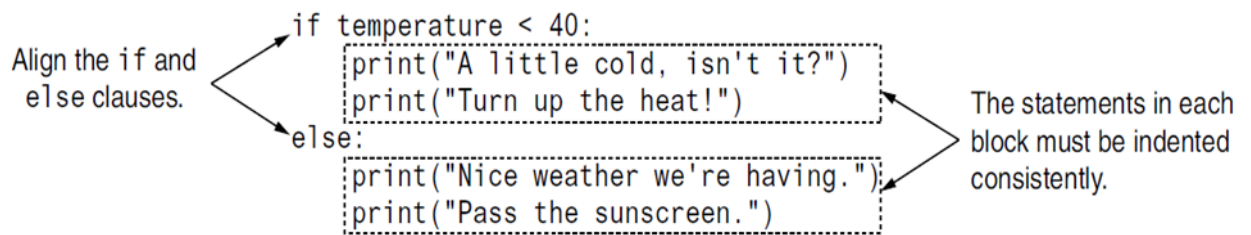


Figure 11: Indentation with an if-else statement

Python lets you compare strings to create decision structures that test string values, just as you can compare numbers. For example, consider the code below.:

```

name1 = 'Mary'
name2 = 'Mark'
if name1 == name2:
    print('The names are the same.')
else:
    print('The names are NOT the same.')
  
```

The `==` operator compares **name1** and **name2** to determine whether they are equal. Because the strings **'Mary'** and **'Mark'** are not equal, the else clause will display the message **'The names are NOT the same.'**

Let's look at another example. Suppose the variable **month** holds a string. The code below uses the **!=** operator to check if its value **is not equal to 'October'**:

```

if month != 'October':
    print('This is the wrong time for Octoberfest!')
  
```

The password program below demonstrates comparing two strings by prompting the user for a password and checking if it matches 'prospero'.

Program (password.py):

1. # This program compares two strings.
2. # Get a password from the user.
3. password = input('Enter the password: ')
4. # Determine whether the correct password
5. # was entered.
6. if password == 'prospero':
7. print('Password accepted.')

8. else:
9. print('Sorry, that is the wrong password.')

Program Output (with input shown in bold):

```
Enter the password: ferdinand Enter  
Sorry, that is the wrong password.  
Program Output (with input shown in bold)  
Enter the password: prospero Enter  
Password accepted.
```

String comparisons are case sensitive. For example, 'saturday' and 'Saturday' are not equal because the first has a lowercase "s" and the second an uppercase "S". The sample session with the password program above shows what happens when the user enters Prospero as the password.

```
Program Output (with input shown in bold)  
Enter the password: Prospero Enter  
Sorry, that is the wrong password.
```

Nested if structure

To test multiple conditions, a decision structure can be placed inside another. Programs are typically built from combinations of control structures, as shown in Figure 12, which combines a decision structure with two sequence structures.

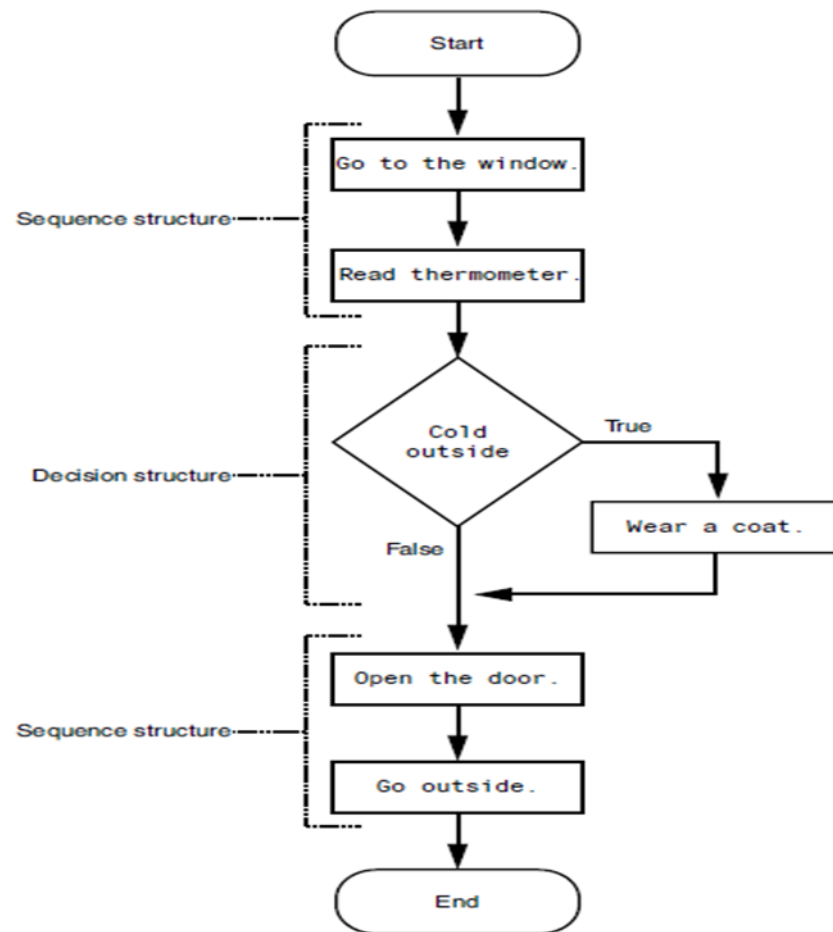


Figure 12: Combining sequence structures with a decision structure

The flowchart begins with a sequence structure. With an outdoor thermometer in your window, the first step is go to the window, followed by read thermometer. A decision structure then tests if it is cold outside. If true, wear a coat. Another sequence structure follows: open the door, then go outside.

Structures are often nested. For example, a decision structure can contain a sequence structure. If the condition cold outside is true, the steps in the sequence are executed as shown in Figure 13.

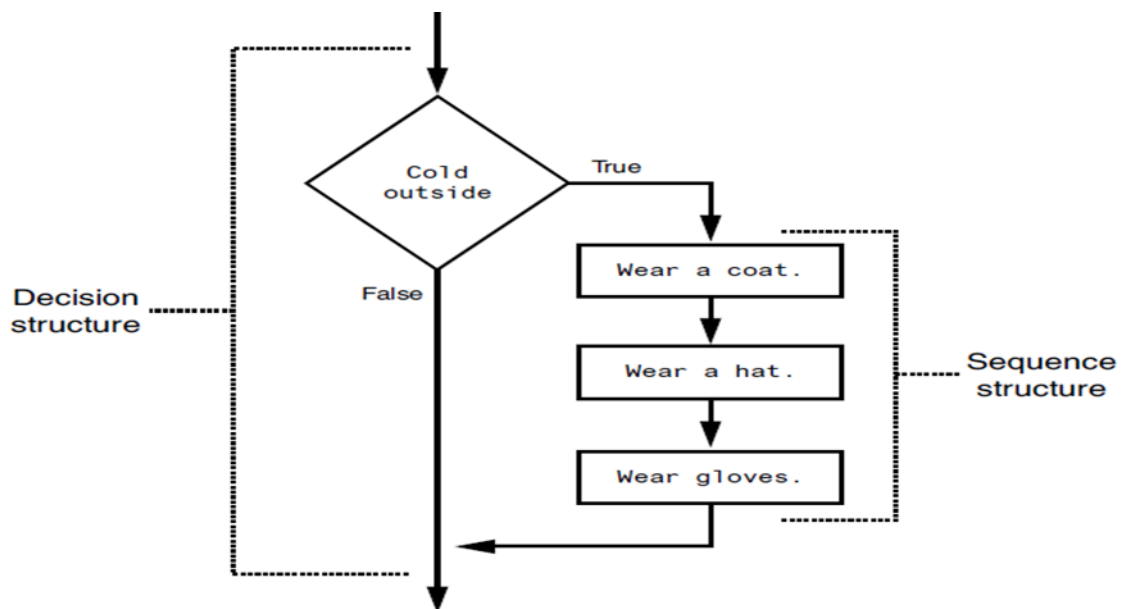


Figure 13: A sequence structure nested inside a decision structure

Decision structures can be nested inside others, which is common when a program must test multiple conditions. For example, a program determining if a bank customer qualifies for a loan may require two conditions to be met:

- 1) The customer must earn at least \$30,000 per year, and
- 2) The customer must have been employed for at least two years.

Figure 14 shows a flowchart for an algorithm in such a program. The salary variable holds the customer's annual salary, and years_on_job holds the number of years the customer has worked in their current job.

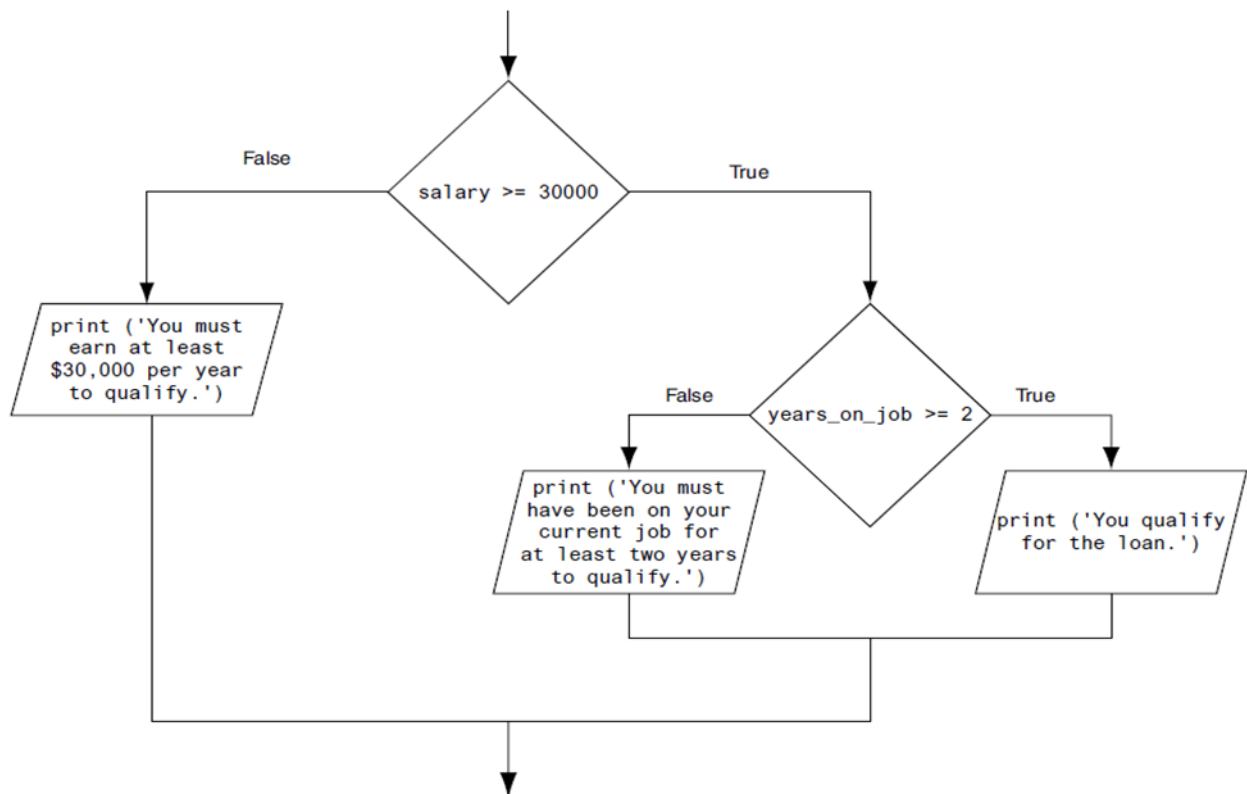


Figure 14: A nested decision structure

Following the execution flow, the program first tests if `salary >= 30000`. If false, the customer does not qualify for the loan. If true, it tests `years_on_job >= 2` using a nested decision. If this is true, the customer qualifies; otherwise, they do not. The `loan_qualifier` shows the complete code.

Program (loan_qualifier.py):

```

# This program determines whether a bank customer
# qualifies for a loan.
MIN_SALARY = 30000.0 # The minimum annual salary
MIN_YEARS = 2        # The minimum years on the job
# Get the customer's annual salary.
salary = float(input('Enter your annual salary: '))

# Get the number of years on the current job.
years_on_job = int(input('Enter the number of years employed:
'))

# Determine whether the customer qualifies.
if salary >= MIN_SALARY:
    if years_on_job >= MIN_YEARS:
        print('You qualify for the loan.')
    else:
        print('You must have been employed for at least',
MIN_YEARS, 'years to qualify.')
else:

```

```
print('You must earn at least $', format(MIN_SALARY,
',.2f'), ' per year to qualify.', sep='')
```

Program Output (with input shown in bold)

Enter your annual salary: **35000 Enter**

Enter the number of years employed: **1 Enter**

You must have been employed for at least 2 years to qualify.

Program Output (with input shown in bold)

Enter your annual salary: **25000 Enter**

Enter the number of years employed: **5 Enter**

You must earn at least \$30,000.00 per year to qualify.

Program Output (with input shown in bold)

Enter your annual salary: **35000 Enter**

Enter the number of years employed: **5 Enter**

You qualify for the loan.

The if-else statement on line 11 tests whether **salary** **>= MIN_SALARY**. If true, the nested if-else starting on line 13 runs; otherwise, the program executes the else clause on line 18, covering lines 19 to 21.

Proper indentation in nested decisions is essential. Python requires it, and it helps you see which actions belong to each part of the structure.

Rules for nested if statements:

- Align each else clause with its matching if clause.
- Indent all statements in a block consistently

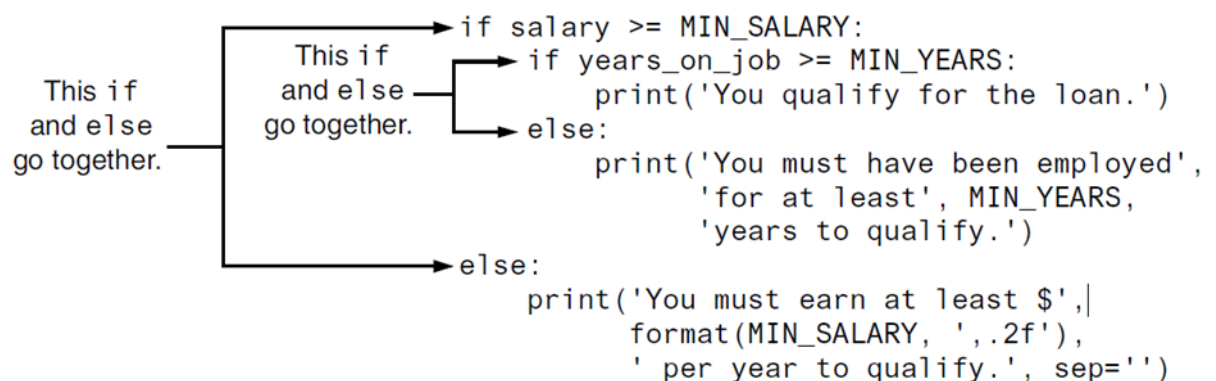


Figure 15: Alignment of if and else clauses

```

if salary >= MIN_SALARY:
    if years_on_job >= MIN_YEARS:
        print('You qualify for the loan.')
    else:
        print('You must have been employed',
              'for at least', MIN_YEARS,
              'years to qualify.')
else:
    print('You must earn at least $',
          format(MIN_SALARY, ',.2f'),
          ' per year to qualify.', sep='')

```

Figure 16: Example of a nested block

If-elif structure

The logic of the nested decision structure is fairly complex. Python provides a special version of the decision structure known as the **if-elif-else** statement, which makes this type of logic simpler to write. Here is the general format of the **if-elif-else** statement:

```

if condition_1:
    statement
    statement
    etc.
elif condition_2:
    statement
    statement
    etc.
Insert as many elif clauses as necessary . .
else:
    statement
    statement
    etc.

```

When the statement executes, **condition_1** is tested. If true, its following block runs and the rest is skipped. If false, the program tests each subsequent elif condition in order, executing the first true block and ignoring the rest. If no conditions are true, the else block is executed.

The following is an example of the **if-elif-else** statement.

```

if score >= A_SCORE:
    print('Your grade is A.')
elif score >= B_SCORE:
    print('Your grade is B.')
elif score >= C_SCORE:

```

```
        print('Your grade is C.')
    elif score >= D_SCORE:
        print('Your grade is D.')
    else:
        print('Your grade is F.')
```

Notice the alignment and indentation in if, elif, and else clauses, with executed blocks properly indented. While the logic can be implemented using nested if-else statements, long chains can become complex, hard to read, and may require scrolling or line wrapping. If-elif-else statements are generally easier to follow and keep lines shorter.

In-Text Questions (ITQs)

1. What is the default control structure in Python, and why might it be insufficient for some problems?
2. How does an if statement work, and what type of expression does it evaluate?
3. What is the purpose of if-else and if-elif-else structures in Python?
4. Why and when are nested if statements used in Python programs?

In-Text Answers (ITAs)

1. Sequence Structure: Code executes top to bottom by default, but some problems need conditional execution.
2. If Statement: Executes its block only when a Boolean condition is True.
3. If-Else / If-Elif-Else: If-else handles two outcomes; if-elif-else checks multiple conditions, executing else only if none are True.
4. Nested If: Tests additional conditions within a True block, allowing multi-level decision-making.

3.1.2 Conclusion

In this unit, you learned Python's decision-making structures—if, if-else, if-elif-else, and nested if—and how Boolean expressions, relational operators, and case-sensitive comparisons enable programs to respond dynamically to different inputs.

Summary

In this unit, you learned how decision structures control program flow in Python. You practiced using if, if-else, and if-elif-else statements, evaluated Boolean expressions, and applied nested conditions to write flexible and intelligent programs.

Self-Assessment Questions

1. Differentiate between sequence and decision structures in Python and give one example of each.
2. Write an if statement that checks if a number is positive and prints "Positive number" if true.
3. Explain how an if-else statement works and write a program that prints "Adult" if age ≥ 18 , otherwise "Minor."
4. Describe the use of if-elif-else statements and write a program that prints grades A–F based on a user's score.

Unit Activities

1. Write a program that checks a user's age and prints "Adult" if 18 or older, otherwise "Minor".
2. Create a password validation program that prints "Welcome" if the input matches 'admin123', else prints "Access Denied".
3. Develop a grading or loan qualification program using if-elif-else statements to handle multiple conditions and provide appropriate messages).

Tutor-Marked Assignment

1. Work Pay Calculator: Write a Python program to calculate pay based on hours worked and hourly rate. Include overtime ($1.5 \times$ rate) for hours > 40 , otherwise regular pay, and display the result formatted as currency.
2. Nested Loan Qualification: Write a Python program using nested if statements to check loan eligibility. Applicant must earn $\geq \text{N}30,000$ and have ≥ 2 years of work experience. Print "Qualified for loan" or an appropriate message if conditions are not met.

3. Grading System: Write a Python program that assigns grades based on numeric scores using if-elif-else:
- A: ≥ 90 , B: ≥ 80 , C: ≥ 70 , D: ≥ 60 , F: < 60 .
 - Display the corresponding grade message

References/Further Readings

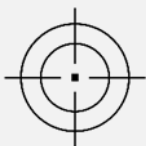
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media.

Unit 2 — Repetitive Structure: Loops and Branching in Python

INTRODUCTION

In this unit, you will learn about loops and branching in Python. Loops allow a statement or set of statements to execute repeatedly, reducing code duplication and improving efficiency. You will explore condition-controlled loops (while), count-controlled loops (for), nested loops, sentinel-controlled loops, and loop control statements (break, continue, pass). These concepts enable you to write flexible and intelligent programs that can handle repetitive tasks efficiently.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Describe control structures and differentiate between sequence and decision structures.
- Construct and implement simple and dual-alternative decision structures using if and if-else.
- Evaluate and apply Boolean expressions and relational operators in conditional statements.
- Develop nested decision structures to handle multiple levels of conditions.
- Use if-elif-else structures effectively to simplify complex branching logic.

3.2.1 What are Repetitive structures in Python

A repetition structure causes a statement or set of statements to execute repeatedly.

Programmers often need to write code that repeats the same task. For example, to calculate a 10 percent sales commission for several salespeople, one could write the code for one salesperson and then repeat it for others, though this is poor design.

There are several disadvantages to this approach, including the following:

- The duplicated code makes the program large.
- Writing a long sequence of statements can be time consuming.
- If part of the duplicated code has to be corrected or changed, then the correction or change has to be done many times.

Instead of repeating the same statements, you can write the operation once and place it in a structure that makes the computer repeat it as needed, called a loop.

3.2.2 Condition-Controlled and Count-Controlled Loops

There are two broad categories of loops: condition-controlled and count-controlled.

A condition-controlled loop uses a true/false condition to control the number of times that it repeats. A count-controlled loop repeats a specific number of times.

In Python, you use the while statement to write a condition-controlled loop, and you use the for statement to write a count-controlled loop.

The while Loop: A Condition-Controlled Loop

A condition-controlled loop causes a statement or set of statements to repeat as long as a condition is true. In Python, you use the while statement to write a condition-controlled loop.

The while loop gets its name from the way it works: while a condition is true, do some tasks. The loop has two parts:

- a condition that is tested for a true or false value, and
- a statement or set of statements that is repeated as long as the condition is true.

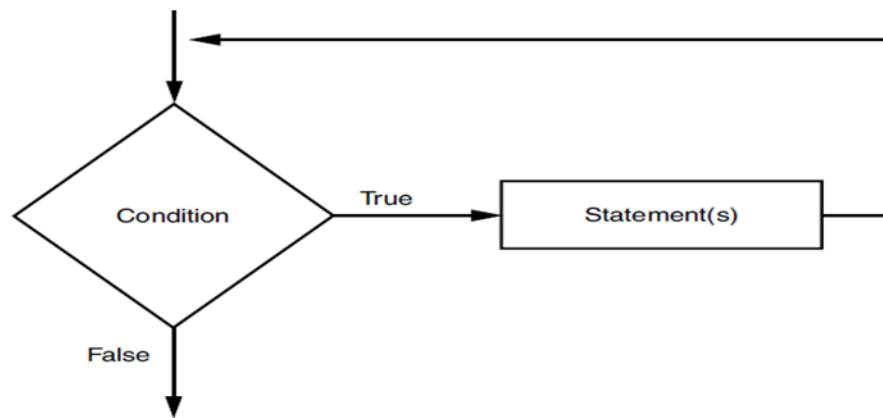


Figure 17: The logic of a while loop

The diamond symbol represents a condition. If true, statements are executed, then the program returns to test the condition again, repeating the process. If false, the program exits the loop. In a flowchart, a loop is indicated by a flow line returning to an earlier point.

Here is the general format of the while loop in Python:

```

while condition:
    statement
    statement
    etc.
  
```

The first line is the while clause, starting with `while` followed by a Boolean condition and a colon. The next indented lines form a block of statements. When the loop runs, the condition is tested: if true, the block executes and the loop repeats; if false, the loop ends.

The for Loop: A Count-Controlled Loop

A count-controlled loop iterates a specific number of times. In Python, you use the for statement to write a count-controlled loop.

A count-controlled loop runs a specific number of times and is commonly used in programs. For example, if a business is open six days a week, a program to calculate weekly sales would need a loop that iterates six times, prompting the user to enter daily sales each time. In Python, the **for** statement is used to create a count-controlled loop and works with a sequence of data items, iterating once for each item. The general format is:

```

for variable in [value1, value2, etc.]:
    statement
    statement
    etc.
  
```

variable is the name of a variable. Inside the brackets is a sequence of values separated by commas. In Python, this comma-separated sequence of items in brackets is called a list. The block of statements below the for loop runs each time the loop iterates.

The **for** loop works as follows: variable is assigned the first value in the list, and the block executes. Then variable is assigned the next value, and the block runs again. This continues until variable has taken the last value in the list.

The simple loop1 program below shows a simple example that uses a for loop to display the numbers 1 through 5.

Program (simple_loop1.py):

```
1 # This program demonstrates a simple for loop
2 # that uses a list of numbers.
3 print('I will display the numbers 1 through 5.')
4 for num in [1, 2, 3, 4, 5]:
5     print(num)
```

Program Output:

```
I will display the numbers 1 through 5.
1
2
3
4
5
```

The first time the loop runs, num is 1 and line 6 displays 1. On the next iteration, num is 2 and line 6 displays 2. This continues until num reaches the last value. With five values in the list, the loop iterates five times.

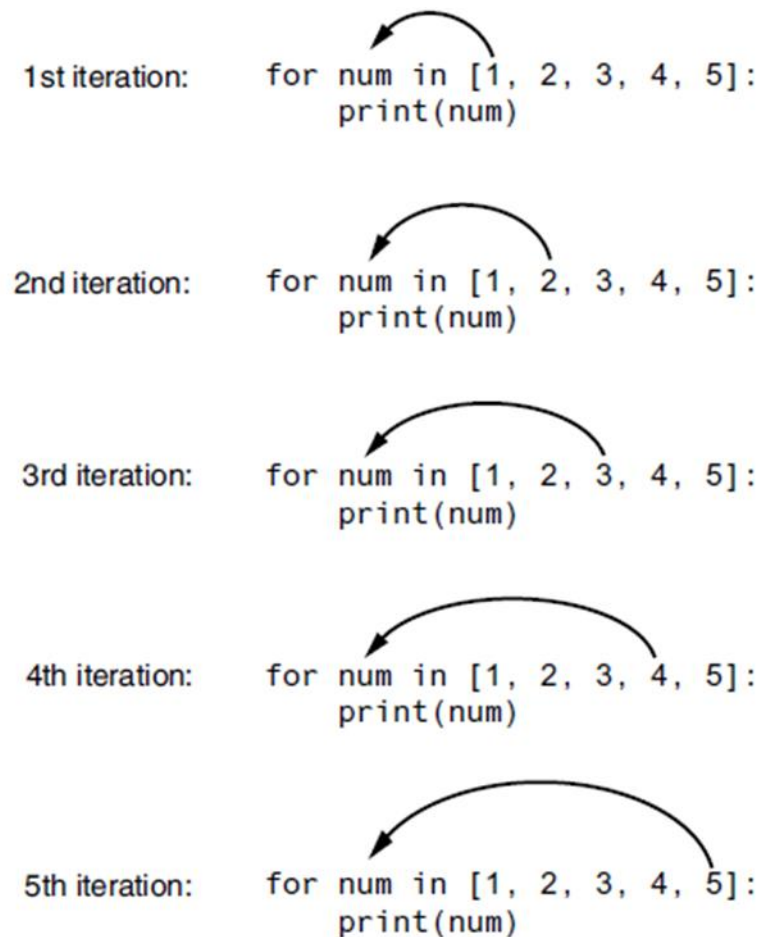


Figure 18: Example of the for loop

In Python, the variable in the for clause is called the target variable because it is assigned a value at the start of each loop iteration. The list values do not need to be consecutive numbers; for example, a loop can display odd numbers from [1, 3, 5, 7, 9]. A for loop can also iterate over a list of strings. For instance, a loop over ['Winken', 'Blinken', 'Nod'] iterates three times.

Program (simple_loop2.py):

```
# This program also demonstrates a simple for
# loop that uses a list of strings.
for name in ['Winken', 'Blinken', 'Nod']:
    print(name)
```

Program Output:

```
Winken
Blinken
Nod
```

Using the range Function with the for Loop

Python provides a built-in function, `range`, to simplify count-controlled for loops. The `range` function generates an iterable, an object similar to a list, containing a sequence of values that can be iterated with a loop. For example:

```
for num in range(5):  
    print(num)
```

This generates numbers from 0 up to (but not including) 5, equivalent to:

```
for num in [0, 1, 2, 3, 4]:  
    print(num)
```

Example program to print "Hello world" three times:

```
# simple_loop4.py  
for x in range(3):  
    print('Hello world')
```

Output:

```
Hello world  
Hello world  
Hello world
```

Range with Start, Stop, and Step

- **One argument:** ending limit of the sequence.

```
for num in range(5): # 0 to 4
```

- **Two arguments:** start and end values.

```
for num in range(1, 5): # 1 to 4
```

- **Three arguments:** start, end, and step value.

```
for num in range(1, 10, 2):  
    print(num)
```

Nested Loop

A nested loop is a loop inside another loop.

A clock illustrates this: the hour hand completes one revolution for every 12 revolutions of the minute hand, and the minute hand completes one revolution for every 60 revolutions of the second hand.

Example: a simple loop for seconds:

```
for seconds in range(60):  
    print(seconds)
```

Adding minutes with a nested loop:

```
for minutes in range(60):
    for seconds in range(60):
        print(minutes, ':', seconds)
```

Adding hours to complete the clock:

```
for hours in range(24):
    for minutes in range(60):
        for seconds in range(60):
            print(hours, ':', minutes, ':', seconds)
```

The innermost loop runs 60 times for each minute, the middle loop runs 60 times for each hour, and the outer loop runs 24 times. In total, the loops execute 86,400 times, simulating every second in a 24-hour day.

3.2.3 Sentinel-Controlled Loops in Python

A **sentinel** is a special value used to **signal the end of input** or **terminate a loop** when the number of iterations is **not known in advance**.

- It acts as a **flag** to stop looping.
- The sentinel value **must be different** from valid input values to avoid premature loop termination.

Why Not Use Fixed Loops?

- for loops require a known range, so they are unsuitable when the number of inputs is unknown.
- while True with break works, but sentinel-controlled loops are clearer and more semantic.

Python Example – Numeric Sentinel

```
total = 0
count = 0

print("Enter numbers to calculate the average.")
print("Enter -1 to finish.\n")

while True:
    number = float(input("Enter a number (-1 to stop):"))
    if number == -1: # Sentinel
        break
    total += number
    count += 1

if count > 0:
    average = total / count
```

```

        print(f"\nYou entered {count} numbers. Average =
{average}")
    else:
        print("\nNo numbers were entered.")

```

Explanation:

- The loop runs indefinitely using while True.
- Each input is checked:
 - If it matches the sentinel (-1), the loop ends.
 - Otherwise, the input is added to the total, and count is incremented.
- The average is calculated only if at least one number was entered.

Choosing a Sentinel Value:

- It should not be a valid input.
 - Positive integers → sentinel = -1
 - Strings → sentinel = 'done' or 'exit'
 - Names → sentinel = 'END'

3.2.4 Break, Continue, and Pass statements

Break Statement

The break statement is used to terminate the loop it is within prematurely. It is typically used to exit a loop when a certain condition is met, without executing the rest of the loop's statements.

Example of 'break' statement

while True:

```

    user_input = input("Enter a number (type 'exit' to quit): ")

```

```

    if user_input == 'exit':

```

```

        break # Exit the loop if user enters 'exit'

```

```

    number = int(user_input)

```

```

    print(f"Square of {number} is {number ** 2}")

```

Explanation:

This infinite while loop repeatedly asks the user for input. If the
user types 'exit', the break

statement immediately ends the loop. Otherwise, it converts the input to an integer and prints its square.

Continue Statement

The continue statement is used to skip the rest of the code inside a loop for the current iteration and continue with the next iteration.

Example of 'continue' statement

```
for i in range(1, 11):  
    if i % 2 == 0:  
        continue # Skip even numbers  
    print(f"Odd number: {i}")
```

Explanation:

This for loop iterates over numbers from 1 to 10. If the number is even, continue skips the print() and goes to the next iteration. So, only odd numbers are printed.

Pass Statement

The pass statement is a null operation; nothing happens when it executes. It is used as a placeholder where syntactically a statement is required, but you have nothing to execute.

Example of 'pass' statement

```
def placeholder_function():  
    pass # Placeholder for future implementation  
  
# Since 'pass' does nothing, calling the function will not produce output  
placeholder_function()
```

Explanation:

This defines an empty function. The pass statement acts as a placeholder, preventing syntax errors and allowing the function to exist without performing any actions. It is also useful for controlling loop flow and creating placeholder functions or classes, making code more readable and flexible.

In-Text Questions (ITQs)

1. What is a condition-controlled loop?
2. How does a count-controlled loop differ from a condition-controlled loop?
3. What is a nested loop, and when is it useful?
4. What is a sentinel value, and why is it used?
5. Explain the difference between break and continue statements.

In-Text Answers (ITAs)

1. A loop that repeats as long as a condition is true (while loop).
2. A count-controlled loop repeats a fixed number of times, while a condition-controlled loop repeats based on a condition.
3. A loop inside another loop, used for multi-level iteration.
4. A special value used to stop a loop when the number of iterations is unknown.
5. break stops the loop entirely; continue skips to the next iteration.

3.2.5 Conclusion

In this unit, you have learned how to use loops and branching in Python to execute statements repeatedly. Understanding these structures allows you to write efficient, flexible, and dynamic programs.

Summary

In this unit, you have learnt that loops reduce code duplication and enable repetition. Condition-controlled (while) and count-controlled (for) loops allow tasks to be repeated based on conditions or counts. Nested loops, sentinel-controlled loops, and loop control statements (break, continue, pass) give additional flexibility in managing program flow.

Self-Assessment Questions

1. With the aid of an example, differentiate between while and for loops.
2. What is the purpose of a nested loop?
3. How does a sentinel-controlled loop differ from a fixed loop?

Unit Activities

1. Read Module 3 Unit 2 content and consult recommended texts (90 minutes).
2. Participate in the forum: original post by Thursday, replies to 2 classmates by Sunday (10 minutes each).
3. Complete the Tutor-Marked Assignment (60 minutes).

Tutor-Marked Assignment

Complete the following tasks in Python and submit your code for grading:

- Write a while loop that prints all even numbers from 2 to 20.
- Write a for loop to display the first 10 natural numbers.
- Create a nested loop to print a 12-hour clock showing hours and minutes.
- Write a sentinel-controlled loop to accept numbers from the user and calculate their average. Use -1 as the sentinel value.
- Demonstrate the use of break, continue, and pass in three separate small loop examples.

References/Further Readings

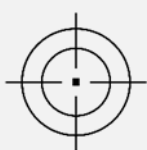
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

Unit 3 — Python Data Structures

INTRODUCTION

In this unit, you will learn about Python data structures, including sequences, lists, tuples, numbers, and strings. You will understand how to store, access, manipulate, and process data efficiently using these structures, which are fundamental for writing flexible and organised Python programs.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain and use Python sequences (lists and tuples).
- Perform operations on lists: append, insert, remove, sort, reverse, slicing, and concatenation.
- Use tuples and convert between lists and tuples.
- Work with Python numbers and perform arithmetic operations.
- Manipulate Python strings, including indexing, slicing, concatenation, and using string methods.

3.3.1 Sequences

A sequence is an object that holds multiple items of data, stored one after the other. You can perform operations on a sequence to examine and manipulate the items stored in it.

A sequence stores multiple items in order. Python supports various operations on sequences. Common types are lists and tuples. Both can hold different data types, but lists can be changed while tuples cannot.

Python Lists

A list is an object that contains multiple data items. Lists are mutable, which means that their contents can be changed during a program's execution.

Lists are dynamic data structures that allow adding or removing items. They are ordered, mutable collections that can store different data types, including other lists.

Key Concepts:

- Defined with square brackets [].
- Common operations: `append()`, `insert()`, `remove()`, `pop()`, `sort()`, `reverse()`.
- Support indexing and iteration.

A list is an object that holds multiple data items, called elements. For example:

```
even_numbers = [2, 4, 6, 8, 10]
```

The items inside the brackets, separated by commas, are the list elements. After this statement runs, the variable `even_numbers` references the list shown in Figure 19.

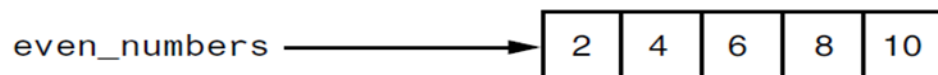


Figure 19: A list of integers

The following is another example:

```
names = ['Molly', 'Steven', 'Will', 'Alicia', 'Adriana']
```

This statement creates a list of five strings. After the statement executes, the name variable will reference the list as shown in Figure 20.

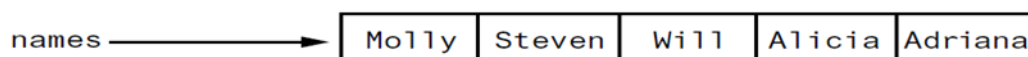


Figure 20: A list of strings

A list can store items of different types, as shown below:

```
info = ['Alicia', 27, 1550.87]
```

This creates a list containing a string, an integer, and a float. After execution, the variable `info` refers to the list shown in Figure 21.

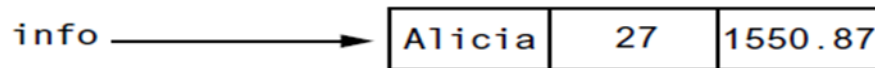


Figure 21: A list holding different types

Concatenating Lists

Concatenation joins two lists using the `+` operator. Only lists can be concatenated; using a non-list will raise an exception. Here is an example:

```
list1 = [1, 2, 3, 4]
list2 = [5, 6, 7, 8]
list3 = list1 + list2
print("Concatenated List:", list3)
```

Output:

Concatenated List: [1, 2, 3, 4, 5, 6, 7, 8]

Note: `list1` and `list2` remain unchanged.

List Slicing

Slicing allows you to extract a part (slice) of a list using a range of indices.

A slice is a portion of items taken from a sequence. Slicing a list returns elements from a specified start index up to, but not including, an end index. The general format is:

```
list_name[start : end]
```

Here, `start` is the index of the first element, and `end` marks the index just after the last element in the slice. For example, consider the following list:

```
days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday']
mid_days = days[2:5]
```

```
print("Mid-week days:", mid_days)
```

Output:

Mid-week days: ['Tuesday', 'Wednesday', 'Thursday']

Note: list[start:end] returns elements from start to end-1.

Finding Items with in Operator

In Python, use the in operator to check if an item is in a list. The general format is:

item in list

In this format, item is what you are searching for, and list is the collection. The expression returns true if the item is in the list, otherwise false. Here is an example:

```
names = ['Alice', 'Bob', 'Charlie']
```

```
print('Bob' in names)    # True
```

```
print('Diana' in names)  # False
```

Output:

True

False

Python lists provide many methods and built-in functions to work with their elements. These methods allow you to add, remove, or reorder elements and perform other manipulations. Below is a table of commonly used list methods with examples:

Table 15: A few of the list methods

Method	Description
<code>append(item)</code>	Adds an item to the end of the list.
<code>index(item)</code>	Returns the index of the first occurrence of item. Raises <code>ValueError</code> if not found.
<code>insert(index, item)</code>	Inserts item at the specified index.
<code>sort()</code>	Sorts the list in ascending order.
<code>remove(item)</code>	Removes the first occurrence of item. Raises <code>ValueError</code> if not found.
<code>reverse()</code>	Reverses the list in place.

The append Method

The append method is commonly used to add items to a list. The item that is passed as an argument is appended to the end of the list's existing elements. Below is an example:

```
fruits = ['apple', 'banana']  
  
fruits.append('orange')  
  
print(fruits)
```

Output:

```
['apple', 'banana', 'orange']
```

The index Method

The in operator checks if an item exists in a list, but sometimes the item's position is needed. The index method returns the index of the first occurrence of a specified item and raises a ValueError if the item is not found. The example in the program illustrates its use.

```
names = ['Alice', 'Bob', 'Charlie']  
  
position = names.index('Bob')  
  
print("Index of Bob:", position)
```

Output:

```
Index of Bob: 1
```

Raises ValueError if the item is not found.

The insert Method

The insert method adds an item to a list at a specific position. It takes two arguments: the index for insertion and the item to insert. The example below demonstrates this.

```
colors = ['red', 'blue', 'green']  
  
colors.insert(1, 'yellow') # Inserts at index 1  
  
print(colors)
```

Output:

```
['red', 'yellow', 'blue', 'green']
```

The sort Method

The sort method rearranges the elements of a list so they appear in ascending order (from the lowest value to the highest value). Here is an example:

```
numbers = [4, 1, 3, 2]

numbers.sort()

print("Sorted numbers:", numbers)
```

Output:

Sorted numbers: [1, 2, 3, 4]

Works with strings too: ['banana', 'apple'].sort() → ['apple', 'banana']

The remove Method

The remove method deletes the first occurrence of a specified item from a list, reducing its size by one. All elements after the removed item shift one position forward. A ValueError is raised if the item is not found. Here is an example:

```
animals = ['cat', 'dog', 'bird']

animals.remove('dog')

print(animals)
```

Output:

```
['cat', 'bird']
```

- Raises ValueError if item does not exist.

The reverse Method

The reverse method simply reverses the order of the items in the list. Here is an example:

```
numbers = [1, 2, 3, 4]

numbers.reverse()

print("Reversed list:", numbers)
```

Output:

Reversed list: [4, 3, 2, 1]

The del Statement

The remove method deletes a specific item from a list if it exists. To delete an element at a specific index, regardless of its value, use the del statement. Here is an example:

```
my_list = [1, 2, 3, 4, 5]

print('Before deletion:', my_list)

del my_list[2]

print('After deletion:', my_list)
```

This code will display the following:

Before deletion: [1, 2, 3, 4, 5]

After deletion: [1, 2, 4, 5]

The min and max Functions

Python has two built-in functions, min and max, that work with sequences. The min function takes a sequence, like a list, and returns the item with the lowest value. For example:

```
my_list = [5, 4, 3, 2, 50, 40, 30]

print('The lowest value is', min(my_list))
```

This code will display the following:

The lowest value is 2

The max function accepts a sequence, such as a list, as an argument and returns the item that has the highest value in the sequence. Here is an example:

```
my_list = [5, 4, 3, 2, 50, 40, 30]

print('The highest value is', max(my_list))
```

This code will display the following:

The highest value is 50

Copying List

To copy a list, you must copy its elements. In Python, assigning one variable to another makes both reference the same object in memory. For example, consider the following code:

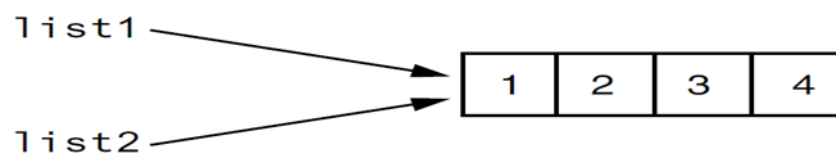
```
# Create a list.
```

```
list1 = [1, 2, 3, 4]
```

```
# Assign the list to the list2 variable.
```

```
list2 = list1
```

After this code executes, both variables `list1` and `list2` will reference the same list in memory.



Example:

```
# Working with lists
```

```
fruits = ["apple", "banana", "cherry"]
```

```
print("Original list:", fruits)
```

```
fruits.append("date")
```

```
print("After append:", fruits)
```

```
fruits.insert(1, "mango")
```

```
print("After insert at index 1:", fruits)
```

```
fruits.remove("banana")
```

```
print("After removing 'banana':", fruits)
```

```
print("Sorted list:", sorted(fruits))
```

Real-world Use: Used in data collection, student lists, inventory systems, and more.

3.3.2 Python Tuples

A tuple is an immutable sequence, which means that its contents cannot be changed. You create a tuple using parentheses ().

Key Concepts:

- Tuples are faster and more memory-efficient than lists.
- Ideal when data should not change (e.g., coordinates, fixed options).

Tuples support all the same operations as lists, except those that change the contents of the list. Tuples support the following:

- Subscript indexing (for retrieving element values only)
- Methods such as index
- Built-in functions such as len, min, and max
- Slicing expressions
- The in operator
- The + and * operators

Tuples do not support methods such as append, remove, insert, reverse, and sort.

A tuple is a sequence, very much like a list. The primary difference between tuples and lists is that tuples are immutable. That means once a tuple is created, it cannot be changed. When you create a tuple, you enclose its elements in a set of parentheses, as shown in the example below:

Example:

```
# Tuple usage
```

```
dimensions = (1920, 1080)
```

```
print("Width:", dimensions[0])
```

```
print("Height:", dimensions[1])
```

```
# Trying to change a value will raise an error:
```

```
# dimensions[0] = 1280 # TypeError
```

Real-world Use: Used for coordinates, dates, settings, and function return values.

Converting Between Lists and Tuples

In Python, the built-in `list()` function is used to convert a tuple into a list, while the `tuple()` function converts a list into a tuple. The following example illustrates this process:

```
# Convert a tuple to a list

number_tuple = (1, 2, 3)

number_list = list(number_tuple)

print(number_list) # Output: [1, 2, 3]

# Convert a list to a tuple

str_list = ['one', 'two', 'three']

str_tuple = tuple(str_list)

print(str_tuple) # Output: ('one', 'two', 'three')
```

Explanation of the statements:

- A tuple containing three integers is created and assigned to the variable `number_tuple`.
- The `list()` function is called with `number_tuple` as its argument, producing a list with the same elements. This list is stored in `number_list`.
- The list `number_list` is then printed, displaying `[1, 2, 3]`.
- A list of string elements is created and assigned to `str_list`.
- The `tuple()` function is used to convert `str_list` into a tuple, which is then assigned to `str_tuple`.
- Finally, `str_tuple` is printed, and the output is `('one', 'two', 'three')`.

This demonstrates how data structures can be easily converted between lists and tuples using simple built-in functions.

3.3.3 Python Numbers

Python has built-in support for working with numbers, making it easy to perform mathematical calculations. The two most commonly used numeric types in Python are:

- **Integers (int):** These are whole numbers without a decimal point, like 12, -5, or 0.

- Floating-point numbers (float): These are numbers that have a decimal point or are written in exponential (scientific) notation, like 3.5, 0.75, or 2.1e3 (which means 2.1×10^3 or 2100).

Key Concepts:

- Automatic Type Detection: Python automatically determines if a number is an integer or a float based on the value you assign. For example, 12 is an integer, while 3.5 is a float.
- Arithmetic Operations: Python supports standard arithmetic operations using these symbols:
 - + (Addition)
 - (Subtraction)
 - (Multiplication)
 - / (Division, always returns a float)
 - // (Integer division, returns the quotient without the remainder)
 - % (Modulus, returns the remainder of a division)
 - ** (Exponentiation, raises a number to the power of another)

Code Example Explained

```
# Integer and float usage

x = 12      # x is assigned the integer value 12

y = 3.5     # y is assigned the float value 3.5

# Perform and display various arithmetic operations

# Adds x and y → 12 + 3.5 = 15.5

print("Addition:", x + y)

# Subtracts y from x → 12 - 3.5 = 8.5

print("Subtraction:", x - y)

# Multiplies x and y → 12 * 3.5 = 42.0

print("Multiplication:", x * y)

# Divides x by y → 12 / 3.5 ≈ 3.42857 (float result)

print("Division:", x / y)

# Divides x by 2 and returns integer quotient → 12 // 2 = 6
```

```
print("Integer Division:", x // 2)

# Finds remainder of x divided by 5 → 12 % 5 = 2

print("Modulus (Remainder):", x % 5)

# Raises x to the power of 2 → 122 = 144

print("Exponentiation:", x ** 2)
```

Explanation

- Addition ($x + y$): Adds the integer and float, automatically converts the integer to float during addition.
- Subtraction ($x - y$): Subtracts y from x , returning a float because one operand is a float.
- Multiplication ($x * y$): Multiplies the two numbers, result is a float due to y .
- Division (x / y): Standard division, returns a float even if both operands are integers.
- Integer Division ($x // 2$): Divides and truncates (removes) the decimal part, returning an integer result.
- Modulus ($x \% 5$): Finds the remainder after dividing x by 5.
- Exponentiation ($x ** 2$): Raises x to the power of 2, i.e., squares x .

Real-world Use

Numbers and arithmetic operations are fundamental in many real-world applications:

- Calculating student grades and averages
- Processing payroll or salary calculations
- Performing financial computations like interest, loans, and budgeting
- Engineering simulations and scientific calculations
- Games and graphics programming for positions and scoring.

3.3.4 Python Strings

A string is a sequence of characters enclosed in quotes. You can use either single or double quotes. Strings are immutable, meaning their content cannot be changed after creation.

Python provides multiple ways to access individual characters and perform operations on strings. So far, most programs have focused on basic input and output, like

reading from the keyboard or displaying text. Real-world applications, however, often involve extensive string manipulation, such as in word processors, email clients, and search engines. Since strings are sequences in Python, operations like indexing, slicing, and iteration can be applied to manipulate them efficiently.

Key Concepts:

Strings are indexed starting from 0.

Strings support slicing, iteration, and many built-in methods like `.upper()`, `.lower()`, `.find()`, `.replace()`, and `.split()`.

Accessing the Individual Characters in a String

Some programming tasks require accessing individual characters in a string. For example, password rules often require at least one uppercase letter, one lowercase letter, and one digit. Programs check each character to ensure these rules are met. In this section, we'll explore two ways to access string characters in Python: for loops and indexing.

Iterating over a String with the for Loop

One of the easiest ways to access individual characters in a string is with the for loop. Here is the general format:

```
for variable in string:
    statement
    statement
```

In this format, `variable` is a placeholder for each character in the string, and `string` can be either a string literal or a variable that references a string. Each time the loop iterates, `variable` holds the next character in the string.

Example:

```
name = 'Juliet'
for ch in name:
    print(ch)
```

Output:

```
J
u
l
i
e
t
```

This loop iterates over each character in the string and displays them one by one.

Accessing Characters Using Indexing

Another way to access the characters in a string is through indexing. Each character in a string has an index, starting at 0.

Example:

```
message = 'Python'
print(message[0]) # Outputs: P
print(message[3]) # Outputs: h
```

If a string contains n characters, their indices range from 0 to $n-1$. Attempting to access an index outside this range results in an `IndexError`.

The len Function

The built-in `len()` function returns the number of characters in a string. This is especially useful when using loops to iterate through the string.

Example:

```
city = 'Boston'
size = len(city)
print('Length of city:', size)
```

Output:

```
Length of city: 6
```

You can also use `len()` in a while loop:

```
city = 'Boston'
index = 0
while index < len(city):
    print(city[index])
    index += 1
```

String Concatenation

A common operation performed on strings is concatenation, which means appending one string to another using the `+` operator. This operator returns a new string that combines the two operand strings.

```
greeting = 'Hello'
name = 'Alice'
message = greeting + ' ' + name
print(message)
```

Output:

```
Hello Alice
```

Note: Although concatenation appears to modify strings, strings in Python are immutable.

Strings Are Immutable

In Python, strings are immutable. This means once a string is created, its contents cannot be changed. Operations such as concatenation or slicing return new strings without altering the original string.

Example:

```
name = 'Alice'

name = name + ' Smith'

print(name)
```

This does not change the original string 'Alice' but creates a new string 'Alice Smith' and reassigns the name variable to reference it.

String Slicing

A slice is a span of characters taken from a string. To get a slice, use the format:

```
string[start:end]
```

This returns a substring from index start up to, but not including, end.

Example:

```
greeting = 'Hello World'
substring = greeting[0:5]
print(substring)
```

Output:

```
Hello
```

Testing Strings with in and not in

The in operator can be used to test whether one string is contained in another. The expression returns True if the substring is found, otherwise False.

Example:

```
message = 'Welcome to Python programming.'
print('Python' in message)      # True
```

```
print('Java' not in message)    # True
```

3.3.5 String Methods in Python

Recall that a method is a function that belongs to an object and performs some operation on that object. Strings in Python have numerous methods. In this section, we will discuss several string methods for performing the following types of operations:

- Testing the values of strings
- Performing various modifications
- Searching for substrings and replacing sequences of characters

The string methods shown below test a string for specific characteristics. For example, the `isdigit()` method returns `True` if the string contains only numeric digits. Otherwise, it returns `False`.

Table 16: Some String Testing Methods

Method	Description
<code>isalnum()</code>	Returns <code>True</code> if the string contains only alphabetic letters or digits and is at least one character in length. Returns <code>False</code> otherwise.
<code>isalpha()</code>	Returns <code>True</code> if the string contains only alphabetic letters and is at least one character in length. Returns <code>False</code> otherwise.
<code>isdigit()</code>	Returns <code>True</code> if the string contains only numeric digits and is at least one character in length. Returns <code>False</code> otherwise.
<code>islower()</code>	Returns <code>True</code> if all of the alphabetic letters in the string are lowercase, and the string contains at least one alphabetic letter. Returns <code>False</code> otherwise.
<code>isspace()</code>	Returns <code>True</code> if the string contains only whitespace characters and is at least one character in length. Returns <code>False</code> otherwise.
<code>isupper()</code>	Returns <code>True</code> if all of the alphabetic letters in the string are uppercase, and the string contains at least one alphabetic letter. Returns <code>False</code> otherwise.

Example Code:

```
s1 = "Python3"
print(s1.isalnum())    # True
s2 = "Hello"
print(s2.isalpha())    # True
s3 = "1234"
print(s3.isdigit())    # True
s4 = "hello"
print(s4.islower())    # True
s5 = "      "
```

```
print(s5.isspace()) # True
s6 = "WELCOME"
print(s6.isupper()) # True
```

Although strings are immutable, meaning they cannot be modified, they do have a number of methods that return modified versions of themselves.

Table 17: String Modification Methods

Method	Description
<code>lower()</code>	Returns a copy of the string with all alphabetic letters converted to lowercase.
<code>lstrip()</code>	Returns a copy of the string with all leading whitespace characters removed.
<code>lstrip(char)</code>	Removes specified leading characters.
<code>rstrip()</code>	Returns a copy of the string with all trailing whitespace characters removed.
<code>rstrip(char)</code>	Removes specified trailing characters.
<code>strip()</code>	Removes both leading and trailing whitespace characters.
<code>strip(char)</code>	Removes specified leading and trailing characters.
<code>upper()</code>	Returns a copy of the string with all alphabetic letters converted to uppercase.

Example Code:

```
text = " Python Programming "
print(text.lower()) # ' python programming '
print(text.lstrip()) # 'Python Programming '
print(text.rstrip()) # ' Python Programming'
print(text.strip()) # 'Python Programming'
print(text.upper()) # ' PYTHON PROGRAMMING '
text2 = "---Welcome---"
print(text2.strip('-')) # 'Welcome'
```

Programs commonly need to search for substrings, or strings that appear within other strings. For example, you might need to search for a word in a document. Python provides convenient methods for these operations.

Table 18: Search and Replace Methods

Method	Description
<code>endswith(sub)</code>	Returns True if the string ends with the specified substring.
<code>find(sub)</code>	Returns the lowest index of the specified substring. Returns -1 if not found.

<code>replace(old, new)</code>	Returns a copy of the string with all occurrences of old replaced by new.
<code>starts-with(sub)</code>	Returns True if the string starts with the specified substring.

These methods enhance the ability to inspect and manipulate text in a powerful yet readable manner, which is essential for tasks involving input validation, parsing, formatting, and data cleaning in many real-world applications.

Splitting a String

Python strings have a method called `split()` that splits the string into a list of words, using whitespace as the default delimiter.

Example:

```
sentence = 'Data Science with Python'
words = sentence.split()
print(words)
```

Output:

```
['Data', 'Science', 'with', 'Python']
```

You can also specify a custom delimiter:

```
data = 'apple,banana,orange'
fruits = data.split(',')
print(fruits)
```

Output:

```
['apple', 'banana', 'orange']
```

Example:

```
# Basic string operations
message = "Welcome to Python!"
print("First character:", message[0])
print("Last character:", message[-1])
print("Slice (0-7):", message[0:7])
```

```
print("Length:", len(message))
```

```
print("Lowercase:", message.lower())
```

```
print("Replace 'Python' with 'Programming':", message.replace("Python", "Program-  
ming"))
```

Real-world Use: Used for user input, messages, file names, report headers, and data parsing.

In-Text Questions (ITQs)

1. What is the difference between a Python list and a tuple?
2. How do you add an item to the end of a list?
3. How do you access individual characters in a string?
4. Explain the difference between mutable and immutable data types.
5. How can you convert a list into a tuple and vice versa?

In-Text Answers (ITAs)

1. Lists are mutable (changeable) while tuples are immutable (cannot be changed).
2. Use the `append ()` method to add an item to a list.
3. Access characters using indexing (`string[index]`) or iteration with a `for` loop.
4. Mutable types (like lists) can be modified after creation; immutable types (like tuples and strings) cannot.
5. Use `tuple(list variable)` to convert a list to a tuple, and `list(tuple_variable)` to convert a tuple to a list.

3.3.6 Conclusion

In this unit, you have learned about Python sequences, lists, tuples, numbers, and strings. You now understand how to store, access, and manipulate data efficiently

using these structures, which form the foundation for building organized and effective Python programs.

Summary

In this unit, you have learned that lists are mutable sequences that allow adding, removing, sorting, and reversing items, while tuples are immutable sequences, ideal for data that should not change. Python numbers include integers and floats, supporting a variety of arithmetic operations. Strings are immutable sequences that support indexing, slicing, concatenation, and methods for searching, modifying, and splitting. Additionally, converting between lists and tuples is straightforward using the `list()` and `tuple()` functions.

Self-Assessment Questions

1. Differentiate between a list and a tuple.
2. Write a Python program to reverse a list.
3. Explain the immutability of strings and tuples with examples.
4. List five string methods that modify or inspect strings.

Unit Activities

1. Read the content of Module 3 Unit 3 in the study guide and explore additional examples online (90 minutes).
2. Complete the self-introductory forum. Post your original contribution and reply to at least two classmates (20 minutes).
3. Practice list, tuple, number, and string operations using Python IDLE or an online editor (60 minutes).

Tutor-Marked Assignment

Complete the following tasks in Python and submit your code for grading:

- Create a list of your five favorite fruits. Add one fruit, remove another, and print the final list.

- Write a program that uses a tuple to store three coordinates and prints each coordinate.
- Convert a list of numbers [10, 20, 30] into a tuple and display it.
- Write a program to calculate the sum, difference, product, and quotient of two numbers.
- Manipulate the string "Python Programming": display the first character, last character, slice "Python", convert to uppercase, and replace "Programming" with "Course".

References/Further Readings

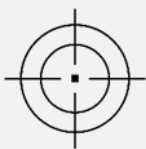
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

Unit 4 — Python Data Structures: Sets and Dictionaries

INTRODUCTION

In this unit, you will learn about Python data structures, specifically sets and dictionaries. You will understand how to create, manipulate, and apply these structures to efficiently store and manage unique data and key-value pairs in Python programs.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain the characteristics of Python sets and dictionaries.
- Create and manipulate sets and dictionaries.
- Use set operations such as union, intersection, difference, and symmetric difference.
- Add, update, and delete elements in a dictionary.
- Iterate through sets and dictionaries effectively.
- Apply appropriate methods to manage sets and dictionaries in Python programs.

3.4.1 Python Sets

A set contains a collection of unique values and works like a mathematical set.

A set is an object that stores a collection of data in the same way as mathematical sets. Here are some important things to know about sets:

- All the elements in a set must be unique. No two elements can have the same value.
- Sets are unordered, which means that the elements in a set are not stored in any particular order.
- The elements that are stored in a set can be of different data types.

Creating a Set

To create a set, you have to call the built-in `set()` function. This function returns a new set object.

```
# Create an empty set
```

```
myset = set()
```

Explanation:

In this code, the variable `myset` is assigned an empty set. This is done by calling the `set()` function with no arguments. The empty set contains no elements and is useful when you want to add elements later.

```
# Create a set from a list
```

```
myset = set(['a', 'b', 'c'])
```

Explanation:

Here, the `set()` function takes a list `['a', 'b', 'c']` as an argument. Each element in the list becomes an element in the new set `myset`. Sets only store unique elements, so all the letters 'a', 'b', and 'c' appear once. This shows how you can convert other iterable objects, like lists, into sets.

```
# Create a set from a string
```

```
myset = set('abc')
```

```
print(myset) # Output might be: {'a', 'b', 'c'}
```

Explanation:

When you pass a string 'abc' to the `set()` function, it treats the string as an iterable sequence of characters. Each character 'a', 'b', and 'c' becomes a separate element of the set. Note that the order of elements in a set is not guaranteed, so the output might show the letters in any order.

```
# Duplicate values are removed automatically
```

```
myset = set('aaabc')
```

```
print(myset) # Output: {'a', 'b', 'c'}
```

Explanation:

Sets cannot contain duplicate elements. In this example, the string 'aaabc' contains multiple 'a' character. When converted to a set, duplicates are removed automatically, leaving only unique characters 'a', 'b', and 'c'. This illustrates one of the key features of sets: enforcing uniqueness.

```
# Correct way to create a set of multi-character strings
```

```
myset = set(['one', 'two', 'three'])
```

```
print(myset)
```

Explanation:

To create a set where each element is a string of more than one character, you must pass a list containing those strings to the `set()` function. Passing multiple string arguments directly to `set()` will cause an error, and passing a single long string will create a set of individual characters instead. Here, `myset` will contain the strings 'one', 'two', and 'three' as separate elements.

Getting the Number of Elements in a Set

As with lists, tuples, and dictionaries, you can use the `len` function to get the number of elements in a set.

```
myset = set(['apple', 'banana', 'cherry'])  
print(len(myset)) # Output: 3
```

Adding and Removing Elements

Sets are mutable objects, so you can add items to them and remove items from them.

```
# Adding elements  
myset = set()
```

```
myset.add('apple')
print(myset) # Output: {'apple'}
# Removing elements
myset.remove('apple')
print(myset) # Output: set()
```

Using the for Loop to Iterate over a Set

You can use the for loop in the following general format to iterate over all the elements in a set:

```
fruits = set(['apple', 'banana', 'cherry'])
for fruit in fruits:
    print(fruit)
```

Using the in and not in Operators to Test for a Value in a Set

You can use the in operator to determine whether a value exists in a set.

```
myset = set(['apple', 'banana'])
print('apple' in myset) # Output: True
print('cherry' not in myset) # Output: True
```

Finding the Union of Sets

The union of two sets is a set that contains all the elements of both sets.

```
set1 = set([1, 2, 3])
set2 = set([3, 4, 5])
union_set = set1.union(set2)
print(union_set) # Output: {1, 2, 3, 4, 5}
```

Finding the Intersection of Sets

The intersection of two sets is a set that contains only the elements that are found in both sets.

```
set1 = set([1, 2, 3])
set2 = set([2, 3, 4])
intersection_set = set1.intersection(set2)
print(intersection_set) # Output: {2, 3}
```

Finding the Difference of Sets

The difference of set1 and set2 is the elements that appear in set1 but do not appear in set2.

```
set1 = set([1, 2, 3])
```

```
set2 = set([2, 3, 4])
diff_set = set1.difference(set2)
print(diff_set) # Output: {1}
```

Finding the Symmetric Difference of Sets

The symmetric difference of two sets is a set that contains the elements that are not shared by the sets.

```
set1 = set([1, 2, 3])
set2 = set([3, 4, 5])
sym_diff_set = set1.symmetric_difference(set2)
print(sym_diff_set) # Output: {1, 2, 4, 5}
```

Finding Subsets and Supersets

Suppose you have two sets, and one of those sets contains all of the elements of the other set.

Suppose you have two sets, and one of those sets contains all of the elements of the other set.

```
set1 = set([1, 2, 3, 4])
set2 = set([2, 3])

# Check if set2 is a subset of set1
print(set2.issubset(set1)) # Output: True

# Check if set1 is a superset of set2
print(set1.issuperset(set2)) # Output: True
```

3.4.2 Python Dictionaries

A dictionary is an object that stores a collection of data. Each element in a dictionary has two parts: a key and a value. You use a key to locate a specific value. In Python, dictionary elements are referred to as key-value pairs. This is similar to a real-world dictionary where you use a word (key) to look up its definition (value).

Dictionaries are useful when you want to associate a unique key with a specific value, such as storing names and phone numbers, employee IDs and employee names, etc.

Creating a Dictionary

A dictionary in Python is a collection of key-value pairs enclosed in curly braces {}. Each element in a dictionary consists of a key and a value, separated by a colon :. Multiple key-value pairs are separated by commas.

In the example below, we create a dictionary named `employee` where the keys are employee ID numbers (101, 102, 103), and the values are the corresponding employee names ('Alice', 'Bob', 'Charlie'):

```
employee = {  
    101: 'Alice',  
    102: 'Bob',  
    103: 'Charlie'  
}  
print(employee)
```

- The curly braces `{}` indicate that we are defining a dictionary.
- Each key is followed by a colon `:` and then its associated value.
- Keys must be unique and immutable types (like integers, strings, or tuples).
- Values can be of any data type and do not need to be unique.
- When the dictionary is printed, it displays the collection of key-value pairs.

Output:

```
{101: 'Alice', 102: 'Bob', 103: 'Charlie'}
```

This dictionary allows you to look up an employee's name by using their ID number as the key.

Retrieving a Value

In Python, dictionaries store data as unordered key-value pairs. Unlike lists or tuples, you cannot access items by position; instead, values are retrieved directly using their keys, with the syntax:

```
dictionary_name[key]
```

- `dictionary_name` is the variable that holds the dictionary.
- `key` is the specific key whose value you want to retrieve.

If the key exists in the dictionary, this expression returns the value associated with that key. For example, consider a dictionary named `employee` that maps employee ID numbers to employee names:

```
employee = {101: 'Alice', 102: 'Bob', 103: 'Charlie'}  
print(employee[102]) # Output: Bob
```

Here, `employee[102]` looks up the key 102 in the dictionary and returns the value 'Bob'.

However, if you try to retrieve a value using a key that does not exist in the dictionary, Python will raise a `KeyError` exception. For example:

```
print(employee[105]) # Raises KeyError because 105 is not a key in employee
```

This behavior helps catch errors when you attempt to access data that is not present in the dictionary.

Using in and not in

In Python, dictionary keys must be unique, and values are accessed via their keys. Accessing a non-existent key raises a `KeyError`, which can crash your program. To prevent this, check if the key exists using the `in` operator before retrieving its value. For example

```
employee = {
    101: "John",
    102: "Mary",
    103: "Alice"
}

# Safely checking if a key exists in the dictionary
if 101 in employee:
    print("Employee exists")
```

Explanation:

The code shows how to safely check for a key in a Python dictionary to avoid a `KeyError`. Using `in`, the program verifies whether a key exists before accessing it. For example, `if 101 in employee:` checks if key 101 is in the employee dictionary and prints "Employee exists" if found. This method prevents errors and makes the code more reliable.

Adding or Updating Elements in a Dictionary

Dictionaries in Python are mutable, meaning you can change their contents after they are created. This includes adding new key-value pairs or updating the values of existing keys.

You can add a new key-value pair or update an existing key's value by using an assignment statement in this general format:

```
dictionary_name[key] = value
```

- `dictionary_name` is the variable referencing the dictionary.
- `key` is the key for the item you want to add or update.
- `value` is the data associated with the key.

When you use this syntax:

- If the key does not already exist in the dictionary, a new key-value pair is created, and the key and value are added to the dictionary.
- If the key already exists, the old value associated with that key is replaced by the new value.

Important: Dictionaries cannot have duplicate keys. Every key must be unique. Assigning a new value to an existing key will overwrite the old value.

Example Code

```
employee = {  
    101: 'John',  
    102: 'Sara',  
    103: 'Mike'  
}  
  
# Adding a new key-value pair  
employee[104] = 'David' # Adds a new employee with ID 104  
  
# Updating an existing key's value  
employee[101] = 'Ann' # Changes the employee with ID 101  
from 'John' to 'Ann'  
  
print(employee)
```

Output:

```
{101: 'Ann', 102: 'Sara', 103: 'Mike', 104: 'David'}
```

- First, the key 104 did not exist in the dictionary, so the pair 104: 'David' was added.
- Then, the value for the existing key 101 was updated from 'John' to 'Ann'.

This feature makes dictionaries very flexible for storing and managing data where you may need to dynamically add or update information.

Deleting Elements from a Dictionary

In Python, dictionaries store data in key-value pairs, where each key uniquely identifies its associated value. Sometimes, you may need to remove a key-value pair from a dictionary — for example, if an employee leaves a company and you want to remove their record.

You can delete an existing key-value pair from a dictionary using the `del` statement. The general syntax is:

```
del dictionary_name[key]
```

- `dictionary_name` is the variable that references the dictionary.

- key is the specific key of the key-value pair you want to remove.

When this statement executes, the key and its associated value are permanently removed from the dictionary.

Important:

- If the key exists in the dictionary, the key-value pair will be deleted successfully.
- If the key does not exist, Python will raise a `KeyError` exception because you are trying to delete something that isn't there.

Example:

```
employee = {  
    101: 'Alice',  
    102: 'Bob',  
    103: 'Charlie'  
}  
  
del employee[102]  # Removes the key 102 and its value  
                  'Bob'  
  
print(employee)
```

Output:

```
{101: 'Alice', 103: 'Charlie'}
```

Explanation:

- The employee dictionary starts with three ID–name pairs.
- `del employee[102]` removes the entry with key 102 ('Bob').
- Printing `employee` after deletion shows that key 102 and 'Bob' are gone.
- Deleting a non-existent key, e.g., `del employee[999]`, raises a `KeyError`.

This method is useful for managing dictionary data by removing obsolete or unwanted entries safely.

Creating and Using an Empty Dictionary

Sometimes you may not know all the data to store initially, so you can create an empty dictionary using `{}` and add key-value pairs dynamically as the program runs.

```
student_scores = {}
```

At this point, `student_scores` is an empty dictionary. It does not contain any data.

You can then add key-value pairs to the dictionary by assigning a value to a new key using the syntax:

```
dictionary_name[key] = value
```

In the example below, we add a key 'John' with the value 85 to the `student_scores` dictionary:

```
student_scores['John'] = 85
```

This statement adds the key 'John' with the value 85 to the dictionary, updating it if 'John' already exists. Now, `student_scores` contains 'John': 85. This technique is useful for incrementally building a dictionary from files, user input, or calculations.

Iterating Through a Dictionary

A dictionary in Python stores data as key-value pairs. Sometimes, you need to go through all the items in a dictionary and perform actions on each key or value.

You can do this using a for loop that loops through the dictionary. When you write:

```
for var in dictionary:  
    # statements
```

- `var` is a variable name you choose, often called the loop variable.
- `dictionary` is the name of the dictionary you want to loop over.
- This loop runs once for each key in the dictionary.
- During each iteration, `var` is assigned the current key from the dictionary.

You can then use this key to access the value associated with it by using the syntax `dictionary[var]`.

Example Explanation

Suppose you have a dictionary called `employee`:

```
} employee = {  
    'ID': 101,  
    'Name': 'John Doe',  
    'Department': 'Finance',  
    'Salary': 50000  
}
```

You can loop through all the keys and print both the keys and their corresponding values like this:

```
for key in employee:  
    print(key, employee[key])
```

How this works:

- The for loop goes through each key in employee one by one.
- In the first iteration, key might be 'ID'. The code prints 'ID' and employee['ID'] which is 101.
- In the second iteration, key might be 'Name'. It prints 'Name' and employee['Name'] which is 'John Doe'.
- This continues for all keys in the dictionary.

The output will be:

```
ID 101
Name John Doe
Department Finance
Salary 50000
```

This way, you can easily access and use both the keys and their values inside the loop.

Dictionary Methods

Python dictionaries offer built-in methods to easily manipulate and access data, such as clearing the dictionary, retrieving values safely, iterating over items, and removing entries. Here are some of the most useful methods:

Table 19: Dictionary Methods in Python

Method	Description
Clear	Removes all key-value pairs, leaving an empty dictionary.
Get	Retrieves the value for a key, returning a default if the key is missing.
Items	Returns all key-value pairs as a sequence of tuples.
Keys	Returns all keys in the dictionary.
Pop	Removes a key and returns its value, or returns a default if key not found.
popitem	Removes and returns a random key-value pair as a tuple.
values	Returns all values in the dictionary.

clear()

The clear method removes all elements from the dictionary, effectively emptying it. This is useful when you want to reuse the dictionary but start fresh.

Example:

```
employee = {101: 'Alice', 102: 'Bob', 103: 'Carol'}
employee.clear()
```

```
print(employee)    # Output: {}
```

Explanation:

After calling `clear()`, the dictionary no longer contains any items.

`get()`

The `get` method lets you retrieve the value for a given key without causing an error if the key doesn't exist. Instead of raising a `KeyError`, you can provide a default value to return.

Example:

```
employee = {101: 'Alice', 102: 'Bob'}  
  
name = employee.get(105, 'Not Found')  
  
print(name) # Output: Not Found
```

Explanation:

If key 105 is not in the dictionary, `get()` returns 'Not Found' instead of raising an error.

`items()`

The `items` method returns a view object containing all key-value pairs as tuples. This is very useful for looping through a dictionary.

Example:

```
employee = {101: 'Alice', 102: 'Bob'}  
for key, value in employee.items():  
    print(key, value)
```

Output:

```
101 Alice  
102 Bob
```

Explanation:

Each iteration gives you a key and its associated value.

`keys()`

The `keys` method returns a view of all the keys in the dictionary.

Example:

```
employee = {101: 'Alice', 102: 'Bob'}  
print(employee.keys())  
# Output: dict_keys([101, 102])
```

Explanation:

You get a list-like object of all dictionary keys.

pop()

The pop method removes a key from the dictionary and returns its value. You can also specify a default return value if the key is not found, which prevents errors.

Example:

```
employee = {101: 'Alice', 102: 'Bob'}  
removed = employee.pop(101, 'Not Found')  
print(removed)           # Output: Alice  
print(employee)          # Output: {102: 'Bob'}
```

Explanation:

The key 101 and its value 'Alice' are removed and returned.

popitem()

The popitem method removes and returns an arbitrary key-value pair from the dictionary as a tuple. This can be useful when you want to process and remove items one by one.

Example:

```
employee = {101: 'Alice', 102: 'Bob'}  
k, v = employee.popitem()  
# Output: 102 Bob (or 101 Alice, order is arbitrary)  
print(k, v)  
# Remaining dictionary after removal  
print(employee)
```

Explanation:

One random key-value pair is removed and returned.

values()

The values method returns a view of all the values in the dictionary without their keys.

Example:

```
employee = {101: 'Alice', 102: 'Bob'}
# Output: dict_values(['Alice', 'Bob'])
print(employee.values())
```

Explanation:

You get a list-like object containing all values only.

Table 20: Summary Table Comparing Python Data Structures

Structure	Mutable	Ordered	Duplicate Values Allowed	Syntax Example
str	No	Yes	Yes	"hello"
list	Yes	Yes	Yes	[1, 2, 3]
tuple	No	Yes	Yes	(1, 2, 3)
set	Yes	No	No	{"a", "b"}
dict	Yes	No	Keys: No, Values: Yes	{"key": "value"}

In-Text Questions (ITQs)

1. What is the key characteristic that distinguishes a set from a list?
2. How do you safely retrieve a value from a dictionary without causing an error?
3. Explain the difference between the union and intersection of sets.

In-Text Answers (ITAs)

1. A set contains only unique elements and is unordered.
2. Use the get() method or check with the in operator before accessing the key.
3. Union combines all elements from two sets, while the intersection contains only the elements present in both sets.

3.4.3 Conclusion

In this unit, you have learned how to use Python sets and dictionaries to store and manipulate data efficiently, including performing operations like adding, removing, and iterating through elements.

Summary

In this unit, you have learned that sets enforce uniqueness and support mathematical operations, while dictionaries store key-value pairs for flexible and dynamic data management. Both data structures also provide built-in methods that simplify programming tasks.

Self-Assessment Questions

1. Explain how a set differs from a list in Python.
2. How would you remove a key-value pair from a dictionary?
3. What will be the output of `set([1,2,2,3])`?
4. Write a Python loop to iterate over a dictionary and print its keys and values.

Unit Activities

1. Create a set from a list of numbers and perform union and intersection with another set.
2. Build a dictionary for five students with their names as keys and scores as values, then update and delete an entry.
3. Test membership for a value in your set and a key in your dictionary)

Tutor-Marked Assignment

- Create a set named fruits containing at least 5 fruit names. Add 2 more fruits, remove 1, and print the final set.
- Create a dictionary inventory with at least 5 items and their quantities. Update one quantity, delete one item, and print all items with their quantities.
- Using two sets of numbers, compute and print the union, intersection, difference, and symmetric difference.

References/Further Readings

Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

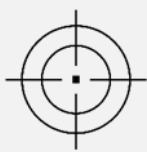
Module Four

Functions, Modules, and File Handling

Unit 1— Functions in Python

INTRODUCTION

In this unit, you will learn about functions in Python, their structure, types, and how to use them to build modular, reusable, and maintainable code. You will also explore built-in functions and modules like math and random to perform common tasks efficiently.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define and explain the purpose of functions in Python.
- Create void and value-returning functions.
- Pass arguments and use local and global variables.
- Utilise built-in functions and standard modules such as math and random.
- Apply modular programming to simplify program design and encourage code reuse.
- Demonstrate proper function naming, indentation, and scope rules.

4.1.1 What are Functions?

A function is a block of organised, reusable code that performs a specific task. Functions are essential in programming because they help in breaking down large programs into smaller, manageable sections. This makes programs more readable, easier to maintain, and promotes code reuse.

In Python, a function is defined using the keyword `def`, followed by a name, parentheses `()`, and a colon `:`. The body of the function, which contains its statements, is indented underneath.

General Format for a Function in Python:

```
def function_name(parameters):  
  
    """  
    Optional documentation string (docstring) describing  
    what the function does, its parameters, and return value.  
    """  
  
    # One or more statements (the function body)  
    statement_1  
    statement_2  
    ...  
    return expression    # (optional)
```

Functions serve as the building blocks of modular programming, making it easier to solve complex problems by combining small, well-defined parts.

Benefits of Modularising a Program with Functions

Modular programming is the practice of dividing a large program into smaller, self-contained units called functions. Each function is responsible for a specific part of the overall task. The benefits of this approach include:

1. Simpler Code

Breaking a program into smaller functions makes it easier to read, understand, and manage. Each function performs a clear, specific role, reducing complexity and promoting clarity.

2. Code Reuse

Functions encourage code reuse by allowing a programmer to write once and use multiple times. Instead of duplicating code, you can call the same function from different parts of a program or even across different programs.

3. Better Testing

When errors occur, it is easier to isolate the problem to a specific function rather than inspecting an entire program. Each function can be tested independently, ensuring accuracy before integration.

4. Faster Development

Reusable functions speed up development, especially in teams or large projects. Once written and tested, a function can be added to a library for future projects.

5. Easier Facilitation of Teamwork

Functions promote teamwork by allowing different developers to work on separate parts of a program simultaneously, increasing productivity and ensuring consistency.

Function Names

Before defining a function, it is important to give it a meaningful and descriptive name. The name should indicate the purpose or action of the function.

Rules for Naming Functions

- Function names cannot use Python keywords.
- They must begin with a letter or an underscore (_).
- They can include letters, digits, or underscores after the first character.
- Function names are case-sensitive.
- Function names should not contain spaces.

Naming Tip

Since functions perform actions, use verbs in their names. Examples include:

- `calculate_total()`
- `get_user_input()`
- `print_invoice()`

4.1.2 Types of Functions in Python

Functions in Python are broadly classified into two types:

1. **Void Functions:** Perform an action but do not return a value.
2. **Value-Returning Functions:** Perform an action and send a value back to the caller using the return statement.

Understanding these categories is essential to knowing when a function should simply perform an operation or compute and return a result.

Defining and Calling a Void Function

A void function executes a block of code but does not return any value. The general structure is:

```
def function_name():  
    statement  
    statement
```

Example

```
def greet():  
    print("Hello, welcome to the Python class!")  
  
greet()
```

Explanation:

- def begins the function definition.
- The function name greet describes its purpose.
- The statements inside the function are indented.
- The function is executed by calling greet().

Output:

```
Hello, welcome to the Python class!
```

Void functions are often used for displaying messages, updating records, or performing repetitive tasks without returning a result.

Function Calls and Execution Flow

When Python encounters a function call, it:

1. Pauses execution at that point.
2. Jumps to the function's definition and executes its code.
3. Returns control to where it was called once complete.

Example

```
def main():  
    print("Start of program.")  
    show_message()  
    print("End of program.")  
  
def show_message():  
    print("Hello from a function!")
```

```
main()
```

Output:

```
Start of program.  
Hello from a function!  
  
End of program.
```

Indentation in Python

Python uses indentation to define blocks of code. Every line inside a function must be indented equally, typically by four spaces.

Correct Example

```
def my_function():  
    print("Line one")  
    print("Line two")
```

Incorrect Example

```
def my_function():  
    print("Line one")  
    print("Line two")
```

Explanation:

The indentation levels are inconsistent; Python will raise an `IndentationError`.

Rule: Always use a consistent number of spaces (recommended: four) per indentation level

Local Variables

A local variable is a variable created inside a function. It exists only while the function is running and cannot be accessed outside that function.

Example

```
def display():  
    message = "Hello from the function!"  
    print(message)  
display()  
# print(message)  # This will cause an error
```

Explanation:

The variable `message` is local to the function `display()`. It cannot be accessed outside.

Local variables prevent naming conflicts and ensure data isolation between functions.

Variable Scope

Scope defines where a variable can be accessed in a program.

There are two main types of scope:

1. Local Scope – variables defined inside a function.
2. Global Scope – variables defined outside all functions.

Example – Local Scope

```
def show_message():
    text = "Inside function"
    print(text)

show_message()

# print(text) # Causes an error
```

Each function has its own scope, so variables inside one function are invisible to others.

Passing Arguments to Functions

Arguments allow data to be passed into a function. The function can then use this data during execution.

Example

```
def greet(name):
    print("Hello, ", name)

greet("Ada")
```

Here, "Ada" is the argument, and name is the parameter.

Passing Multiple Arguments

Functions can accept more than one parameter.

Example

```
def show_sum(num1, num2):
    total = num1 + num2
    print("The sum is:", total)

show_sum(10, 15)
```

Output:

The sum is: 25

Global Variables and Constants

A global variable is defined outside any function and can be accessed anywhere in the program.

Example – Global Variable

```
name = "Alice"
def greet():
    print("Hello", name)
greet()
```

Output

Hello Alice

Explanation

In this program, the variable name is a global variable because it is defined outside of any function.

This means:

- It exists throughout the entire program.
- Any function in the program can access it directly (read its value).

When the function greet() is called, it prints "Hello" followed by the value of the global variable name, which is "Alice". Since name is available globally, the function can use it without needing it passed as a parameter.

To modify a global variable inside a function, use the keyword global, for example:

```
counter = 0
def increment():
    global counter
    counter += 1
increment()
print(counter)
```

Explanation

Normally, variables created inside a function are local and cannot change global variables outside the function.

To modify a global variable from within a function, you must use the global keyword.

In this example,

```
global counter
```

tells Python that counter refers to the global variable declared outside the function.

So, when counter += 1 runs, it increases the global counter value (from 0 to 1).

Output: 1

Global Constants

Constants are global variables that should not change. They are written in uppercase letters. For example:

```
PI = 3.14159
def area_of_circle(r):
    return PI * r * r
```

In this example, PI is a constant used to calculate the area of a circle.

The function area_of_circle(r) uses PI and the radius r to return the computed area without altering the value of PI.

Value-Returning Functions

A value-returning function computes a result and returns it to the caller using the return statement.

Example

```
def square(num):
    return num * num

result = square(4)
print("Square:", result)
```

Output:

```
Square: 16
```

Explanation

A value-returning function performs a calculation and sends the result back using the return statement.

In this example, the function square(num) computes num * num and returns the value to the caller.

When square(4) is called, it returns 16, which is stored in result and printed as:

Square: 16

These functions are useful for performing calculations and returning the results to other parts of a program.

4.1.3 Using Built-in Functions

Certain types, functions, and variables are always available to the interpreter and can be used in any source module. Although you do not need to perform any extra imports to access these functions, they are contained in a module *builtins* in Python

Within other modules that you import, the variable `__builtins__` is also bound to this module.

The Python interpreter has a number of functions built into it that are always available, some of which have been discussed earlier. They are listed here in alphabetical order.

Table 21: List of Built-in Functions in Python

<code>abs()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>all()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>any()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>ascii()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bin()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>bool()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>import()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	
<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>	

Using the math and random Modules

Python provides several built-in modules that contain predefined functions for performing common tasks. Two of the most widely used are the *math* and *random* modules.

The math Module

The *math* module provides access to a variety of mathematical functions and constants that go beyond Python's basic arithmetic operations. It is especially useful for calculations involving square roots, powers, logarithms, trigonometric functions, and mathematical constants such as π (*pi*) and *e*.

To use the module, you must first import it using the statement:

```
import math
```

Example – Using math

```
import math

num = 25
root = math.sqrt(num)
print("Square root:", root)
```

Explanation:

- The import math statement loads the module into the program.
- math.sqrt(num) calls the square root function from the module to compute $\sqrt{25}$.
- The result, 5.0, is stored in root and printed.

Output:

```
Square root: 5.0
```

Table 22: Some Common Math Module Functions

Function	Description	Example	Output
math.sqrt(x)	Returns the square root of x	math.sqrt(9)	3.0
math.pow(x, y)	Returns x raised to the power y	math.pow(2, 3)	8.0
math.ceil(x)	Rounds x up to the nearest integer	math.ceil(4.3)	5
math.floor(x)	Rounds x down to the nearest integer	math.floor(4.7)	4
math.pi	Represents the constant π (3.14159...)	math.pi	3.141592653589793

The random Module

The random module is used to generate random numbers and make random selections. It is especially useful in simulations, games, data sampling, and testing.

To use it, import the module using:

```
import random
```

Example – Using random

```
import random
```

```
number = random.randint(1, 10)
print("Random number:", number)
```

Explanation:

- The **random.randint(1, 10)** function generates a random integer between **1** and **10** (inclusive).
- Each time the program runs, a different number within that range may be displayed.

Sample Output:

```
Random number: 7
```

Table 23: Other Common random Module Functions

Function	Description	Example	Possible Output
random.random()	Returns a random float between 0.0 and 1.0	random.random()	0.6384
random.uniform(a, b)	Returns a random float between <i>a</i> and <i>b</i>	random.uniform(5, 10)	7.89
random.randrange(a, b)	Returns a random integer between <i>a</i> and <i>b-1</i>	random.randrange(1, 10)	4
random.choice(sequence)	Returns a random element from a list or tuple	random.choice(['apple', 'banana', 'mango'])	'banana'

In-Text Questions (ITQs)

1. What is the primary purpose of functions in Python?
2. Differentiate between a void function and a value-returning function.
3. How does Python distinguish local and global variables?
4. Name two built-in modules in Python and their uses.

In-Text Answers (ITAs)

1. Functions are blocks of reusable code that perform specific tasks, helping to modularise and simplify programs.
2. A void function performs an action without returning a value, while a value-returning function computes a result and returns it to the caller.
3. Local variables are defined inside a function and accessible only there, while global variables are defined outside functions and accessible throughout the program.
4. The math module provides advanced mathematical functions; the random module generates random numbers and selections

4.1.4 Conclusion

In this unit, you have learned how to define, call, and use functions in Python, including managing arguments, variable scopes, and leveraging built-in and standard library modules for efficient programming.

Summary

In this unit, you have learned that functions enable modular and reusable code, and that proper naming, indentation, and scope management are essential for ensuring function reliability. Python provides both void and value-returning functions to serve different purposes, while built-in functions and modules, such as math and random, further extend the language's capabilities.

Self-Assessment Questions

1. Explain the difference between local and global variables.
2. Write a void function that prints a greeting message.
3. Create a value-returning function that calculates the cube of a number.
4. Demonstrate how to generate a random integer between 1 and 50.

Unit Activities

1. Write a function to add two numbers and display the result.
2. Create a global variable, modify it inside a function, and print it before and after.
3. Use the math module to calculate the area of a circle given its radius.
4. Generate a random floating-point number between 10 and 20 using the random module.

Tutor-Marked Assignment

- a. Write a void function `print_multiples(n)` that prints the first 5 multiples of a given number `n`.
- b. Create a value-returning function `factorial(num)` that computes and returns the factorial of a number.
- c. Write a program that:
 - Uses a global variable to count function calls.
 - Uses local variables inside at least two functions.
 - Uses the math module to calculate the area of a circle.
 - Uses the random module to generate a random integer between 1 and 50.
- d. Develop a small function library containing three reusable functions for addition, subtraction, and multiplication of two numbers, and demonstrate their use in a program.

References/Further Readings

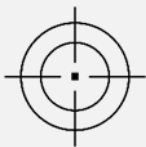
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

Unit 2— Modules and Packages

INTRODUCTION

In this unit, you will learn how to organise Python programs using modules and packages. You will understand how to create, import, and use modules to reuse code, manage namespaces, and improve program readability. You will also explore built-in modules like math and random and learn how packages help structure large programs into logical, hierarchical components.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain the purpose of modules and packages in Python.
- Create and import custom modules.
- Use built-in modules such as math and random for computations.
- Define and organise packages with submodules.
- Access functions, classes, and variables from modules and packages.

4.2.1 Overview

As Python programs grow larger, it becomes necessary to divide them into smaller, manageable parts.

Modules and packages provide an effective way to organise code into logical sections that can be easily reused and maintained.

A module is a single Python file containing code such as functions, variables, or classes, while a package is a collection of related modules organised within directories.

Python Modules

A module is a Python file (.py) that contains a collection of functions, classes, and variables designed to be reused in other programs. Modules help you logically organise your code, improve readability, and promote reusability.

Key Benefits of Using Modules

1. **Code Reuse:** Write code once and use it in multiple programs.
2. **Simplified Debugging:** Smaller code files are easier to test and debug.
3. **Namespace Management:** Each module creates its own namespace, preventing name conflicts.
4. **Better Organization:** Related functions can be grouped together logically.

Creating and Importing Modules

Any Python file can serve as a module. For example, a file named `display.py` containing functions can be imported into another program.

Importing a Module

To use a module in another program, you must import it using the `import` statement.

Example:

```
import display
```

This imports the entire module, allowing access to its contents using dot notation, e.g., `display.function_name()`.

You can also import specific functions or classes from a module:

```
from display import show_message
```

Or rename a module when importing:

```
import display as dsp
```

Accessing Module Attributes

When a module is imported, Python automatically creates a global variable `__name__` inside it, which stores the module's name as a string.

You can list all items (attributes) inside a module using the `dir()` function:

```
import math
print(dir(math))
```

This displays all available functions, constants, and variables in the `math` module.

Example: Creating and Using Custom Modules

circle_module.py

```
import math

def circle_area(radius):
    return math.pi * radius ** 2

def circle_circumference(radius):
    return 2 * math.pi * radius
```

rectangle_module.py

```
def rectangle_area(length, width):
    return length * width

def rectangle_perimeter(length, width):
    return 2 * (length + width)
```

main.py

```
import circle_module
import rectangle_module

r = 5
l = 10
w = 4

print("Circle Area:", circle_module.circle_area(r))
print("Circle Circumference:",
      circle_module.circle_circumference(r))
print("Rectangle Area:", rectangle_module.rectangle_area(l, w))
```

```
print("Rectangle Perimeter:", rectangle_module.rectangle_perimeter(l, w))
```

Explanation

- Each module (circle_module.py and rectangle_module.py) contains functions related to a specific shape.
- The main.py program imports both modules and calls their functions.
- This modular approach enhances organization, reusability, and clarity.

The math Module

Python's standard library includes many built-in modules. The math module provides mathematical functions and constants for performing complex calculations.

Table 24: Common Functions in the **math** Module

Function	Description
<code>math.sqrt(x)</code>	Returns the square root of x
<code>math.pow(x, y)</code>	Returns x raised to the power of y
<code>math.ceil(x)</code>	Rounds x up to the nearest integer
<code>math.floor(x)</code>	Rounds x down to the nearest integer
<code>math.sin(x)</code>	Sine of x radians
<code>math.cos(x)</code>	Cosine of x radians
<code>math.tan(x)</code>	Tangent of x radians
<code>math.radians(x)</code>	Converts degrees to radians
<code>math.degrees(x)</code>	Converts radians to degrees
<code>math.hypot(x, y)</code>	Returns the hypotenuse $\sqrt{x^2 + y^2}$
<code>math.pi</code>	Mathematical constant π
<code>math.e</code>	Mathematical constant e

Examples Using the math Module

Example 1: Square Root

```
import math
num = 16
result = math.sqrt(num)
print("Square root of", num, "is", result)
```

Output:

```
Square root of 16 is 4.0
```

Example 2: Ceiling and Floor

```
import math
num = 7.3
print("Ceiling:", math.ceil(num))
print("Floor:", math.floor(num))
```

Output:

```
Ceiling: 8
Floor: 7
```

Example 3: Trigonometric Calculation

```
import math
degrees = 90
radians = math.radians(degrees)
sin_value = math.sin(radians)

print(degrees, "degrees is", radians, "radians")
print("Sine value:", sin_value)
```

Output:

```
90 degrees is 1.5707963267948966 radians
Sine value: 1.0
```

Example 4: Hypotenuse Calculation

```
import math

side1 = 3
side2 = 4

hypotenuse = math.hypot(side1, side2)
print("Length of hypotenuse:", hypotenuse)
```

Output:

```
Length of hypotenuse: 5.0
```

The random Module

The random module provides functions to generate random numbers and make random selections. It is useful for simulations, games, and probabilistic computations.

Table 25: Common Functions in the random Module

Function	Description	Example
<code>random.randint(a, b)</code>	Returns a random integer between <i>a</i> and <i>b</i> (inclusive)	<code>random.randint(1, 10)</code>
<code>random.random()</code>	Returns a random float between 0.0 and 1.0	<code>random.random()</code>
<code>random.uniform(a, b)</code>	Returns a random float between <i>a</i> and <i>b</i>	<code>random.uniform(5, 10)</code>
<code>random.randrange(a, b)</code>	Returns a random integer between <i>a</i> and <i>b</i> - 1	<code>random.randrange(1, 10)</code>
<code>random.choice(seq)</code>	Returns a random element from a sequence	<code>random.choice(['apple', 'banana', 'mango'])</code>

Examples Using the random Module

Example 1: Generate a Random Integer

```
import random
number = random.randint(1, 10)
print("Random number:", number)
```

Explanation

- The function `random.randint(1, 10)` generates a random integer between 1 and 10 (inclusive).
- Each time the program runs, a different number within this range may appear.

Example 2: Generate a Random Float

```
import random
value = random.uniform(5, 10)
print("Random float between 5 and 10:", value)
```

Explanation

- The function `random.uniform(5, 10)` generates a random floating-point number between 5 and 10.
- The output is a decimal value, which changes each time the code runs.

4.2.2 Python Packages

A package is a directory that contains a collection of related Python modules and a special file named `__init__.py`. Packages help organize large programs into hierarchical structures.

Example of a Package Structure

```
admin/                                # Top-level package
  __init__.py
  Register/
    __init__.py
    staff.py
    students.py
  Finance/
    __init__.py
    salary.py
    tuition.py
  Results/
    __init__.py
    alpha.py
    omega.py
```

Each subfolder (like Register, Finance, and Results) is a subpackage, and each `.py` file inside them is a module.

Steps to Create a Package

1. **Create a directory** with your package name.
2. **Add an `__init__.py` file** inside the directory (even if it is empty).
3. **Place your modules** (e.g., `staff.py`, `students.py`) inside the directory.
4. **Import and use the modules** in your main program.

Example: Importing from a Package

```
from admin.Register import staff
staff.add()
```

You can also import specific functions:

```
from admin.Register.staff import add
```

Or import all names:

```
from admin.Register.staff import *
```

Note: Use `import *` carefully, as it can cause naming conflicts.

In-Text Questions (ITQs)

1. What is the difference between a module and a package?
2. How do you import a specific function from a module?

In-Text Answers (ITAs)

1. A module is a single Python file containing functions, classes, or variables, while a package is a directory containing related modules and an `__init__.py` file.
2. Use the syntax `from module_name import function_name`.

4.2.3 Conclusion

In this unit, you have learned how Python modules and packages help organize code, improve readability, and promote reuse. You can now create custom modules, use built-in modules like `math` and `random`, and structure large programs using packages.

Summary

In this unit, you have learnt that a module is a reusable Python file containing code, while a package is a directory of related modules. Importing modules allows access to functions, classes, and variables, improving code reuse and organization. Python's built-in modules, such as `math` and `random`, provide powerful functions for computations and random number generation. Packages help structure large programs hierarchically, with subpackages and modules that can be imported as needed.

Self-Assessment Questions

1. What is a Python module, and why is it useful?
2. How do you import a module and access its functions?
3. Explain the role of the `__init__.py` file in a package.
4. List three commonly used functions from the `math` module.

5. How can you generate a random integer between 1 and 100 using the random module?

Unit Activities

1. Create a module named calculator.py with functions for addition, subtraction, multiplication, and division.
2. Write a main program to import and use the calculator module.
3. Explore the math module and write a program to calculate the square root, power, and hypotenuse of given numbers.
4. Create a package shapes with modules circle.py and rectangle.py, each containing relevant functions, and demonstrate importing them in a main program.

Tutor-Marked Assignment

1. Create a module temperature.py with functions to convert Celsius to Fahrenheit and Fahrenheit to Celsius. Demonstrate its use in a main program.
2. Write a program that uses the math module to:
 - Calculate the area of a circle
 - Compute the sine and cosine of a given angle
3. Create a package school with subpackages Students and Staff. Each subpackage should contain a module with at least two functions. Write a main program to import and call these functions.
4. Use the random module to simulate rolling two dice 10 times and display the results.

References/Further Readings

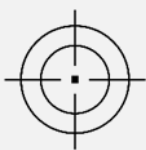
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

Unit 3— File Handling in Python

INTRODUCTION

In this unit, you will learn how to make Python programs store and retrieve data persistently using files. You will understand the types of files, file access methods, and how to open, read, write, append, and manage numeric and text data. You will also learn to handle file paths and process text files efficiently.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Explain the importance of file handling in Python.
- Distinguish between text and binary files.
- Open, read, write, and append data to files.
- Process numeric and string data in files.
- Work with file paths and directories for reading and writing data.
- Apply text file processing techniques such as filtering or transforming content.

4.3.1 Introduction to File Handling

When a Python program is running, all data is temporarily stored in Random Access Memory (RAM). RAM is volatile, meaning that once the program ends or the computer shuts down, any data stored in memory is lost.

To make data persistent—that is, to save it for later use—it must be stored on a non-volatile storage medium, such as a hard drive, flash drive, or CD. Data saved on such storage devices is kept in files.

A file is a named location on storage that holds information. By reading from and writing to files, Python programs can store and retrieve data between program runs, enabling persistent data management.

Importance of File Handling

File handling enables programs to store data permanently for future use. Unlike variables that lose their values when a program ends, files preserve data beyond execution. This is essential for applications that need to save and retrieve user information, settings, or processed results.

In practice, almost every software application depends on file handling:

- Word processors (e.g., Microsoft Word) store user-created text documents in files for later editing and printing.
- Image editing software (e.g., Adobe Photoshop) saves graphical modifications as image files for continued use.
- Spreadsheet applications (e.g., Microsoft Excel) maintain structured data and computational formulas within files.
- Games utilise files to record user progress, scores, and settings.
- Web browsers store cookies, cache data, and user preferences as files to improve performance and personalisation.

In Python, file handling provides the tools to create, read, write, update, and delete files. This ensures that data remains accessible across multiple program executions, supporting efficient data management and long-term storage of information.

Types of Files

Python primarily works with two main types of files: text files and binary files.

1. **Text Files:** Text files contain human-readable characters and whitespace such as letters, digits, and symbols. The data is stored as plain text, typically

using standard encodings like ASCII or UTF-8. These files can be easily opened and viewed with any text editor. For example, a file named data.txt might contain:

```
Alice
Bob
Carol
```

2. **Binary Files:** Binary files store data in a raw, machine-readable format composed of 0s and 1s. They are commonly used for storing non-textual data such as images, audio, videos, and executable programs. Unlike text files, binary files cannot be easily read or edited manually. Examples include image.jpg, music.mp3, and program.exe.

4.3.2 File Access Methods

There are two main methods for accessing file data:

- **Sequential Access:** Data is read from the beginning to the end of the file, in order. This method is simple and common in text file processing.
- **Direct (Random) Access:** The program can jump to any location in the file to read or write data. This is useful for large databases or structured binary files.

Opening and Closing Files

In Python, the built-in **open()** function is used to open a file and create a **file object** that connects the program to the file.

Syntax:

```
file_variable = open(filename, mode)
```

Table 26: Key Parameters Used When Opening a File in Python

Parameter	Description
Filename	Name or path of the file (e.g., 'data.txt')
Mode	Access mode ('r', 'w', or 'a')
file_variable	A variable referencing the file object

Table 27: Common File Modes

Mode	Description
'r'	Read mode (file must exist)

'w'	Write mode (creates a new file or overwrites existing content)
'a'	Append mode (adds new data to the end of the file)

Example – Reading a File

```
file = open('data.txt', 'r')
print(file.read())
file.close()
```

Explanation:

The program opens data.txt in **read mode** and prints its entire content. The **close()** method is then called to release system resources.

Writing Data to a File

To store data permanently, you can write to a file using the **write()** method.

Syntax:

```
file_variable.write(string)
```

Example – Writing to a File

```
file = open('students.txt', 'w')
file.write('John Doe\n')
file.write('Jane Smith\n')
file.close()
```

Explanation:

- 'w' mode creates or overwrites students.txt.
- Each call to write() adds text followed by a newline \n.
- Closing the file ensures all data is saved from memory to disk.

Reading Data from a File

Files can be read all at once or line by line.

Table 28: Methods for Reading Files

Method	Description
read()	Reads the entire file as a single string
readline()	Reads one line at a time
readlines()	Reads all lines and returns a list of strings

Example – Reading Line by Line

```
file = open('students.txt', 'r')
line = file.readline()
while line:
    print(line.strip())
    line = file.readline()
file.close()
```

Explanation:

- Each line is read until the file ends.
- The `strip()` method removes newline characters, ensuring clean output.

Appending Data to a File

When you want to add new data without erasing existing content, use append mode ('a').

Example – Appending to a File

```
file = open('log.txt', 'a')
file.write('New log entry.\n')
file.close()
```

Explanation:

This code opens `log.txt` in append mode and adds a new line of text. Existing data remains unchanged.

Writing and Reading Numeric Data

Python's `write()` method only accepts strings, so numeric values must be converted using `str()` before writing, and reconverted to numbers when reading.

Example – Writing Numeric Data

```
score = 95
file = open('scores.txt', 'w')
file.write(str(score) + '\n')
file.close()
```

Example – Reading Numeric Data

```
file = open('scores.txt', 'r')
line = file.readline()
score = int(line.strip())
print('Score:', score)
file.close()
```

Explanation:

- `str()` converts numbers to strings for writing.

- `int()` or `float()` converts strings back to numbers for processing.

File Paths and Directories

If a file is not in the same folder as the program, you must specify its path.

Example – Writing to a Specific Directory (Windows)

```
file = open(r'C:\Users\User\Documents\data.txt', 'w')
file.write('This is a test.')
file.close()
```

Note: The prefix `r` before the string makes it a **raw string**, preventing backslashes (`\`) from being interpreted as escape characters.

Text File Processing Example

The following example filters lines that do not begin with a hash symbol (`#`):

```
infile = open('oldfile.txt', 'r')
outfile = open('newfile.txt', 'w')

for line in infile:
    if line.startswith('#'):
        continue
    outfile.write(line)

infile.close()
outfile.close()
```

Explanation:

Lines beginning with `#` are skipped, and all others are copied to `newfile.txt`. The `continue` statement skips to the next iteration of the loop.

In-Text Questions (ITQs)

1. What is the difference between text and binary files?
2. How do you append data to an existing file?

In-Text Answers (ITAs)

1. Text files store human-readable characters, while binary files store raw, machine-readable data such as images or audio.
2. Open the file in append mode 'a' and use the write() method.

4.3.3 Conclusion

In this unit, you have learned how Python programs can store and retrieve data using files, ensuring persistence beyond program execution. You can now create, read, write, and append data to text files, handle numeric data, and manage file paths effectively.

Summary

In this unit, you have learnt that file handling allows Python programs to save and retrieve data on non-volatile storage, making information persistent. Python primarily uses text and binary files, which can be accessed sequentially or directly. You can open files in read, write, or append modes, process numeric and string data, handle file paths and directories, and implement text processing tasks such as filtering lines or skipping comments. Mastery of these techniques ensures efficient and reliable data management in your programs.

Self-Assessment Questions

1. Why is file handling important in programming?
2. Explain the difference between sequential and direct access.
3. What file modes are commonly used in Python, and what do they do?
4. How do you convert numeric data for writing to a text file?
5. How can you skip lines starting with # when processing a file?

Unit Activities

1. Create a text file students.txt and write the names of five students to it.
2. Write a program to read all lines from students.txt and display them.

3. Modify your program to append three more names to the file.
4. Write a program that reads numeric scores from a file, converts them to integers, and calculates the average.
5. Write a program that copies lines from one text file to another, skipping lines that start with #.

Tutor-Marked Assignment

1. Create a file inventory.txt and write at least five product names with their quantities.
2. Write a program to read the file and display each product with its quantity.
3. Append two more products to inventory.txt and display the updated list.
4. Write a program to calculate the total quantity of all products from the file.
5. Implement a program that reads a text file notes.txt and creates a new file filtered.txt containing only the lines that do not start with the # symbol.

References/Further Readings

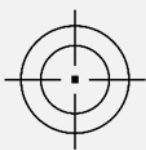
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

Unit 4— Exception Handling in Python

INTRODUCTION

In this unit, you will learn how Python handles errors that occur during program execution, known as exceptions. You will understand the types of built-in exceptions, how to catch and manage errors using try and except, how to raise exceptions deliberately, and how to make your programs more robust and user-friendly.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Define what exceptions are and explain their importance in Python programs.
- Identify common built-in and runtime exceptions.
- Handle exceptions gracefully using try and except blocks.
- Use exception handling within functions.
- Raise exceptions intentionally to enforce program rules.
- Apply exception handling to improve program reliability and user experience.

4.4.1 What is an Exception?

In programming, **exceptions** are errors that occur during program execution. When Python detects an error, it generates an exception object. If the exception is not handled properly, the program terminates abruptly and displays an error message (traceback). Exception handling allows the programmer to manage such errors gracefully, ensuring that the program continues executing or fails safely.

Built-in Exceptions

Python provides several **built-in exceptions** that help identify and respond to different types of errors. These are defined as *classes* and are automatically available when Python runs any program.

Exception Base Classes

1. **BaseException**
 - The root class for all exceptions in Python.
 - Every built-in exception is derived from this class.
2. **Exception**
 - The base class for most program-related exceptions.
 - All user-defined exceptions should inherit from this class (except `SystemExit`, `KeyboardInterrupt`, and `GeneratorExit`).
3. **ArithmeticError**
 - The base class for arithmetic-related exceptions such as:
 - `ZeroDivisionError`
 - `OverflowError`
 - `FloatingPointError`
4. **LookupError**
 - The base class for indexing and key errors such as:
 - `IndexError` (invalid index access)
 - `KeyError` (missing dictionary key)
5. **EnvironmentError**
 - The base class for errors that occur due to external factors, such as:
 - `IOError` (file input/output issues)
 - `OSError` (operating system-related issues)

These classes can be used to catch groups of related exceptions and handle them effectively.

Example – Handling Arithmetic Errors

```
try:
    result = 10 / 0
except ArithmeticError as e:
    print("Arithmetic error occurred:", e)
```

Output:

```
Arithmetic error occurred: division by zero
```

Explanation:

In this example, Python attempts to divide 10 by 0, which is mathematically undefined. This operation raises a `ZeroDivisionError`, a type of `ArithmeticError`. The `except` block catches the error and prints a user-friendly message instead of stopping the program abruptly. Hence, the output shows: Arithmetic error occurred: division by zero.

Common Runtime Exceptions

Whenever a runtime error occurs, Python halts execution and prints the traceback. Examples include:

Example 1 – Division by Zero

```
print(10 / 0)
```

Output:

```
ZeroDivisionError: division by zero
```

Example 2 – Accessing Non-existent List Item

```
numbers = [1, 2, 3]
print(numbers[5])
```

Output:

```
IndexError: list index out of range
```

Example 3 – Modifying an Immutable Object

```
data = (10, 20, 30)
data[0] = 5
```

Output:

```
TypeError: 'tuple' object does not support item
assignment
```

In all cases, the last line of the error message shows the **exception type** and details of the error.

Handling Exceptions with `try` and `except`

To prevent a program from terminating unexpectedly, we can use the **try-except** statement to handle exceptions.

Syntax:

```
try:
    # Code that might cause an exception
except ExceptionType:
    # Code to handle the exception
```

Example – Handling File Errors

```
try:
    file = open("data.txt", "r")
    content = file.read()
    file.close()
except FileNotFoundError:
    print("File not found. Please check the file name.")
```

In this example, if the file does not exist, the program will not crash but will instead display a friendly message.

Using Exception Handling in Functions

You can also handle exceptions inside functions to make your program more robust.

Example – Checking if a File Exists

```
def file_exists(filename):
    try:
        open(filename)
        return True
    except FileNotFoundError:
        return False

print(file_exists("info.txt"))
```

If the file exists, it returns True; otherwise, it returns False.

Raising Exceptions

Sometimes, a program needs to raise an exception deliberately when an invalid condition is detected. The **raise** statement is used for this purpose.

Example – Raising an Exception

```
def get_age():  
    age = int(input("Enter your age: "))  
    if age < 0:  
        raise ValueError("Age cannot be negative.")  
    return age  
  
try:  
    print("Your age is:", get_age())  
except ValueError as e:  
    print("Error:", e)
```

Explanation:

If the user enters a negative age, a ValueError is raised with a custom error message.

In-Text Questions (ITQs)

1. What is an exception in Python?
2. How do you prevent a program from crashing when an error occurs?
3. How can you deliberately raise an error in Python?

In-Text Answers (ITAs)

1. An exception is an error that occurs during program execution, which can terminate the program if not handled.
2. Use a try-except block to catch and handle the exception.
3. Use the raise statement with an appropriate exception type.

4.4.2 Conclusion

In this unit, you have learned how exceptions represent runtime errors and how to handle them effectively using Python's built-in mechanisms. You can now manage errors gracefully, prevent program crashes, raise exceptions deliberately, and write more robust and user-friendly programs.

Summary

In this unit, you have learnt that exceptions are runtime errors, such as division by zero or invalid indexing. Python's built-in exception classes can be handled using try-except blocks to maintain program flow. Functions can include exception handling for reliability, and the raise statement allows custom errors. Mastering exceptions ensures programs fail safely and behave predictably.

Self-Assessment Questions

1. Define an exception and explain its significance in programming.
2. List at least three common runtime exceptions in Python.
3. How does the try-except block work?
4. How can exceptions be handled within a function?
5. Demonstrate how to raise a ValueError when invalid input is detected.

Unit Activities

1. Write a program that divides two numbers and handles division by zero.
2. Create a list and try accessing an invalid index, handling the resulting exception.
3. Write a function that opens a file and returns its content, handling the case when the file does not exist.

Tutor-Marked Assignment

1. Write a program that takes two numbers as input and divides them. Use exception handling to manage division by zero and invalid input types.
2. Create a function `read_file(filename)` that attempts to open and read a file. Return the content if the file exists; otherwise, catch the exception and display a friendly message.
3. Write a function `calculate_age()` that prompts the user for their age. Raise a ValueError if a negative age is entered, and handle the exception in the main program.

4. Write a program that iterates over a list of integers and prints the reciprocal of each number. Handle any division by zero errors.

References/Further Readings

Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

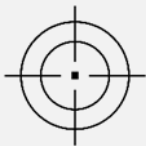
MODULE 5

Python Database and Gui Development

Unit 1— Python and Databases

INTRODUCTION

In this unit, you will learn how Python interacts with databases to store, retrieve, and manage structured data efficiently. You will explore database concepts, types, management systems, SQL commands, and practical integration of SQLite with Python for CRUD operations.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand databases, DBMS, and common data models.
- Describe relational and NoSQL databases.
- Use SQL to create, read, update, and delete data.
- Integrate SQLite with Python using the sqlite3 module.
- Perform basic CRUD operations programmatically.

5.1.1 Introduction to Databases

A database is a structured way to store, organise, and manage data so it can be easily accessed, updated, and maintained. Instead of keeping information in multiple files, databases centralise and structure data for efficiency and accuracy.

A database consists of related data organised in tables made up of rows and columns. Each row represents a record, and each column represents a field or attribute.

Databases make it easy to store and retrieve information systematically. For example,

id	name	age
1	John	20
2	Fatima	22
3	Peter	19
4	Mary	25

Types of Databases

There are different kinds of databases depending on how they store, organise, and manage data. Each type is suited for specific applications or data models.

1. Relational Databases

These databases store data in tables made up of rows (records or tuples) and columns (Field / Attribute). Relationships between tables are established through keys, and data is managed using Structured Query Language (SQL).

Examples: MySQL, SQLite, PostgreSQL.

2. NoSQL Databases

NoSQL databases are designed to handle unstructured or semi-structured data. They are flexible and scalable, making them ideal for large volumes of data such as that used in big data and web applications. Examples: MongoDB, Cassandra, CouchDB.

3. Hierarchical Databases

Data is organised in a tree-like structure, where each record has a single parent and possibly multiple children. This model is fast for data retrieval when relationships follow a strict hierarchy.

Example: IBM IMS.

4. Network Databases

This model allows many-to-many relationships between records, forming a more complex network structure. It is useful when data items have multiple interconnections.

Example: CODASYL DBTG.

5. Object-Oriented Databases

These databases store data as objects, similar to how data is represented in object-oriented programming. Each object contains both data (attributes) and methods (functions) that operate on the data.

Example: db4o, ObjectDB.

Table 29: Common Types of Databases and Their Characteristics

Type	Description	Examples
Relational	Uses tables with rows and columns	MySQL, SQLite
NoSQL	Stores unstructured data	MongoDB
Hierarchical	Tree-like structure	IBM IMS
Network	Allows many-to-many links	CODASYL
Object-Oriented	Stores objects with data and behavior	db4o

5.1.2 Database Management Systems (DBMS) and Data Models

Database Management Systems (DBMS)

A Database Management System (DBMS) is a specialised software designed to store, organise, and manage data efficiently. It provides users and applications with a systematic way to create, retrieve, update, and manage data within a database. A DBMS acts as an interface between the user and the physical database, ensuring that data remains consistent, secure, and accessible.

A key advantage of a DBMS is that it enables multiple users to access and manipulate data concurrently without conflicts or data loss. It also provides essential features such as data integrity, security control, backup and recovery, and query optimization.

Examples of popular DBMSs include:

- MySQL
- PostgreSQL
- SQLite

- Oracle Database
- Microsoft SQL Server

DBMSs are used across diverse applications such as banking systems, e-commerce platforms, educational databases, and healthcare systems, where structured data management and accuracy are essential.

Data Models

A data model defines the logical structure, organization, and relationships of data within a database. It serves as a blueprint that guides how data is stored, retrieved, and related to other data items. Data models help developers and database administrators visualize and design databases that reflect real-world entities and their interactions.

Different data models exist, each providing a unique way to represent and organize data. The relational model is the most widely used, representing data in tables (relations) that can be connected through keys.

Table 30: Common Database Models and Their Characteristics

Model	Description
Hierarchical Model	Organizes data in a tree-like structure , where each child record has one parent. It is efficient for applications with well-defined hierarchies, such as organizational charts or file systems.
Network Model	Allows many-to-many relationships between records, forming a more flexible network of connections. It is suitable for complex relationships such as supply chains or social networks.
Relational Model	Represents data in tables (rows and columns) connected by keys. This model uses Structured Query Language (SQL) for managing and querying data efficiently.
Object-Oriented Model	Stores data as objects —combining both data and the methods that operate on it. This model is ideal for applications that integrate closely with object-oriented programming.

In modern systems, hybrid approaches are also used—for example, object-relational databases that combine relational structure with object-oriented features, and NoSQL databases for handling unstructured or big data.

5.1.3 Introduction to Structured Query Language

Structured Query Language (SQL) is a standard language used for managing and manipulating data stored in relational databases. It enables users to create, modify,

query, and control access to data efficiently. SQL serves as the bridge between users (or applications) and the underlying database system.

SQL operates through a set of well-defined commands, which are generally divided into categories such as:

- Data Definition Language (DDL): Used to define and modify the structure of database objects (e.g., CREATE, ALTER, DROP).
- Data Manipulation Language (DML): Used to manage data within tables (e.g., INSERT, UPDATE, DELETE, SELECT).
- Data Control Language (DCL): Used to control user permissions and access to data (e.g., GRANT, REVOKE).
- Transaction Control Language (TCL): Used to manage changes made during transactions (e.g., COMMIT, ROLLBACK).

SQL is widely supported across relational database systems such as MySQL, PostgreSQL, Oracle, and SQLite.

Importance of SQL in Application Development

SQL is a fundamental skill for anyone working with data-driven systems. It allows developers and analysts to:

- Retrieve meaningful insights from large datasets.
- Maintain consistency and accuracy in data storage.
- Automate database interactions through programming languages like Python.
- Build scalable applications that rely on structured data management.

In modern business environments, SQL proficiency is essential for fields such as data analytics, software development, business intelligence, and web application.

SQLite and Python Integration

SQLite is a lightweight, self-contained database engine that stores the entire database in a single file on disk. Unlike other database systems such as MySQL or PostgreSQL, it does not require a separate server process to run. This makes it ideal for learning, testing, and small- to medium-scale applications.

SQLite supports most SQL commands, including CREATE, INSERT, UPDATE, DELETE, and SELECT, making it a fully functional relational database management system.

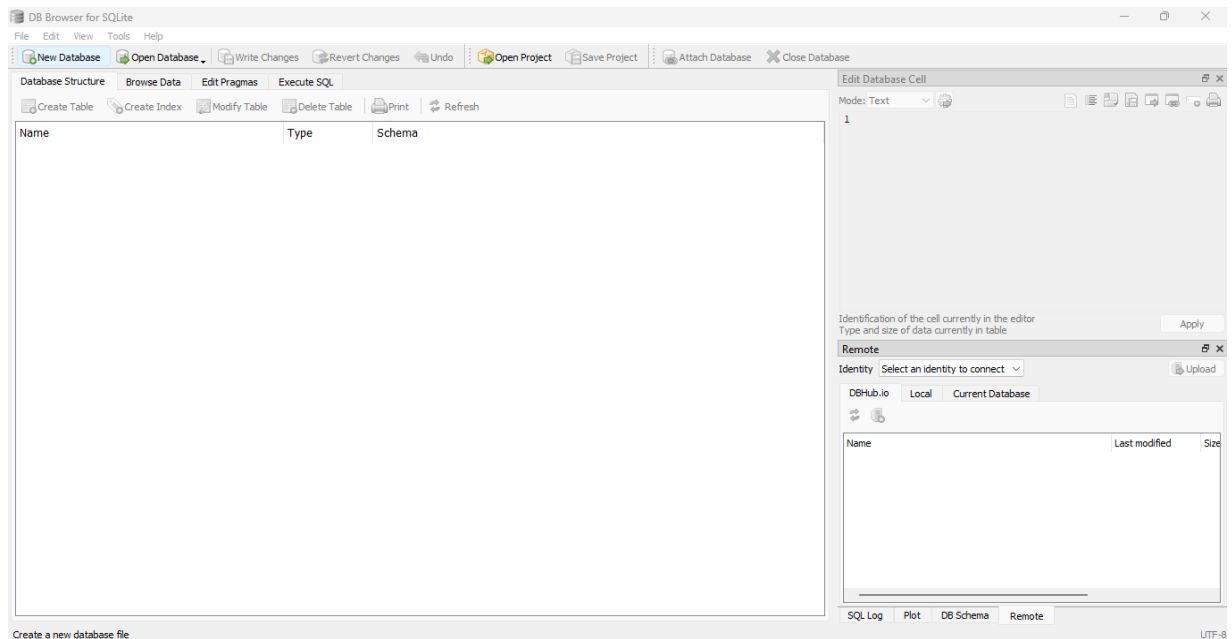


Figure 22: SQLite 3 Interface

The sqlite3 Module in Python

Python comes with a built-in module called `sqlite3` that provides an easy interface for interacting with SQLite databases. Using this module, you can:

- Create new databases or connect to existing ones.
- Execute SQL commands directly from Python programs.
- Store and retrieve data using SQL queries.

Because it is built into Python, there is no need for additional installation — `sqlite3` is available by default in Python 3 and later.

Installing SQLite (Optional)

Although Python's `sqlite3` module works without any setup, you may still want to install the SQLite command-line tool to manage and explore databases manually.

For Windows:

- Visit the official SQLite download page: <https://www.sqlite.org/download.html>.
- Download the *Precompiled Binaries for Windows*.
- Extract the files to a folder (e.g., `C:\sqlite`).
- Add that folder to your system PATH so that you can open SQLite from the command prompt.

For macOS and Linux:

SQLite is usually preinstalled. You can verify this by typing the following command in the terminal:

```
sqlite3 --version
```

If it is not installed, you can install it using:

- **macOS:** brew install sqlite
- **Linux (Ubuntu/Debian):** sudo apt install sqlite3

Opening SQLite for Use:

You can work with SQLite in two main ways:

1. Using the Command-Line Interface (CLI)

After installation, open the command prompt or terminal and type:

```
sqlite3 mydatabase.db
```

This command opens (or creates) a database file named *mydatabase.db*. You can then run SQL commands directly, such as:

```
CREATE TABLE students (id INTEGER PRIMARY KEY,  
name TEXT, age INTEGER);
```

2. Using Python's sqlite3 Module

You can also create and interact with a database directly within Python. Example:

```
import sqlite3  
  
# Connect to (or create) a database  
connection = sqlite3.connect("students.db")  
  
# Create a cursor to execute SQL commands  
cursor = connection.cursor()  
  
# Create a table  
cursor.execute("""  
CREATE TABLE IF NOT EXISTS students (  
    id INTEGER PRIMARY KEY,  
    name TEXT,  
    age INTEGER  
)  
""")  
  
# Commit changes and close connection  
connection.commit()
```

```
connection.close()

print("Database and table created successfully!")
```

This code creates a new SQLite database file named **students.db** in your working directory.

All subsequent operations — inserting, updating, deleting, or retrieving data — can be performed through this connection.

SQLite Data Types

SQLite uses a simplified type system known as type affinity, which allows flexible storage of data. The main SQL data types supported in SQLite3 are:

- **INTEGER** – Used for whole numbers. Automatically increments if set as a primary key.
- **REAL** – Stores decimal (floating-point) numbers.
- **TEXT** – Used for strings or character data.
- **BLOB** – Holds binary data, such as images or files, exactly as input.
- **NULL** – Represents missing or undefined values.

SQLite automatically converts data types when needed, making it more flexible than strictly typed database systems.

5.1.4 Working with SQLite Databases in Python

Working with SQLite databases in Python involves establishing a connection, creating tables, inserting, updating, retrieving, and managing data through SQL commands executed within Python programs. Each of these operations forms the foundation of database interaction and management, as discussed in the following subsections.

Connecting to a Database in Python

Before performing any database operations, the first step is to establish a connection to the database file. This is done using the `sqlite3.connect()` method.

If the specified database file does not exist, SQLite automatically creates one in the current working directory.

```
import sqlite3
# Connect to database (creates file if not found)
conn = sqlite3.connect('school.db')
cursor = conn.cursor()
print("Connected to database successfully.")
```

Explanation:

- `sqlite3.connect('school.db')` creates or opens the database file named *school.db*.
- `cursor = conn.cursor()` creates a cursor object that allows SQL commands to be executed.

Output:

```
Connected to database successfully.
```

Creating a Database Table

A **table** is the primary structure where data is stored in a relational database. The **CREATE TABLE** statement defines the columns (fields) and their data types.

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    age INTEGER,
    grade TEXT
)
""")
conn.commit()
print("Table 'students' created successfully.")
```

Resulting Table Structure:

Id	name	age	grade
----	------	-----	-------

Explanation:

This command creates a table named *students* with four columns — *id*, *name*, *age*, and *grade*. The *id* column automatically generates unique numbers for each record.

Database Keys

In relational databases, keys are special fields used to uniquely identify, relate, and ensure the integrity of records within and across tables. They prevent duplication, establish relationships between tables, and enforce data consistency.

1. Primary Key

A Primary Key uniquely identifies each record in a table. Every table should have one, and its value must be unique and not NULL.

In SQLite, you can define a primary key using the PRIMARY KEY constraint.

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    age INTEGER,
    grade TEXT
)
""")
```

Explanation:

- The column id serves as the primary key.
- The AUTOINCREMENT keyword automatically generates a unique ID for every new record.

Resulting Table Example:

id	name	age	Grade
1	Alice	21	A
2	Bob	20	B

2. Foreign Key

A Foreign Key creates a link between two tables. It identifies a column in one table that references a Primary Key in another, maintaining referential integrity.

Example – Linking students and courses tables:

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS enrollments (
    enroll_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY(student_id) REFERENCES students(id),
    FOREIGN KEY(course_id) REFERENCES courses(course_id)
)
""")
```

```
""")
```

Explanation:

- `student_id` in *enrollments* refers to the `id` in *students*.
- `course_id` in *enrollments* refers to the `course_id` in *courses*. This ensures that only valid students and courses can be linked.

Database keys are crucial for maintaining **data integrity** and **consistency**. They prevent duplicate entries, ensure valid relationships, and make searching and referencing data faster and more reliable.

Inserting Data

After creating a table, you can add data to it using the **INSERT INTO** statement. In Python, parameterized queries (using `?` placeholders) help prevent SQL injection and safely handle user input.

a) Inserting a Single Record

You can insert one record at a time using the `execute()` method.

```
import sqlite3

# Connect to the database
conn = sqlite3.connect("school.db")
cursor = conn.cursor()

# Insert a single record into the table
cursor.execute("INSERT INTO students (name, age, grade) VALUES (?, ?, ?)",
               ("Alice", 21, "A"))

# Save changes
conn.commit()
print("Single record inserted successfully.")

# Close connection
conn.close()
```

b) Inserting Multiple Records

To add several records at once, use the **executemany()** method.

```
import sqlite3

conn = sqlite3.connect("school.db")
```

```

cursor = conn.cursor()

# List of multiple records to insert
records = [
    ("Bob", 22, "B"),
    ("Charlie", 20, "A"),
    ("Diana", 23, "C")
]

# Insert all records in one command
cursor.executemany("INSERT INTO students (name, age,
grade) VALUES (?, ?, ?)", records)

conn.commit()
print("Multiple records inserted successfully.")
conn.close()

```

Table After Insertions:

Id	Name	Age	Grade
1	Alice	21	A
2	Bob	22	B
3	Charlie	20	A
4	Diana	23	C

Note:

The Id column is usually defined as an INTEGER PRIMARY KEY AUTOINCREMENT in SQLite.

This means SQLite automatically assigns a unique number (1, 2, 3, ...) to each new record inserted, so you don't need to include it in your INSERT statements.

Working with SQLite Databases in Python

This section covers how Python uses the sqlite3 module to create, manage, and query databases through basic SQL operations (CRUD — Create, Read, Update, Delete).

Reading Data (SELECT and WHERE)

The SELECT statement retrieves records from a table.

The optional WHERE clause filters results based on specific conditions.

Example 1 – Read All Records

- This retrieves all rows from the *students* table.

```

cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():

```

```
print(row)
```

Output:

```
(1, 'Alice', 21, 'A')
(2, 'Bob', 22, 'B')
(3, 'Charlie', 20, 'A')
(4, 'Diana', 23, 'C')
```

Example 2 – Filter with WHERE

- The **WHERE** clause is used to retrieve only records that meet a certain condition.

```
cursor.execute("SELECT name, age FROM students
WHERE age > 20")
for row in cursor.fetchall():
    print(row)
```

Output:

```
('Alice', 21)
('Bob', 22)
('Diana', 23)
```

Query Operators in Python SQL

Operators such as AND, OR, and ORDER BY provide flexibility in querying data.

```
cursor.execute("SELECT * FROM students WHERE age
> 20 AND grade = 'A'")
print(cursor.fetchall())
```

```
cursor.execute("SELECT * FROM students WHERE
grade = 'A' OR grade = 'B'")
print(cursor.fetchall())
```

```
cursor.execute("SELECT * FROM students ORDER BY
age DESC")
print(cursor.fetchall())
```

Sample Output:

```
[(1, 'Alice', 21, 'A')]
[(1, 'Alice', 21, 'A'), (2, 'Bob', 22, 'B'), (3,
'Charlie', 20, 'A')]
[(4, 'Diana', 23, 'C'), (2, 'Bob', 22, 'B'), (1,
'Alice', 21, 'A'), (3, 'Charlie', 20, 'A')]
```

Explanation:

- AND ensures both conditions are true.
- OR returns records that meet either condition.
- ORDER BY sorts results (here, by descending age).

Combining Result Sets (UNION)

The UNION operator combines results from two or more SELECT statements. It automatically removes duplicate records.

```
cursor.execute("""
SELECT name FROM students WHERE grade = 'A'
UNION
SELECT name FROM students WHERE grade = 'B'
""")
print(cursor.fetchall())
```

Resulting Output:

```
[('Alice',), ('Bob',), ('Charlie',)]
```

Grouping Data (GROUP BY and Aggregates)

The GROUP BY statement groups rows that share the same column value.

Aggregate functions like COUNT(), AVG(), and SUM() summarize grouped data.

```
cursor.execute("SELECT grade, COUNT(*) FROM stu-
dents GROUP BY grade")
for group in cursor.fetchall():
    print(group)
```

Output Table:

Grade	COUNT(*)
A	2
B	1
C	1

Updating Records

Existing records can be modified using the UPDATE statement. It is important to include a WHERE clause to specify which records to update.

```
cursor.execute("UPDATE students SET grade = ?
WHERE name = ?", ("B+", "Alice"))
conn.commit()
print("Record updated successfully.")
```

Updated Table:

Id	Name	Age	Grade
1	Alice	21	B+
2	Bob	22	B
3	Charlie	20	A
4	Diana	23	C

Deleting Records

The **DELETE FROM** statement removes records from a table. To avoid deleting all data, always use a **WHERE** clause.

```
cursor.execute("DELETE FROM students WHERE name = ?",
              ("Diana",))
conn.commit()
print("Record deleted successfully.")
```

Table After Deletion:

Id	Name	Age	Grade
1	Alice	21	B+
2	Bob	22	B
3	Charlie	20	A

Closing the Database Connection

After completing database operations, always close the connection. This ensures data integrity and releases system resources.

```
conn.close()
print("Database connection closed.")
```

Complete Python Program Example

Below is a complete program demonstrating all CRUD operations — Create, Read, Update, and Delete.

```
import sqlite3

# Step 1: Connect

conn = sqlite3.connect('school.db')
cursor = conn.cursor()
```

Step 2: Create table

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    age INTEGER,
    grade TEXT
)
""")
```

Step 3: Insert data

```
students = [
    ("Alice", 21, "A"),
    ("Bob", 22, "B"),
    ("Charlie", 20, "A"),
    ("Diana", 23, "C")
]
cursor.executemany("INSERT INTO students (name,
age, grade) VALUES (?, ?, ?)", students)
conn.commit()
```

Step 4: Query examples

```
cursor.execute("SELECT * FROM students WHERE age
> 20 ORDER BY name ASC")
print("Students older than 20:", cursor.fetch-
all())

cursor.execute("SELECT grade, COUNT(*) FROM stu-
dents GROUP BY grade")
print("Grade summary:", cursor.fetchall())
```

Step 5: Update and delete

```
cursor.execute("UPDATE students SET grade = 'B+'
WHERE name = 'Alice'")
cursor.execute("DELETE FROM students WHERE name =
'Diana'")
conn.commit()
```

Step 6: Display final data

```
cursor.execute("SELECT * FROM students")
print("Final Records:", cursor.fetchall())
```

Step 7: Close

```
conn.close()
```

Expected Output:

```
Students older than 20: [(1, 'Alice', 21, 'A'),  
 (2, 'Bob', 22, 'B'), (4, 'Diana', 23, 'C')]  
Grade summary: [('A', 2), ('B', 1), ('C', 1)]  
Final Records: [(1, 'Alice', 21, 'B+'), (2, 'Bob',  
 22, 'B'), (3, 'Charlie', 20, 'A')]
```

Explanation: The program demonstrates all CRUD operations such as creating a table, inserting, querying, updating, and deleting records. Each fetchall() returns the results as a list of tuples, making it easy to process or display database rows in Python.

In summary, SQLite integration with Python provides a lightweight and efficient way to manage structured data within applications. Using the sqlite3 module, developers can create tables, insert, query, update, and delete records without an external database server

In-Text Questions (ITQs)

1. What is the difference between a relational and a NoSQL database?
2. Name two advantages of using a DBMS.
3. What Python module allows interaction with SQLite databases?

In-Text Answers (ITAs)

1. Relational databases use tables with rows and columns and SQL, while NoSQL databases store unstructured data flexibly.
2. DBMS enables multi-user access, data integrity, security, and backup/recovery.
3. sqlite3.

5.1.5 Conclusion

In this unit, you have learned how databases provide structured storage for persistent data, and how DBMSs help manage, query, and secure that data. You also explored SQL as the standard language for relational databases and integrated SQLite with Python to perform essential database operations.

Summary

In this unit, you have learnt that databases centralize and structure data efficiently. Python's `sqlite3` module allows interaction with SQLite databases for creating tables, inserting, updating, deleting, and querying records. Using database keys ensures data integrity, while SQL operators, clauses, and aggregation functions facilitate powerful data retrieval and analysis.

Self-Assessment Questions

1. Explain the role of a primary key in a database.
2. Describe the steps to insert multiple records into a SQLite table using Python.
3. How do GROUP BY and aggregate functions help summarize data in SQL?

Unit Activities

1. Read the content of Module 5 Unit 1 in the study guide. Consult the recommended texts and research more information on the internet. (90 minutes).

Tutor-Marked Assignment

Design a Python program that:

- a. Creates a database for an e-commerce application with tables for users, products, and orders.
- b. Inserts sample records into each table.
- c. Updates at least one record and deletes one record.

- d. Retrieves all users who have made purchases over a certain amount and displays the results.
- e. Demonstrates the use of at least one aggregate function and one JOIN query.

References/Further Readings

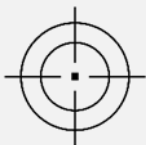
Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

Unit 2— Python GUI Programming

INTRODUCTION

In this unit, you will learn how to create interactive desktop applications in Python using Graphical User Interfaces (GUIs). You will explore Tkinter, the standard Python GUI toolkit, and learn how to design windows and widgets and handle user interactions.



LEARNING OUTCOMES

At the end of this unit, you should be able to:

- Understand the concept and purpose of GUIs.
- Use Tkinter to create and manage windows and widgets.
- Implement labels, buttons, entry fields, checkbuttons, radio-buttons, spinboxes, listboxes, scales, comboboxes, menus, and toolbars.
- Handle user input and events to control program flow.

5.2.1 Overview

Graphical User Interfaces (GUIs) make applications user-friendly by allowing interaction through windows, buttons, and other visual elements instead of text-based commands.

Python provides several libraries for creating GUIs, such as Tkinter, wxPython, PyQt, and GTK.

Among these, Tkinter is the standard Python GUI toolkit, built into Python and available on all major platforms (Windows, macOS, Linux). This unit introduces you to GUI programming in Python using Tkinter.

5.2.2 Tkinter Library

Tkinter provides a simple yet powerful object-oriented interface to build desktop applications. It contains a wide range of GUI elements called widgets, such as labels, buttons, and text boxes, which you can combine to create full-featured applications.

Example: Creating a Simple Tkinter Window

```
import tkinter as tk

root = tk.Tk()
root.title("My First Window")
root.geometry("300x200")
root.mainloop()
```

Explanation:

- Tk() creates the main application window.
- title() sets the window title.
- geometry() defines its size.
- mainloop() starts the GUI event loop, keeping the window open.

Tkinter Widgets

Widgets are building blocks of a GUI application. Each widget has properties and methods that define how it appears and behaves.

Common widgets include:

- **Label** – displays text or images.
- **Button** – triggers an action when clicked.

- **Entry** – accepts single-line text input.
- **Frame** – groups related widgets.
- **Checkbutton / Radiobutton** – allow selections.
- **Listbox / Combobox** – display multiple choices.

1. Window Widget

A window serves as the main container for all other widgets.

Every Tkinter application starts with a main window created using Tk().

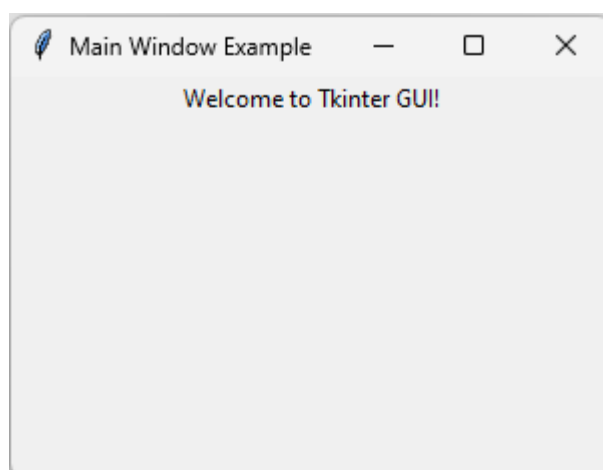
Code:

```
import tkinter as tk

window = tk.Tk()
window.title("Main Window Example")
window.geometry("300x200")

lbl = tk.Label(window, text="Welcome to
Tkinter GUI!")
lbl.pack()
window.mainloop()
```

Output:



A small window appears with a label "Welcome to Tkinter GUI!"

2. Label Widget

A label widget is used to display text or images that users do not interact with directly.

Code:

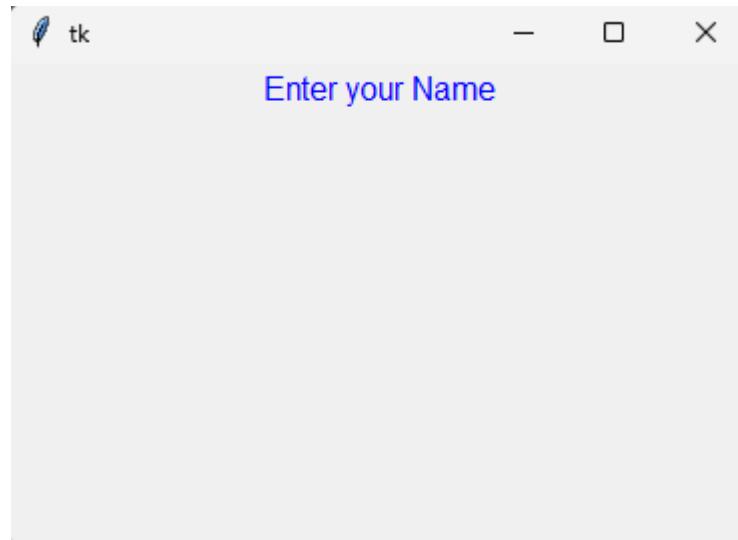
```
import tkinter as tk
```

```

window = tk.Tk()
label = tk.Label(window, text="Enter your Name",
fg="blue", font=("Arial", 12))
label.pack()
window.mainloop()

```

Output:



Explanation:

The Label() widget displays static content. You can change its color, font, or background.

3. Entry Widget

Entry Widget is used for **single-line user input**.

Code:

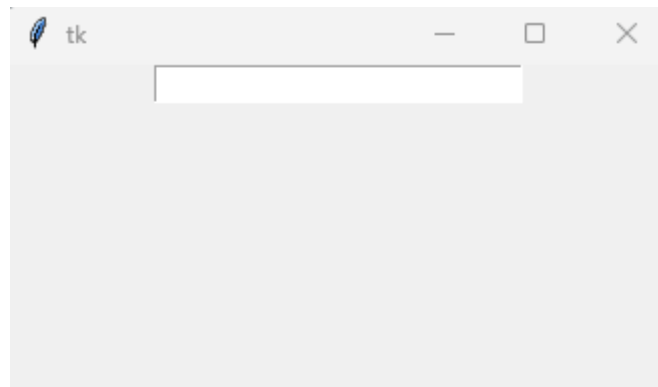
```

import tkinter as tk

window = tk.Tk()
entry = tk.Entry(window, width=30)
entry.pack()
window.mainloop()

```

Output:



Explanation:

The user can type text in this field. You can later retrieve it using `entry.get()`.

4. Button Widget

Button Widget creates an interactive button that triggers a function when clicked.

Code:

```
import tkinter as tk
from tkinter import messagebox

window = tk.Tk()

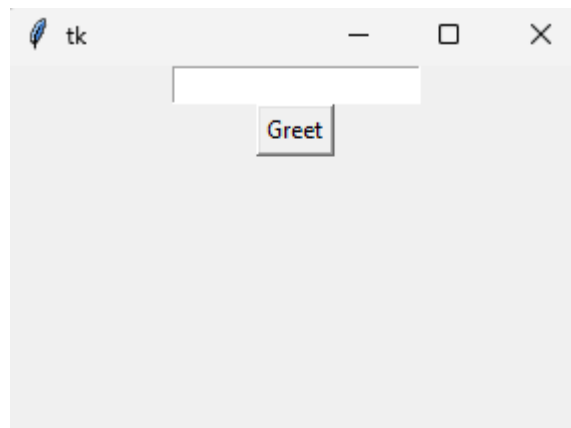
def show_message():
    messagebox.showinfo("Greeting", "Hello, " +
        entry.get())

entry = tk.Entry(window)
entry.pack()

btn = tk.Button(window, text="Greet", com-
    mand=show_message)
btn.pack()

window.mainloop()
```

Output:



Explanation:

The command option connects the button to a function that runs when clicked.

5. Frame Widget

Frames are containers for grouping widgets.

Code:

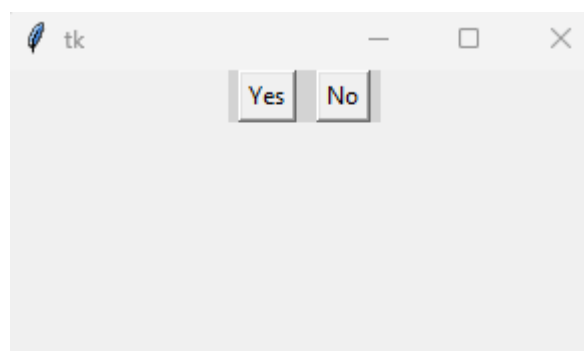
```
import tkinter as tk

window = tk.Tk()
frame = tk.Frame(window, bg="lightgray")
frame.pack()

tk.Button(frame, text="Yes").pack(side="left",
padx=5)
tk.Button(frame, text="No").pack(side="left",
padx=5)

window.mainloop()
```

Output:



Explanation:

Frames make layout management easier by grouping related elements.

6. Checkbutton Widget

The checkbutton widget is used when you want the user to select multiple options.

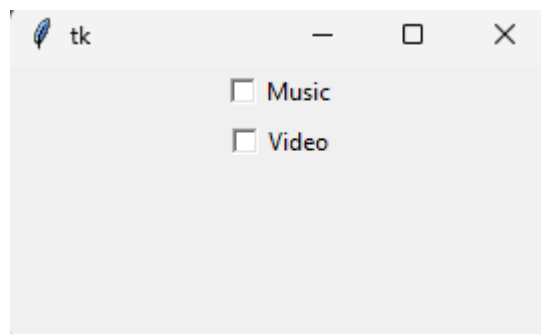
Code:

```
import tkinter as tk

window = tk.Tk()
music = tk.IntVar()
video = tk.IntVar()

tk.Checkbutton(window, text="Music", variable=music).pack()
tk.Checkbutton(window, text="Video", variable=video).pack()
window.mainloop()
```

Output:



Explanation:

Each checkbutton stores a value (1 or 0) in its linked variable.

7. Radiobutton Widget

Radiobutton Widget is used when the user must select one option from a set.

Code:

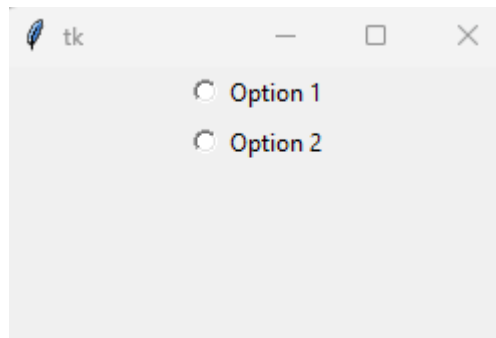
```
import tkinter as tk

window = tk.Tk()
choice = tk.IntVar()

def show_choice():
    lbl.config(text=f"You      selected      Option {choice.get()}")
```

```
tk.Radiobutton(window, text="Option 1", variable=choice, value=1, command=show_choice).pack()
tk.Radiobutton(window, text="Option 2", variable=choice, value=2, command=show_choice).pack()
lbl = tk.Label(window)
lbl.pack()
window.mainloop()
```

Output:



Explanation:

This program creates two radio buttons that let the user select only one option at a time. The selected option's value (1 or 2) is stored in the variable choice, and the label updates dynamically to display the chosen option when a button is clicked.

8. Spinbox Widget

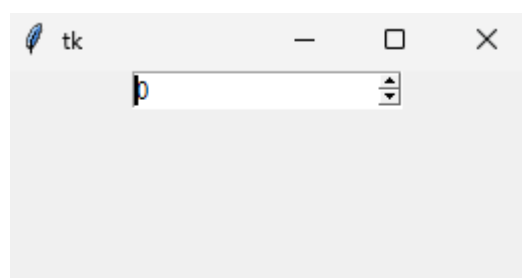
The spinbox widget allows selection from a range of numbers.

Code:

```
import tkinter as tk

window = tk.Tk()
spin = tk.Spinbox(window, from_=0, to=10)
spin.pack()
window.mainloop()
```

Output:



Explanation:

The Spinbox provides a convenient way to select numeric values.

9. MessageBox Widget

MessageBox widget displays popup messages for alerts or feedback.

Code:

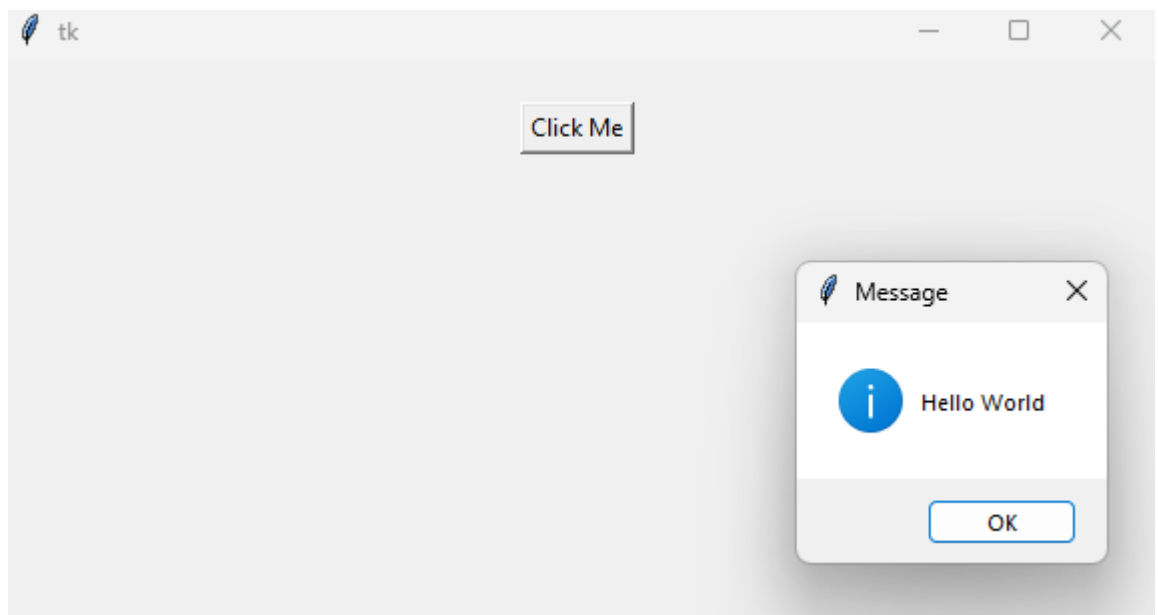
```
from tkinter import messagebox, Tk, Button

window = Tk()

def greet():
    messagebox.showinfo("Message", "Hello World")

Button(window, text="Click Me", command=greet).pack(pady=20)
window.mainloop()
```

Output:



Explanation:

The messagebox module provides prebuilt dialog boxes such as showinfo, showwarning, and showerror.

10. Listbox Widget

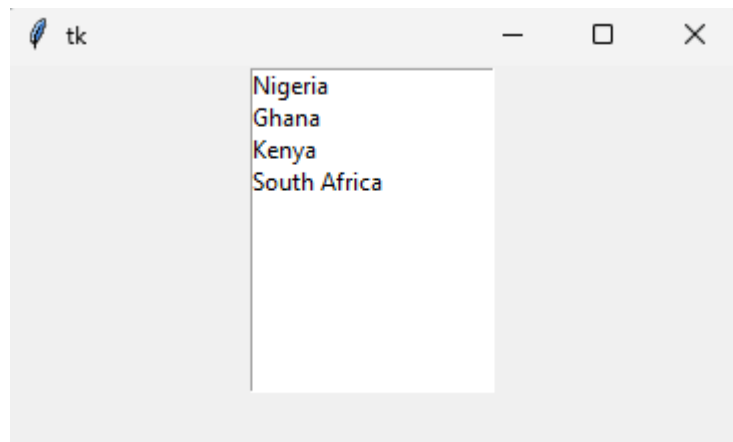
The listbox widget displays a scrollable list of items for user selection.

Code:

```
import tkinter as tk

window = tk.Tk()
countries = ["Nigeria", "Ghana", "Kenya", "South Africa"]
lst = tk.Listbox(window)
for country in countries:
    lst.insert(tk.END, country)
lst.pack()
window.mainloop()
```

Output:



Explanation:

This program creates a listbox displaying a list of countries. Each country is inserted as an item in the list using a loop. The user can scroll through and select any item from the list when the program runs.

11.Scale Widget

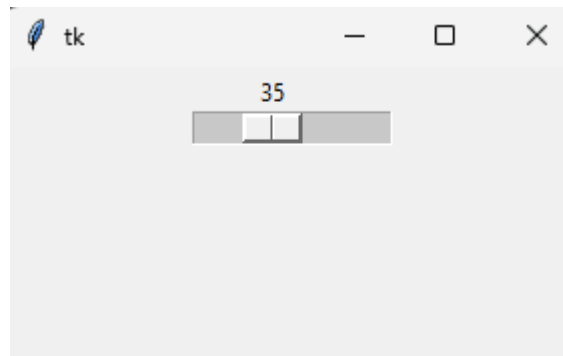
The scale widget provides a slider control to choose a value.

Code:

```
import tkinter as tk

window = tk.Tk()
scale = tk.Scale(window, from_=0, to=100, orient="horizontal")
scale.pack()
window.mainloop()
```

Output:



Explanation:

This program creates a horizontal slider that allows the user to select a numeric value between 0 and 100. The `from_` and `to` parameters define the range, while `orient="horizontal"` sets the slider's orientation.

12.Combobox Widget

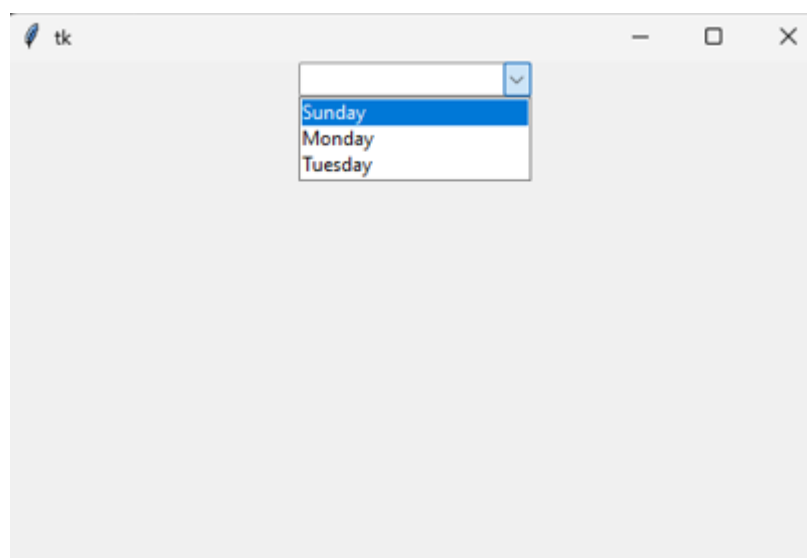
A combobox widget is a dropdown list provided by the `ttk` module.

Code:

```
import tkinter as tk
from tkinter import ttk

window = tk.Tk()
days = ("Sunday", "Monday", "Tuesday")
combo = ttk.Combobox(window, values=days)
combo.pack()
window.mainloop()
```

Output:



Explanation:

This program creates a combo box (a dropdown list) using the ttk module. It displays a list of days from which the user can select one. The values parameter defines the available options, and the selected value can later be retrieved or used in the program.

13.Menu and Submenu Widgets

Menus provide structured navigation options.

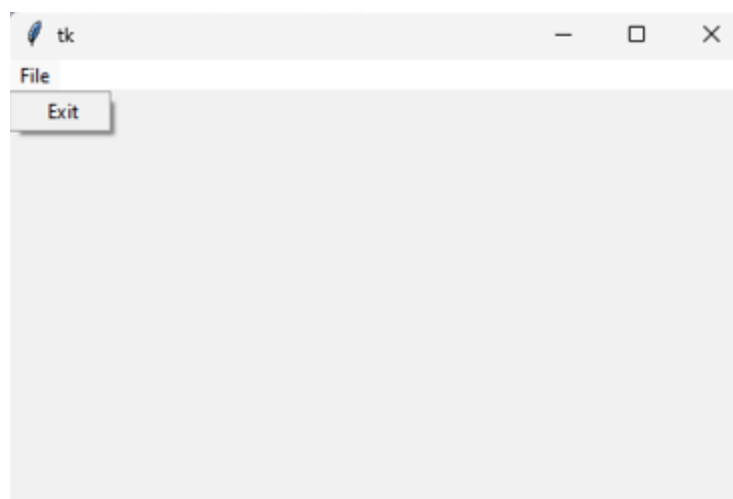
Code:

```
import tkinter as tk

def on_exit():
    root.quit()

root = tk.Tk()
menubar = tk.Menu(root)
filemenu = tk.Menu(menubar, tearoff=0)
filemenu.add_command(label="Exit", command=on_exit)
menubar.add_cascade(label="File", menu=filemenu)
root.config(menu=menubar)
root.mainloop()
```

Output:



Explanation:

Menus help organise options like File, Edit, or Help.

Submenus can be nested within menus using add_cascade().

14. Toolbar Widget

A toolbar provides quick-access buttons, typically arranged horizontally.

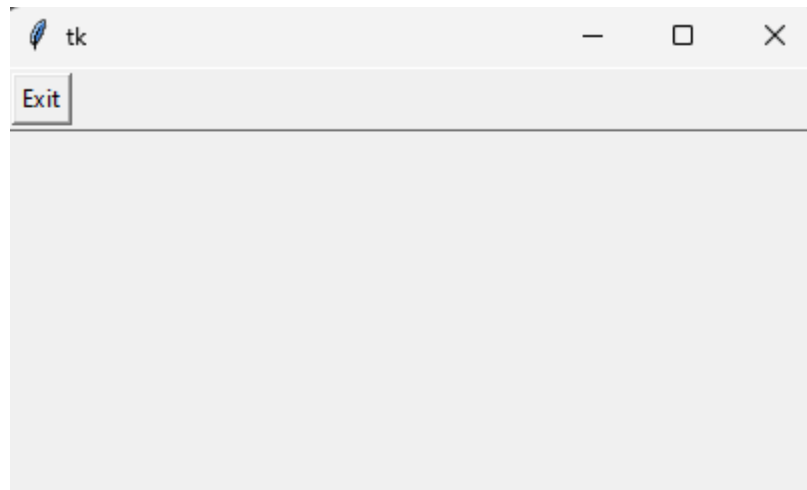
Code:

```
from tkinter import Tk, Frame, Button, LEFT, TOP, X

window = Tk()
toolbar = Frame(window, bd=1, relief="raised")

Button(toolbar, text="Exit", command=window.quit).pack(side=LEFT, padx=2, pady=2)
toolbar.pack(side=TOP, fill=X)
window.mainloop()
```

Output:



Explanation:

Toolbars enhance usability by offering shortcut buttons for frequent actions.

Tkinter remains one of the easiest and most versatile ways to develop Python desktop applications.

Graphical user interfaces (GUIs) make Python programs more interactive and user-friendly by allowing users to engage through visual components such as buttons, labels, entry boxes, and menus. Using the Tkinter library, developers can design windows and widgets that collect input, display information, and control program flow.

In-Text Questions (ITQs)

1. What is the role of a GUI in a Python application?
2. Name three common Tkinter widgets and their uses.
3. How do you connect a button to a function in Tkinter?

In-Text Answers (ITAs)

1. A GUI provides a visual interface for users to interact with the program instead of using text commands.
2. Examples: Label (display text), Entry (accept input), Button (trigger actions).
3. Using the command parameter of the Button widget.

5.2.3 Conclusion

In this unit, you have learned how GUIs enhance user interaction and usability in Python applications. You explored Tkinter and practised creating windows, adding widgets, and handling user inputs and events.

Summary

In this unit, you have learned that Tkinter allows developers to design and manage GUI applications easily. Widgets such as labels, buttons, entries, checkbuttons, radiobuttons, spinboxes, listboxes, scales, comboboxes, menus, and toolbars provide interactive elements. Event handling links user actions to functions, enabling dynamic program behaviour.

Self-Assessment Questions

1. Explain the difference between a Label and an Entry widget.
2. How would you use a Frame to organise widgets?
3. Describe how to create a dropdown menu using Tkinter.

Unit Activities

1. Create a Tkinter window with multiple widgets (labels, buttons, entries).
2. Implement checkbuttons and radiobuttons to capture user selections.
3. Add a menu bar and toolbar with functional commands.
4. Practice retrieving and displaying user input from Entry and Combobox widgets.

Tutor-Marked Assignment

Design a Python program that:

- a. Creates a main window with a title and a defined size.
- b. Contains labels, entry fields, buttons, checkbuttons, radiobuttons, and a spinbox.
- c. Implements a menu bar with at least one submenu.
- d. Includes a toolbar with shortcut buttons.
- e. Collects user input, processes it, and displays the output dynamically.

References/Further Readings

Gaddis, T. (2018). Starting out with Python (4th ed.). Pearson.

Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). O'Reilly Media

ANSWERS TO SELF ASSESSMENT QUESTIONS

Module One

6.1.1 Unit 1: Answers to Self-Assessment Questions

Q1: What are the primary characteristics of Python that make it a preferred choice for beginners?

Python is widely preferred because it uses a clear and readable syntax that resembles natural language. It also provides extensive libraries and a supportive community, making it easier for beginners to learn without feeling overwhelmed.

Q2: Name three popular web frameworks in Python and their applications

Django is used for full scale web platforms, Flask supports small to medium applications, and FastAPI is useful for building high performance application programming interfaces.

Q3: How is Python used in data science and machine learning?

Python supports data science tasks through libraries such as NumPy and Pandas for data manipulation, while Scikit-learn and TensorFlow help build and train machine learning models for prediction and analysis.

6.1.2 Unit 2: Answers to Self-Assessment Questions

Q1: What is the recommended version of Python for current development?

Python 3 is recommended because it contains updated features, modern libraries, and improved performance that support current programming needs.

Q2: How can you check if Python is installed on your Windows system?

You can open the command prompt and type `python --version` to confirm whether Python is already installed on the system.

Q3: What command is used to install Jupyter Notebook via the command line?

The installation is done by entering `pip install notebook` in the command prompt.

Q4: What is the purpose of writing the Hello World program after Python installation?

This short program is used to confirm that Python is functioning properly and that the installation and environment setup were completed successfully.

Module Two

6.2.1 Unit 1: Answers to Self-Assessment Questions

Q1. What are the two primary modes of writing and executing Python programs?

Interactive mode allows commands to run immediately for testing ideas, while script mode executes a saved file as a complete program.

Q2. How can you run a Python script saved as `example.py` from the command line?

You can navigate to its folder in the command prompt and type `python example.py` to run the script.

Q3. What rules guide the creation of valid Python identifiers?

Identifiers must begin with a letter or an underscore, avoid starting with digits, and must not use reserved words. These rules help the interpreter read variable names correctly.

Q4. What happens when a Python code block is not properly indented?

The interpreter produces an indentation error because indentation is how Python recognises which statements belong to a particular block of code.

6.2.2 Unit 2: Answers to Self-Assessment Questions

Q1. What is a variable in Python and how is it created?

A variable stores a value in memory and is created by assigning a value to a name, such as `age = 20`. It helps the program retain information for later use.

Q2. What does the assignment operator (=) do in Python?

The operator connects a value to a variable name, allowing the program to refer to the value through that name.

Q3. What is dynamic typing in Python?

Dynamic typing means the interpreter automatically determines a variable's data type based on the value it receives, allowing flexible and adaptable code.

Q4. How can you check the data type of a value in Python?

You can use the `type()` function, which displays the precise data type of the value being examined.

6.2.3 Unit 3: Answers to Self-Assessment Questions

Q1. Differentiate between the assignment operator = and the comparison operator ==.

The assignment operator places a value into a variable, while the comparison operator checks whether two values are equal.

Q2. List five Python operators and classify them.

The plus symbol is an arithmetic operator, the equality symbol is a comparison operator, the word and is a logical operator, the word is is an identity operator, and the equal sign is an assignment operator.

Q3. Explain with an example how operator precedence affects an expression.

Python evaluates operators in a specific order. For example, in $3 + 4 * 2$, multiplication is processed first, leading to the result 11 instead of 14.

Q4. What is the purpose of using parentheses in a complex expression? Provide a code example.

Parentheses adjust the natural order of evaluation, such as $(3 + 4) * 2$, which forces addition to occur before multiplication.

Q5. Why are named constants recommended in programming? Give two examples of how they are used.

Named constants improve clarity and ensure important values are not changed by mistake. For instance, $PI = 3.14$ and $MAX_STUDENTS = 50$ make programs easier to understand.

Module Three

6.3.1 Unit 1: Answers to Self-Assessment Questions

Q1. Differentiate between sequence and decision structures in Python and give one example of each.

A sequence structure executes commands in order, such as printing two lines consecutively, while a decision structure evaluates conditions before choosing an action, such as using an if statement to check a value.

Q2. Write an if statement that checks if a number is positive and prints Positive number if true.

```
if num > 0:
    print("Positive number")
```

Q3. Explain how an if else statement works with an example

An if else structure chooses between two alternatives depending on a condition.

```
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Q4. Describe the use of if elif else statements and write a program that prints grades.

This structure evaluates several conditions in order until one is true.

```
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
```

```
        print("D")
    else:
        print("F")
```

6.3.2 Unit 2: Answers to Self-Assessment Questions

Q1. With the aid of an example, differentiate between while and for loops.

A while loop continues as long as a condition remains true, while a for loop runs through a sequence.

while x < 5: increases a variable gradually, while for i in range(5): repeats a set number of times.

Q2. What is the purpose of a nested loop?

A nested loop allows one loop to run inside another, making it useful for working with tables, matrices, or repeated patterns.

Q3. How does a sentinel-controlled loop differ from a fixed loop?

A sentinel loop stops when a special value is encountered, while a fixed loop has a known number of repetitions.

6.3.3 Unit 3: Answers to Self-Assessment Questions

Q1. Differentiate between a list and a tuple.

A list can be changed after creation, while a tuple remains unchanged. This makes tuples suitable for constant data.

Q2. Write a Python program to reverse a list.

```
data = [1, 2, 3]
data.reverse()
```

Q3. Explain the immutability of strings and tuples with examples.

Once created, their values cannot be altered. For instance, modifying a string creates a new one, and tuples do not permit item reassignment.

Q4. List five string methods that modify or inspect strings

upper, lower, strip, replace, and split.

- Upper: Converts all characters in a string to uppercase, which is useful when standardising text for comparison or display.
- Lower: Converts all characters in a string to lowercase, often used to make user input uniform before processing.
- Strip: Removes leading and trailing whitespace, helping clean input data before storage or analysis.
- Replace: Substitutes one substring with another, allowing modification of text without altering the original structure.
- Split: Breaks a string into a list of substrings based on a delimiter, making it easier to process sentences or structured text.

6.3.4 Unit 4: Answers to Self-Assessment Questions

Q1. Explain how a set differs from a list in Python.

A set stores unique items without a specific order, while a list keeps items in the order they are added and may contain duplicates.

Q2. How would you remove a key-value pair from a dictionary?

You can delete it using `del dictionary[key]` or retrieve and remove it using `dictionary.pop(key)`.

Q3. What will be the output of `set([1,2,2,3])`?

The result contains only unique values, giving `{1, 2, 3}`.

Q4. Write a Python loop to iterate over a dictionary and print its keys and values

```
for key, value in data.items():  
    print(key, value)
```

Module Four

6.4.1 Unit 1: Answers to Self-Assessment Questions

Q1. Explain the difference between local and global variables.

Local variables exist only inside a function and disappear when it finishes, while global variables remain available to the entire program.

Q2. Write a void function that prints a greeting message.

```
def greet():  
    print("Hello")
```

Q3. Create a value-returning function that calculates the cube of a number

```
def cube(n):  
    return n ** 3
```

Q4. Demonstrate how to generate a random integer between 1 and 50

```
import random  
random.randint(1, 50)
```

6.4.2 Unit 2: Answers to Self-Assessment Questions

Q1 What is a Python module and why is it useful?

A module is a file that contains reusable code. It encourages better organisation and prevents duplication.

Q2. What is the emotional impact of cybercrime on an individual?

You can import the module using `import math` and call its functions as in `math.sqrt(25)`.

Q3. Explain the role of the init file in a package

This file helps Python recognise a folder as a package by enabling it to import modules from the folder.

Q4. List three commonly used functions from the math module.

`sqrt`, `ceil`, and `floor`.

- `Sqrt`: Returns the square root of a number, which is useful for mathematical computations such as distance or variance calculations.
- `Ceil`: Rounds a number upward to the nearest whole number, helping ensure values meet minimum required thresholds.
- `Floor`: Rounds a number downward to the nearest whole number, often used when estimating counts or simplifying numeric results.

Q5. How can you generate a random integer between 1 and 100 using the random module?

```
import random
random.randint(1, 100)
```

6.4.3 Unit 3: Answers to Self-Assessment Questions

Q1. Why is file handling important in programming?

It allows programs to store, retrieve, and update information, which is essential for persistent data.

Q2. Explain the difference between sequential and direct access.

Sequential access reads data in order, while direct access allows the program to jump to a particular part of the file.

Q3. What file modes are commonly used in Python and what do they do?

Read mode opens a file for reading, write mode replaces content, and append mode adds new content at the end.

Q4. Read mode opens a file for reading, write mode replaces content, and append mode adds new content at the end

You convert it to a string using the str function before writing.

Q5. How can you skip lines starting with the number sign when processing a file?

Check whether a line begins with the character and ignore it before processing.

6.4.4 Unit 4: Answers to Self-Assessment Questions

Q1. Define an exception and explain its significance in programming.

An exception is an error that arises during execution. Handling it prevents the program from stopping unexpectedly

Q2 List at least three common runtime exceptions in Python.

Zero division error, index error, and type error.

- `ZeroDivisionError`: Occurs when a program attempts to divide a number by zero, which is mathematically undefined.
- `IndexError`: Happens when you try to access a list or sequence position that does not exist.
- `TypeError`: Raised when an operation is applied to an object of an inappropriate type, such as adding a string to a number.

Q3. How does the try except block work?

The program attempts to run the code in the try section and moves to the except section only when an error occurs.

Q4. How can exceptions be handled within a function?

You can surround risky statements with a try except block inside the function to prevent it from failing.

Q5. Demonstrate how to raise a value error when invalid input is detected.

```
if age < 0:
    raise ValueError("Invalid age entered")
```

Module Five

6.5.1 Unit 1: Answers to Self-Assessment Questions

Q1. Explain the role of a primary key in a database.

A primary key uniquely identifies each record in a table, ensuring accurate data retrieval and preventing duplication.

Q2. Describe the steps to insert multiple records into a SQLite table using Python.

You connect to the database, create a cursor, prepare an insert statement, use the executemany function to add several records, then save the changes.

Q3. How do group by and aggregate functions help summarise data in SQL?

They organise rows with similar values and compute summaries such as totals, averages, or counts to support data analysis.

6.5.2 Unit 2: Answers to Self-Assessment Questions

Q1. Explain the difference between a label and an entry widget.

A label displays non-editable text, while an entry widget allows the user to type information into the application.

Q2. How would you use a frame to organise widgets?

A frame acts as a container where widgets can be grouped and arranged neatly to improve layout structure.

Q3. Describe how to create a dropdown menu using Tkinter.

You can use an option menu or a combobox and supply a list of items from which the user can make a selection.

GLOSSARY

Argument: A value passed into a function when it is called, which the function can use in its operations.

Arithmetic Operators: Symbols such as +, -, *, /, //, %, and ** used to perform mathematical calculations.

Append(): A list method that adds a new element to the end of a list.

Assignment Statement: A statement that gives a value to a variable using the = operator.

Block: A group of related statements indented together under a control structure to form a logical unit of code.

Boolean: A data type representing True or False values, commonly used in decision-making.

Boolean Expression: An expression that evaluates to either True or False based on conditions or comparisons.

Break Statement: A command used to exit a loop before it completes all iterations.

CamelCase: A naming convention where each word starts with a capital letter, often used for class names.

Case-Sensitive: The property of Python treating uppercase and lowercase letters as distinct in identifiers.

Class: A blueprint for creating objects that encapsulate data (attributes) and behaviors (methods).

Colon (:): A punctuation mark used in Python to indicate the beginning of an indented code block after control statements.

Comment: Text included in code to explain its function, ignored by Python during execution (# for single-line, triple quotes for multi-line).

Comparison/Relational Operators: Symbols like ==, !=, >, <, >=, <= used to compare values and return a Boolean result.

Condition: A logical statement evaluated to guide the execution of decision or loop structures.

Condition-Controlled Loop: A loop that executes repeatedly as long as a specified condition remains True.

Concatenation: The process of joining sequences, such as strings or lists, using the + operator.

Continue Statement: A command used to skip the rest of the current loop iteration and proceed to the next iteration.

Data Type: Classification of values that determines the operations that can be performed on them, e.g., int, float, str.

Del Statement: Removes an element from a list or dictionary using its index or key.

Dictionary: An unordered collection of key-value pairs enclosed in {}, where each key is unique.

Exception: An error that occurs during program execution and interrupts normal flow.

Exception Handling: The mechanism of using try and except blocks to manage runtime errors gracefully.

Expression: A combination of values, variables, and operators that produces a result.

f-string: A formatted string literal prefixed with f, allowing expressions to be embedded within strings, e.g., f"My age is {age}".

Find(): A string method that returns the index of the first occurrence of a substring.

Float: A numeric type representing numbers with decimal points.

Floor Division (//): Division operation that returns only the integer quotient without the remainder.

For Loop: A loop that iterates over elements of a sequence or range for a specified number of times.

Format Specifiers: Symbols used to control how values are presented in output, e.g., .2f, :05d, :b, :x.

Function: A reusable block of code defined using def that performs a specific task.

Function Call: The statement that executes a function by its name and provides required arguments.

Global Variable: A variable defined outside any function that can be accessed throughout the program.

If Clause: The part of an if statement that specifies a condition to evaluate.

If-Else Statement: A decision structure that executes one block if a condition is True and another if it is False.

If-Elif-Else Statement: A multi-path decision structure allowing several conditions to be tested sequentially, executing the first true block or the else block.

Immutable: Data types whose contents cannot be changed after creation, such as strings and tuples.

Import Statement: Command used to include modules or specific functions from modules into a program.

Indentation: The leading spaces at the beginning of lines that define the structure of code blocks in Python.

Input Function (input()): Reads user input from the keyboard and returns it as a string.

Integer (int): Numeric type representing whole numbers.

Insert(): A list method that adds an element at a specified index.

Intersection: A set operation returning elements common to two sets.

Iteration: The repeated execution of a block of code, often over elements of a sequence.

isalpha(): Returns True if all characters in a string are letters.

isalnum(): Returns True if all characters in a string are letters or digits.

isdigit(): Returns True if all characters in a string are numeric digits.

islower(): Returns True if all alphabetic characters in a string are lowercase.

isupper(): Returns True if all alphabetic characters in a string are uppercase.

isspace(): Returns True if all characters in a string are whitespace.

Keyword: A reserved word in Python that cannot be used as an identifier because it has a predefined meaning.

Lambda Function: An anonymous, single-line function defined using the lambda keyword. Example: `square = lambda x: x**2` creates a function that returns the square of x.

List: A mutable, ordered collection of items enclosed in square brackets [].

List Methods: Functions used to perform operations on lists, such as `append()`, `insert()`, `remove()`, `pop()`, `sort()`, `reverse()`.

Lower(): Converts all letters in a string to lowercase.

Module: A Python file containing functions, classes, and variables that can be imported into other programs.

Nested Loop: A loop placed inside another loop, where the inner loop runs completely for each iteration of the outer loop.

Not in Operator: Checks whether an element does not exist in a sequence, set, or dictionary keys.

Object-Oriented Programming (OOP): A programming paradigm that organises code using classes and objects to

encapsulate data (attributes) and behaviours (methods).

Operand: A value on which an operator performs an operation.

Operator: Symbol used to perform an operation on one or more values (e.g., +, -, *, /).

Operator Precedence: Rules determining the order in which operators in an expression are evaluated.

Parameter: A variable listed in a function definition that receives the value of an argument when the function is called.

Pass Statement: A placeholder statement used where syntactic requirements exist but no action is needed.

Pop(): Removes and returns an element from a list or dictionary.

Print Function (print()): Displays information or results to the user, including text and variable values.

Range: A sequence of numbers, commonly used to control the iterations of a for loop.

Reassignment: Assigning a new value to an existing variable.

Return Statement: Sends a value back from a function to the caller.

Set: An unordered collection of unique elements.

Slice / Slicing: Extracting a portion of a sequence using the [start:end] syntax.

Startswith(): Returns True if a string begins with a specified substring.

Statement: A single instruction or line of Python code that performs an action.

String (str): An immutable sequence of characters enclosed in quotes.

String Methods: Functions performing operations on strings, including `upper()`, `lower()`, `strip()`, `replace()`, `find()`, `startswith()`, `endswith()`, `split()`.

Symmetric Difference: Set operation returning elements present in either of two sets but not in both.

Tuple: An immutable, ordered collection of items enclosed in parentheses `()`.

Type Conversion / Casting: Converting a value from one data type to another, such as using `int()`, `float()`, or `str()`.

Upper(): Converts all letters in a string to uppercase.

Value: The actual data associated with a variable or object.

Variable: A name that references data stored in memory, which can be manipulated in a program.

Variable Naming Rules: Guidelines for valid identifiers: start with a letter or underscore, contain letters, digits, or underscores, no spaces, and avoid keywords.

While Loop: A loop that continues to execute as long as a specified condition is `True`.

Whitespace: Spaces or tabs that are significant in Python for code structure and readability.

